LaTeX 2ε Interfaces

The LaTeX Project*

Released 2026-01-23

Abstract

This document contains user (and programmer) documentation for much of the "new" code added to the LaTeX kernel in recent years. Over time, this coverage is expected to be expanded to include the majority of user commands defined in `latex.ltx`.

The individual parts are written at different times and were originally meant to stand on their own. This means that explanations are not necessary in the most logical order, and there is some duplication. Headings are also somewhat inconsistent. We expect this to be sorted out over time, as material is revised to form a consistent whole.

*E-mail: latex-team@latex-project.org

Contents

I Document commands **1**

0.1 Creating document commands . . . 1
0.1.1 Overview . . . 1
0.1.2 Describing argument types . . . 2
0.1.3 Modifying argument descriptions . . . 3
0.1.4 Creating document commands and environments . . . 3
0.1.5 Optional arguments . . . 4
0.1.6 Spacing and optional arguments . . . 5
0.1.7 'Embellishments' . . . 6
0.1.8 Testing special values . . . 6
0.1.9 Auto-converting to key–value format . . . 8
0.1.10 Argument processors . . . 9
0.1.11 Body of an environment . . . 11
0.1.12 Fully-expandable document commands . . . 12
0.1.13 Commands at the start of tabular cells . . . 12
0.1.14 Using the verbatim argument types . . . 13
0.1.15 Typesetting verbatim-like material . . . 13
0.1.16 Verbatim environments . . . 13
0.1.17 Performance . . . 14
0.1.18 Details about argument delimiters . . . 14
Character tokens . . . 15
Control sequence tokens . . . 15
0.1.19 Creating new argument processors . . . 15

II Hooks **17**

1 LaTeX's hook management **18**

1.1 Introduction . . . 18
1.2 Package writer interface . . . 18
1.2.1 LaTeX 2ε interfaces . . . 18
Declaring hooks . . . 18
Special declarations for generic hooks . . . 19
Using hooks in code . . . 20
Updating code for hooks . . . 21
Hook names and default labels . . . 24
The `top-level` label . . . 26
Defining relations between hook code . . . 27
Querying hooks . . . 29
Displaying hook code . . . 30
Debugging hook code . . . 31
1.2.2 L3 programming layer (`expl3`) interfaces . . . 31
1.2.3 On the order of hook code execution . . . 34
1.2.4 The use of "reversed" hooks . . . 36
1.2.5 Difference between "normal" and "one-time" hooks . . . 37
1.2.6 Generic hooks provided by packages . . . 37
1.2.7 Hooks with arguments . . . 38

1.2.8 Private LaTeX kernel hooks . . . 40
1.2.9 Legacy LaTeX 2ε interfaces . . . 40
1.3 LaTeX 2ε commands and environments augmented by hooks . . . 41
1.3.1 Generic hooks . . . 41
Generic hooks for all environments . . . 42
Generic hooks for commands . . . 43
Generic hooks provided by file loading operations . . . 43
1.3.2 Hooks provided by `\begin{document}` . . . 44
1.3.3 Hooks provided by `\end{document}` . . . 44
1.3.4 Hooks provided by `\shipout` operations . . . 46
1.3.5 Hooks provided for paragraphs . . . 46
1.3.6 Hooks provided in NFSS commands . . . 46
1.3.7 Hook provided by the mark mechanism . . . 47

2 LaTeX's hook management for files 48
2.1 Introduction . . . 48
2.1.1 Provided hooks . . . 48
2.1.2 General hooks for file reading . . . 48
2.1.3 Hooks for package and class files . . . 49
2.1.4 Hooks for `\include` files . . . 50
2.1.5 High-level interfaces for LaTeX . . . 51
2.1.6 Kernel, class, and package interfaces for LaTeX . . . 52
2.1.7 A sample package for structuring the log output . . . 52

3 Hook management for commands 54
3.1 Introduction . . . 54
3.2 Restrictions and operational details . . . 55
3.2.1 Patching . . . 56
Timing . . . 56
3.2.2 Command copies . . . 56
3.2.3 Grouping . . . 57
3.2.4 Commands that look ahead . . . 57
3.3 Package author interface . . . 57
3.3.1 Arguments and redefining commands . . . 58

4 Paragraph building and hooks 60
4.1 Introduction . . . 60
4.1.1 The default processing done by the engine . . . 60
4.2 The new mechanism implemented for LaTeX . . . 62
4.2.1 The provided hooks . . . 63
4.2.2 Altered and newly provided commands . . . 64
4.2.3 Examples . . . 65
Testing the mechanism . . . 65
Mark the first paragraph of each `itemize` . . . 67
4.2.4 Some technical notes . . . 67
Glue items between paragraphs (found with fancypar) . . . 67

5 The shipout routine: hooks and interfaces **69**
5.1 Introduction . . . 69
5.1.1 Overloading the `\shipout` primitive . . . 69
5.1.2 Provided hooks . . . 71
5.1.3 Legacy LaTeX commands . . . 72
5.1.4 Special commands for use inside the hooks . . . 73
5.1.5 Provided LuaTeX callbacks . . . 73
5.1.6 Information counters . . . 74
5.1.7 Debugging shipout code . . . 74
5.2 Emulating commands from other packages . . . 75
5.2.1 Emulating atbegshi . . . 75
5.2.2 Emulating everyshi . . . 76
5.2.3 Emulating atenddvi . . . 76
5.2.4 Emulating everypage . . . 76

III Run data and page design **77**

6 The marks mechanism **78**
6.1 Introduction . . . 78
6.2 Design-level and code-level interfaces . . . 79
6.2.1 Use cases for conditionals . . . 81
6.2.2 Understanding regions . . . 81
6.2.3 Debugging mark code . . . 83
6.3 Application examples . . . 83
6.4 Legacy LaTeX 2_ε interface . . . 83
6.4.1 Legacy design-level and document-level interfaces . . . 84
6.4.2 Legacy interface extensions . . . 84
6.5 Notes on the mechanism . . . 85
6.6 Public interfaces for packages such as multicol . . . 86
6.7 Internal functions for the standard output routine of LaTeX . . . 87

7 Recording and cross-referencing document properties **88**
7.1 Introduction . . . 88
7.2 Design discussion . . . 88
7.3 Handling unknown labels and properties . . . 89
7.4 Rerun messages . . . 89
7.5 Open points . . . 89
7.6 Code interfaces . . . 90
7.7 Auxiliary file interfaces . . . 92
7.8 LaTeX 2_ε interface . . . 92
7.9 Pre-declared properties . . . 93

IV Design-level tools **95**

8 LaTeX's socket management **96**
8.1 Introduction . . . 96
8.2 Configuration of the transformation process . . . 96
8.2.1 The template mechanism . . . 96
8.2.2 The hook mechanism . . . 97
8.2.3 The socket mechanism . . . 98
Examples . . . 99
Details and semantics . . . 100
8.2.4 Socket and plug names . . . 102
Command syntax . . . 102
Rationale for error handling . . . 104

9 Templates: Prototype document functions **105**
9.1 Introduction . . . 105
9.2 What is a document? . . . 106
9.3 Types, templates, and instances . . . 106
9.4 Template types . . . 106
9.5 Templates . . . 107
9.6 Multiple choices . . . 111
9.7 Instances . . . 112
9.8 Document interface . . . 113
9.9 Changing existing definitions . . . 114
9.9.1 Expanding the values of keys . . . 114
9.10 Creating equivalent keys . . . 115
9.10.1 Modifying existing key values . . . 115
9.11 Getting information about templates and instances . . . 116
9.12 Debugging support . . . 116

Index **117**

Part I
Document commands

0.1 Creating document commands

0.1.1 Overview

Creating document commands and environments using the LaTeX3 toolset is based around the idea that a common set of descriptions can be used to cover almost all argument types used in real documents. Thus parsing is reduced to a simple description of which arguments a command takes: this description provides the "glue" between the document syntax and the implementation of the command.

First, we will describe the argument types, then move on to explain how these can be used to create both document commands and environments. Various more specialized features are then described, which allow an even richer application of a simple interface set up.

The details here are intended to help users create document commands in general. More technical detail, suitable for TeX programmers, is included in `interface3`.

0.1.2 Describing argument types

In order to allow each argument to be defined independently, the parser does not simply need to know the number of arguments for a function, but also the nature of each one. This is done by constructing an *argument specification*, which defines the number of arguments, the type of each argument and any additional information needed for the parser to read the user input and properly pass it through to internal functions.

The basic form of the argument specifier is a list of letters, where each letter defines a type of argument. As will be described below, some of the types need additional information, such as default values. The argument types can be divided into two, those which define arguments that are mandatory (potentially raising an error if not found) and those which define optional arguments. The mandatory types

`m` A standard mandatory argument, which can either be a single token alone or multiple tokens surrounded by curly braces `{}`. Regardless of the input, the argument will be passed to the internal code without the outer braces. This is the type specifier for a normal TEX argument.

`r` Given as `r`⟨*token1*⟩⟨*token2*⟩, this denotes a "required" delimited argument, where the delimiters are ⟨*token1*⟩ and ⟨*token2*⟩. If the opening delimiter ⟨*token1*⟩ is missing, the default marker `\NoValue` will be inserted after a suitable error.

`R` Given as `R`⟨*token1*⟩⟨*token2*⟩`{`⟨*default*⟩`}`, this is a "required" delimited argument as for `r`, but it has a user-definable recovery ⟨*default*⟩ instead of `\NoValue`.

`v` Reads an argument "verbatim", between the following character and its next occurrence, in a way similar to the argument of the LATEX 2ε command `\verb`. Thus a `v`-type argument is read between two identical characters, which cannot be any of `%`, `\`, `#`, `{`, `}` or ␣. The verbatim argument can also be enclosed between braces, `{` and `}`. A command with a verbatim argument will produce an error when it appears within an argument of another command.

`b` Only suitable in the argument specification of an environment, it denotes the body of the environment, between `\begin{`⟨*environment*⟩`}` and `\end{`⟨*environment*⟩`}`. See Section 0.1.11 for details.

`c` Only suitable in the argument specification of an environment, it denotes collection of the environment verbatim, between `\begin{`⟨*environment*⟩`}` and `\end{`⟨*environment*⟩`}`. See Section 0.1.16 for details.

The types which define optional arguments are:

`o` A standard LATEX optional argument, surrounded with square brackets, which will supply the special `\NoValue` marker if not given (as described later).

`d` Given as `d`⟨*token1*⟩⟨*token2*⟩, an optional argument which is delimited by ⟨*token1*⟩ and ⟨*token2*⟩. As with `o`, if no value is given the special marker `\NoValue` is returned.

`O` Given as `O{`⟨*default*⟩`}`, is like `o`, but returns ⟨*default*⟩ if no value is given.

`D` Given as `D`⟨*token1*⟩⟨*token2*⟩`{`⟨*default*⟩`}`, it is as for `d`, but returns ⟨*default*⟩ if no value is given. Internally, the `o`, `d` and `O` types are short-cuts to an appropriated-constructed `D` type argument.

`s` An optional star, which will result in a value `\BooleanTrue` if a star is present and `\BooleanFalse` otherwise (as described later).

`t` An optional ⟨*token*⟩, which will result in a value `\BooleanTrue` if ⟨*token*⟩ is present and `\BooleanFalse` otherwise. Given as `t`⟨*token*⟩.

`e` Given as `e{`⟨*tokens*⟩`}`, a set of optional *embellishments*, each of which requires a *value*. If an embellishment is not present, `\NoValue` is returned. Each embellishment gives one argument, ordered as for the list of ⟨*tokens*⟩ in the argument specification. All ⟨*tokens*⟩ must be distinct.

`E` As for `e` but returns one or more ⟨*defaults*⟩ if values are not given: `E{`⟨*tokens*⟩`}{`⟨*defaults*⟩`}`. See Section 0.1.7 for more details.

0.1.3 Modifying argument descriptions

In addition to the argument *types* discussed above, the argument description also gives special meaning to three other characters.

First, `+` is used to make an argument long (to accept paragraph tokens). In contrast to `\newcommand`, this applies on an argument-by-argument basis. So modifying the example to "`s o o +m Odefault`" means that the mandatory argument is now `\long`, whereas the optional arguments are not.

Secondly, `!` is used to control whether spaces are allowed before optional arguments. There are some subtleties to this, as TEX itself has some restrictions on where spaces can be "detected": more detail is given in Section 0.1.6.

Thirdly, `=` is used to declare that the following argument should be interpreted as a series of keyvals. See Section 0.1.9 for more details.

Finally, the character `>` is used to declare so-called 'argument processors', which can be used to modify the contents of an argument before it is passed to the macro definition. The use of argument processors is a somewhat advanced topic, (or at least a less commonly used feature) and is covered in Section 0.1.10.

0.1.4 Creating document commands and environments

`\NewDocumentCommand`
`\RenewDocumentCommand`
`\ProvideDocumentCommand`
`\DeclareDocumentCommand`

`\NewDocumentCommand {`⟨*cmd*⟩`} {`⟨*arg spec*⟩`} {`⟨*code*⟩`}`

This family of commands are used to create a ⟨*cmd*⟩. The argument specification for the function is given by ⟨*arg spec*⟩, and the command uses the ⟨*code*⟩ with `#1`, `#2`, etc. replaced by the arguments found by the parser.

An example:

```
\NewDocumentCommand\chapter{s o m}
  {%
    \IfBooleanTF{#1}%
      {\typesetstarchapter{#3}}%
      {\typesetnormalchapter{#2}{#3}}%
  }
```

would be a way to define a `\chapter` command which would essentially behave like the current LaTeX 2_ε command (except that it would accept an optional argument even when a `*` was parsed). The `\typesetnormalchapter` could test its first argument for being `\NoValue` to see if an optional argument was present. (See Section 0.1.8 for details of `\IfBooleanTF` and testing for `\NoValue`.)

The difference between the `\New...` `\Renew...`, `\Provide...` and `\Declare...` versions is the behavior if ⟨*cmd*⟩ is already defined.

- `\NewDocumentCommand` will issue an error if ⟨*cmd*⟩ has already been defined.
- `\RenewDocumentCommand` will issue an error if ⟨*cmd*⟩ has not previously been defined.
- `\ProvideDocumentCommand` creates a new definition for ⟨*cmd*⟩ only if one has not already been given.
- `\DeclareDocumentCommand` will always create the new definition, irrespective of any existing ⟨*cmd*⟩ with the same name. This should be used sparingly.

If the ⟨*cmd*⟩ can't be provided as a single token but needs "constructing", you can use `\ExpandArgs` as explained in Section **??** which also gives an example in which this is needed.

`\NewDocumentEnvironment`
`\RenewDocumentEnvironment`
`\ProvideDocumentEnvironment`
`\DeclareDocumentEnvironment`

```
\NewDocumentEnvironment {⟨env⟩} {⟨arg spec⟩} {⟨beg-code⟩} {⟨end-code⟩}
```

These commands work in the same way as `\NewDocumentCommand`, etc., but create environments (`\begin{`⟨*env*⟩`}` ... `\end{`⟨*env*⟩`}`). Both the ⟨*beg-code*⟩ and ⟨*end-code*⟩ may access the arguments as defined by ⟨*arg spec*⟩. The arguments will be given following `\begin{`⟨*env*⟩`}`. Any spaces at the start and end of the `{`⟨*env*⟩`}` are removed before the definition takes place, thus

```
\NewDocumentEnvironment{foo}
```

and

```
\NewDocumentEnvironment{ foo }
```

both create the same "`foo`" environment.

0.1.5 Optional arguments

In contrast to commands created using LaTeX 2_ε's `\newcommand`, optional arguments created using `\NewDocumentCommand` may safely be nested. Thus for example, following

```
\NewDocumentCommand\foo{om}{I grabbed ‘#1’ and ‘#2’}
\NewDocumentCommand\baz{o}{#1-#1}
```

using the command as

```
\foo[\baz[stuff]]{more stuff}
```

will print

I grabbed ‘stuff-stuff’ and ‘more stuff’

This is particularly useful when placing a command with an optional argument *inside* the optional argument of a second command.

When an optional argument is followed by a mandatory argument with the same delimiter, the parser issues a warning because the optional argument could not be omitted by the user, thus becoming in effect mandatory. This can apply to `o`, `d`, `O`, `D`, `s`, `t`, `e`, and `E` type arguments followed by `r` or `R`-type required arguments.

The default for `O`, `D` and `E` arguments can be the result of grabbing another argument. Thus for example

```
\NewDocumentCommand\foo{O{#2} m}
```

would use the mandatory argument as the default for the leading optional one.

0.1.6 Spacing and optional arguments

TEX will find the first argument after a function name irrespective of any intervening spaces. This is true for both mandatory and optional arguments. So `\foo[arg]` and `\foo␣[arg]` are equivalent. Spaces are also ignored when collecting arguments up to the last mandatory argument to be collected (as it must exist). So after

```
\NewDocumentCommand\foo{m o m}{ ... }
```

the user input `\foo{arg1}[arg2]{arg3}` and `\foo{arg1}␣[arg2]␣{arg3}` will both be parsed in the same way.

The behavior of optional arguments *after* any mandatory arguments is selectable. The standard settings will allow spaces here, and thus with

```
\NewDocumentCommand\foobar{m o}{ ... }
```

both `\foobar{arg1}[arg2]` and `\foobar{arg1}␣[arg2]` will find an optional argument. This can be changed by giving the modified `!` in the argument specification:

```
\NewDocumentCommand\foobar{m !o}{ ... }
```

where `\foobar{arg1}␣[arg2]` will not find an optional argument.

There is one subtlety here due to the difference in handling by TEX of ‘control symbols’, where the command name is made up of a single character, such as ‘`\\`’. Spaces are not ignored by TEX here, and thus it is possible to require an optional argument directly follow such a command. The most common example is the use of `\\` in amsmath environments, which in the terms here would be defined as

```
\NewDocumentCommand\\{!s !o}{ ... }
```

Also notable when using optional arguments in the last position is that TEX will necessarily look ahead for the argument opening token. This means that the value of `\inputlineno` will be ‘out by one’ if such a trailing optional argument is *not* present and the command ends a line; it will be one greater than the line number containing the last mandatory argument.

0.1.7 'Embellishments'

The E-type argument allows one default value per test token. This is achieved by giving a list of defaults for each entry in the list, for example:

```
E{^_}{{UP}{DOWN}}
```

If the list of default values is *shorter* than the list of test tokens, the special \NoValue marker will be returned (as for the e-type argument). Thus for example

```
E{^_}{{UP}}
```

has default UP for the ^ test character, but will return the \NoValue marker as a default for _. This allows mixing of explicit defaults with testing for missing values.

0.1.8 Testing special values

Optional arguments make use of dedicated variables to return information about the nature of the argument received.

\IfNoValueTF
\IfNoValueT
\IfNoValueF

```
\IfNoValueTF {⟨arg⟩} {⟨true code⟩} {⟨false code⟩}
```

The \IfNoValue(TF) tests are used to check if ⟨*argument*⟩ (#1, #2, *etc.*) is the special \NoValue marker. For example

```
\NewDocumentCommand\foo{o m}
  {%
    \IfNoValueTF {#1}%
      {\DoSomethingJustWithMandatoryArgument{#2}}%
      {\DoSomethingWithBothArguments{#1}{#2}}%
  }
```

will use a different internal function if the optional argument is given than if it is not present.

Note that three tests are available, depending on which outcome branches are required: \IfNoValueTF, \IfNoValueT and \IfNoValueF.

As the \IfNoValue(TF) tests are expandable, it is possible to test these values later, for example at the point of typesetting or in an expansion context.

When two optional arguments follow each other (a syntax we typically discourage), it can make sense to allow users of the command to specify only the second argument by providing an empty first argument. If you wish to test if an argument is blank or not, but are not concerned with distinguishing an entirely absent argument from an empty one, use the O type specifier with the conditional \IfBlankTF (described below).

\IfValueTF
\IfValueT
\IfValueF

```
\IfValueTF {⟨arg⟩} {⟨true code⟩} {⟨false code⟩}
```

The reverse form of the \IfNoValue(TF) tests are also available as \IfValue(TF). The context will determine which logical form makes the most sense for a given code scenario.

\IfBlankTF \IfBlankTF {⟨*arg*⟩} {⟨*true code*⟩} {⟨*false code*⟩}
\IfBlankT
\IfBlankF

The \IfNoValueTF command chooses the ⟨*true code*⟩ if the optional argument has not been used at all (and it returns the special \NoValue marker), but not if it has been given an empty value. In contrast \IfBlankTF returns true if its argument is either truly empty or only contains one or more normal blanks. For example

```
\NewDocumentCommand\foo{m!o}{\par #1:
  \IfNoValueTF{#2}
    {No optional}%
    {%
      \IfBlankTF{#2}
        {Blanks in or empty}%
        {Real content in}%
    }%
  \space argument!}
\foo{1}[bar] \foo{2}[  ] \foo{3}[] \foo{4}[\space] \foo{5} [x]
```

results in the following output:

1: Real content in argument!

2: Blanks in or empty argument!

3: Blanks in or empty argument!

4: Real content in argument!

5: No optional argument! [x]

Note that the \space in (4) is considered real content—because it is a command and not a "space" character—even though it results in producing a space. You can also observe in (5) the effect of the ! specifier, preventing the last \foo from interpreting [x] as its optional argument.

\BooleanTrue
\BooleanFalse

The true and false flags set when searching for an optional character (using s or t⟨*char*⟩) have names which are accessible outside of code blocks.

\IfBooleanTF \IfBooleanTF {⟨*arg*⟩} {⟨*true code*⟩} {⟨*false code*⟩}
\IfBooleanT
\IfBooleanF

Used to test if ⟨*argument*⟩ (#1, #2, *etc.*) is \BooleanTrue or \BooleanFalse. For example

```
\NewDocumentCommand\foo{sm}
  {%
    \IfBooleanTF {#1}%
      {\DoSomethingWithStar{#2}}%
      {\DoSomethingWithoutStar{#2}}%
  }
```

checks for a star as the first argument, then chooses the action to take based on this information.

0.1.9 Auto-converting to key–value format

Some document commands have a long history of accepting a 'free text' optional argument, for example `\caption` and the sectioning commands `\section`, etc. Introducing more sophisticated (keyval) options to these commands therefore needs a method to interpret the optional argument *either* as free text *or* as a series of keyvals. This needs to take place during argument grabbing as there is a need for careful treatment of braces to obtain the correct result.

The `=` modifier is available to allow ltcmd to correctly implement this process. The modifier guarantees that the argument will be passed to further code as a series of keyvals. To do that, the `=` should be followed by an argument containing the default key name. This is used as the key in a key–value pair *if* the "raw" argument does *not* have the correct form to be interpreted as a set of keyvals.

Taking `\caption` as an example, with the demonstration implementation

```
\DeclareDocumentCommand\caption{s ={short-text} +O{#3} +m}
  {%
    \showtokens{Grabbed arguments:^^J(#2)^^Jand^^J(#3)}%
  }
```

the default key name is `short-text`. When the command `\caption` is then used, if the optional argument is free text such as

```
\caption[Some short text]{A much longer and more detailed text for
  demonstration purposes}
```

then the output will be

```
Grabbed arguments:
(short-text={Some short text})
and
(A much longer and more detailed text for demonstration purposes)
```

On the other hand, if the caption is given with a keyval-form argument

```
\caption[label = cap:demo]%
  {A much longer and more detailed text for demonstration purposes}
```

then this will be respected

```
Grabbed arguments:
(label = cap:demo)
and
(A much longer and more detailed text for demonstration purposes)
```

Interpretation as keyval form is determined by the presence of `=` characters within the argument. Those in inline math mode (enclosed within `$...$` or `\(...\)`) are ignored. An argument can be forced to be read as keyvals by including an empty entry at the start

```
\caption[=,This is now a keyval]%
% ...
\caption[This is not $=$ keyval]%
```

This empty entry is *not* passed to the underlying code, so will not lead to issues with keyval parsers that do not allow an empty key name. Any text-mode `=` signs will need to be braced to avoid being misinterpreted: this is likely most conveniently handled by bracing the entire argument

```
\caption[{Not = to a keyval!}]%
```

which will be passed correctly as

```
Grabbed arguments:
(short-text = {Not = to a keyval!})
```

If the argument is completely blank, the conversion results in an empty keyval list, not `short-text␣=␣`. This reflects the fact that with a move toward keyval processing, an empty argument is best modelled as an empty keyval list. If the user does want an empty classical argument, using `[{}]` will work with both the new processor code and older formats.

0.1.10 Argument processors

Argument processor are applied to an argument *after* it has been grabbed by the underlying system but before it is passed to ⟨*code*⟩. An argument processor can therefore be used to regularize input at an early stage, allowing the internal functions to be completely independent of input form. Processors are applied to user input and to default values for optional arguments, but *not* to the special `\NoValue` marker.

Each argument processor is specified by the syntax `>{`⟨*processor*⟩`}` in the argument specification. Processors are applied from right to left, so that

```
>{\ProcessorB} >{\ProcessorA} m
```

would apply `\ProcessorA` followed by `\ProcessorB` to the tokens grabbed by the `m` argument.

`\SplitArgument` `\SplitArgument {`⟨*number*⟩`} {`⟨*token(s)*⟩`}`

This processor splits the argument given at each occurrence of the ⟨*tokens*⟩ up to a maximum of ⟨*number*⟩ tokens (thus dividing the input into ⟨*number*⟩ + 1 parts). An error is given if too many ⟨*tokens*⟩ are present in the input. The processed input is placed inside ⟨*number*⟩ + 1 sets of braces for further use. If there are fewer than `{`⟨*number*⟩`}` of `{`⟨*tokens*⟩`}` in the argument then `\NoValue` markers are added at the end of the processed argument.

```
\NewDocumentCommand\foo{>{\SplitArgument{2}{;}} m}
  {\InternalFunctionOfThreeArguments#1}
```

If only a single character ⟨*token*⟩ is used for the split, any category code 13 (active) character matching the ⟨*token*⟩ will be replaced before the split takes place. Spaces are trimmed at each end of each item parsed.

The `E` argument type is somewhat special, because with a single `E` in the command declaration you may end up with several arguments in a command (one formal argument

per embellishment token). Therefore, when an argument processor is applied to an e/E-type argument, all the arguments pass through that processor before being fed to the ⟨*code*⟩. For example, this command

```
\NewDocumentCommand\foo{ >{\TrimSpaces} e{_^} }
  { [#1](#2) }
```

applies `\TrimSpaces` to both arguments.

`\SplitList` `\SplitList {`⟨*token(s)*⟩`}`

This processor splits the argument given at each occurrence of the ⟨*token(s)*⟩ where the number of items is not fixed. Each item is then wrapped in braces within `#1`. The result is that the processed argument can be further processed using a mapping function (see below).

```
\NewDocumentCommand\foo{>{\SplitList{;}} m}
  {\MappingFunction#1}
```

If only a single character ⟨*token*⟩ is used for the split, it will take account of the possibility that the ⟨*token*⟩ has been made active (category code 13) and will split at such tokens. Spaces are trimmed at each end of each item parsed. Exactly one set of braces will be stripped if an entire item is surrounded by them, i.e. the following inputs and outputs result (each separate item as a brace group).

```
a      ==> {a}
{a}    ==> {a}
{a}b   ==> {{a}b}
a,b    ==> {a}{b}
{a},b  ==> {a}{b}
a,{b}  ==> {a}{b}
a,{b}c ==> {a}{{b}c}
```

`\ProcessList` `\ProcessList {`⟨*list*⟩`} {`⟨*token(s)*⟩`}`

To support `\SplitList`, the function `\ProcessList` is available to apply ⟨*tokens*⟩ to every entry in a ⟨*list*⟩. The ⟨*tokens*⟩ can be arbitrary contents that should expect one argument after it: the list entry. For example

```
\NewDocumentCommand\foo{>{\SplitList{;}} m}
  {\ProcessList{#1}{\SomeDocumentCommand}}
```

or

```
\NewDocumentCommand\foo{>{\SplitList{;}} m}
  {\ProcessList{#1}{Abc \SomeDocumentCommand}}
```

`\ReverseBoolean` `\ReverseBoolean`

This processor reverses the logic of `\BooleanTrue` and `\BooleanFalse`, so that the example from earlier would become

```
\NewDocumentCommand\foo{>{\ReverseBoolean} s m}
  {%
    \IfBooleanTF#1%
      {\DoSomethingWithoutStar{#2}}%
      {\DoSomethingWithStar{#2}}%
  }
```

`\TrimSpaces` `\TrimSpaces`

Removes any leading and trailing spaces (tokens with character code 32 and category code 10) for the ends of the argument. Thus for example declaring a function

```
\NewDocumentCommand\foo{>{\TrimSpaces} m}
  {\showtokens{#1}}
```

and using it in a document as

```
\foo{␣hello␣world␣}
```

will show '`hello␣world`' at the terminal, with the space at each end removed. `\TrimSpaces` will remove multiple spaces from the ends of the input in cases where these have been included such that the standard TEX conversion of multiple spaces to a single space does not apply.

0.1.11 Body of an environment

While environments `\begin{`⟨*environment*⟩`}` ... `\end{`⟨*environment*⟩`}` are typically used in cases where the code implementing the ⟨`environment`⟩ does not need to access the contents of the environment (its "body"), it is sometimes useful to have the body as a standard argument.

This is achieved by ending the argument specification with `b`, which is a dedicated argument type for this situation. For instance

```
\NewDocumentEnvironment{twice}{O{\ttfamily} +b}
  {#2#1#2} {}
\begin{twice}[\itshape]
  Hello world!
\end{twice}
```

typesets 'Hello world!*Hello world!*'.

The prefix `+` is used to allow multiple paragraphs in the environment's body. Argument processors can also be applied to `b` arguments. By default, spaces are trimmed at both ends of the body: in the example there would otherwise be spaces coming from the ends the lines after `[\itshape]` and `world!`. Putting the prefix `!` before `b` suppresses space-trimming.

When `b` is used in the argument specification, the last argument of the environment declaration (e.g., `\NewDocumentEnvironment`), which consists of an ⟨`end code`⟩ to insert at `\end{`⟨*environment*⟩`}`, is redundant since one can simply put that code at the end of the ⟨`start code`⟩. Nevertheless this (empty) ⟨`end code`⟩ must be provided.

Environments that use this feature can be nested.

0.1.12 Fully-expandable document commands

Document commands created using `\NewDocumentCommand`, etc., are normally created so that they do not expand unexpectedly. This is done using engine features, so is more powerful than LaTeX 2$_\varepsilon$'s `\protect` mechanism. There are *very rare* occasion when it may be useful to create functions using a expansion-only grabber. This imposes a number of restrictions on the nature of the arguments accepted by a function, and the code it implements. This facility should only be used when *necessary*.

`\NewExpandableDocumentCommand`
`\RenewExpandableDocumentCommand`
`\ProvideExpandableDocumentCommand`
`\DeclareExpandableDocumentCommand`

`\NewExpandableDocumentCommand {⟨cmd⟩} {⟨arg spec⟩} {⟨code⟩}`

This family of commands is used to create a document-level ⟨*cmd*⟩, which will grab its arguments in a fully-expandable manner. The argument specification for the function is given by ⟨*arg spec*⟩, and the ⟨*cmd*⟩ will execute ⟨*code*⟩. In general, ⟨*code*⟩ will also be fully expandable, although it is possible that this will not be the case (for example, a function for use in a table might expand so that `\omit` is the first non-expandable non-space token).

Parsing arguments by pure expansion imposes a number of restrictions on both the type of arguments that can be read and the error checking available:

- The last argument (if any are present) must be one of the mandatory types `m`, `r` or `R`.
- The "verbatim" argument type `v` is not available.
- Argument processors (using `>`) are not available.
- It is not possible to differentiate between, for example `\foo[` and `\foo{[}`: in both cases the `[` will be interpreted as the start of an optional argument. As a result, checking for optional arguments is less robust than in the standard version.

0.1.13 Commands at the start of tabular cells

Creating commands that are used at the start of tabular cells imposes some restrictions on the underlying implementation. The standard LaTeX tabular environments (`tabular`, etc.) use a mechanism which requires that any command wrapping `\multicolumn` or similar must be "expandable". This is *not* the case for commands created using `\NewDocumentCommand`, etc., which as detailed in Section 0.1.12 use an engine feature which prevents such "expansion". Therefore, to create such wrappers for use at the start of tabular cells, you must use `\NewExpandableDocumentCommand`, for example

```
\NewExpandableDocumentCommand\MyMultiCol{m}{\multicolumn{3}{c}{#1}}
\begin{tabular}{lcr}
a & b & c \\
\MyMultiCol{stuff} \\
\end{tabular}
```

0.1.14 Using the verbatim argument types

As described above, the `v`-type argument may be viewed as similar to `\verb`. Before looking at exactly what that means, it is important to highlight some key differences. Most notably, *grabbing* a verbatim-like argument is separate from *typesetting* it: the latter is covered in the next section.

When grabbing a `v`-type argument, LaTeX first uses the kernel command `\dospecials` to turn off the "special" nature of characters. It then makes both spaces and tabs "active", so that they can be given a custom definition. Any other characters are grabbed as-is: this means that if any characters have been made "special" and are not listed in `\dospecials`, an error will arise (see below).

The characters that are grabbed as the argument are all those between two identical: in contrast to `\verb`, the characters `\`, `{`, `}` and `%` *cannot* be used as the delimiter character. If any of the grabbed tokens have "special" meaning, an error will be issued.

For the `+v`-type argument, which allows line breaks within the argument, newline characters are converted into `\obeyedline` commands. The standard definition of `\obeyedline` is simple `\par`, thus allowing the grabbed tokens to be used directly in typesetting. A local redefinition of `\obeyedline` can be used to achieve other outputs. For example, to retain blank lines whilst typesetting, one could use

```
\renewcommand*\obeyedline{\mbox{}\par}
```

More information about using these arguments in typesetting is in the following subsection.

Some additional details that may be useful for those with more TeX knowledge: do not worry if this does not make sense to you! Spaces and tabs are stored as active characters. In 8-bit engines, non-ASCII characters are "active", whilst other than the letters a–zA–Z, ASCII characters are "other". In Unicode engines, non-ASCII codepoints will be either letters or "other", based on the standard LaTeX settings derived from Unicode data. For token-based comparisons, it is likely that the active spaces and tabs should be replaced: this can be done conveniently by expansion.

0.1.15 Typesetting verbatim-like material

In contrast to `\verb`, the `(+)v`-type argument is only about *grabbing* the argument, not *typesetting* it. As such, features that users often associate with "verbatim" are not automatically activated, e.g., selecting a monospaced font. Material grabbed by the `v`-type argument does not automatically suppress ligatures: with modern TeX engines, this largely can be done without the token manipulation which `\verb` uses. (In `\verb`, ligatures are suppressed by making characters active and inserting a zero-width kern before the character itself.)

The `\verb` command also selects a monospaced font: this is not intrinsic to verbatim material, so will need to be set up using for example `\ttfamily`. Similarly, the `verbatim` environment sets up the meaning of `\par` suitable for breaking lines.

0.1.16 Verbatim environments

In some cases, when grabbing the body of an environment you will want the contents to be treated verbatim. This is available using the argument specification `c`. Like the `b` specification, this has to be the last one. Thus for example

```
\NewDocumentEnvironment{MyVerbatim}{!O{\ttfamily} c}
  {\begin{center} #1 #2\end{center}} {}
\begin{MyVerbatim}[\ttfamily\itshape]
  % Some code is shown here
  $y = mx + c$
\end{MyVerbatim}
```

will typeset verbatim the content, thus:

```
%␣␣␣␣%␣Some␣code␣is␣shown␣here
%␣␣␣␣$y␣=␣mx␣+␣c$
%␣
```

Since grabbing the entire contents verbatim will result in there being no `\par` tokens, newlines are always permitted: there is no need for a `+` modifier here. As for the `v` specification, newlines are stored as `\obeyedline`. In a similar fashion to the `b` specification, by default *newlines* are trimmed at both ends of the body. Putting the prefix `!` before `c` suppresses this trimming.

Collection of the body takes place on a line-by-line basis: content is collected up to the end-of-line in the source, then examined before storage. This means that the line ending the environment (containing in the example above `\end{MyVerbatim}`) cannot have any text *after* the end of the environment. Text *before* the end of environment is treated normally, but note that there is no trailing `\obeyedline` added if there is text here. Other than optional arguments, no text is allowed on the opening line of the environment.

Special handling is applied to a `o`, `O`, `d` or `D` specification argument immediately before an `c` specification. This means that when the optional argument is absent, the first character of the next line will be read with the correctly applied verbatim category code. Issues may arise if *multiple* optional arguments are used before a `c` specification: this will only work reliably where the optional tokens are "other" characters.

For technical reasons, we recommend that spaces are *not* ignored when searching for an optional argument before an `c` specification: this can be achieved by adding the `!` modifier as shown in the example. However, this is left as a choice for the user.

0.1.17 Performance

For document commands where the argument specification is entirely comprised of `m` or `+m` entries (or is entirely empty), the internal structure created by `\NewDocumentCommand` is essentially as efficient as provided by `\newcommand(*)`. As such, document commands may replace constructs arising from `\newcommand`, etc., without a need to be concerned about performance. It should be noted that `\newcommand(*)` produces expandable results, so the direct replacement is `\NewExpandableDocumentCommand`; in most cases, however, it is better to use `\NewDocumentCommand` to give more robust structures.

0.1.18 Details about argument delimiters

In normal (non-expandable) commands, the delimited types look for the initial delimiter by peeking ahead (using expl3's `\peek_...` functions) looking for the delimiter token. The token has to have the same meaning and "shape" of the token defined as delimiter. There are three possible cases of delimiters: character tokens, control sequence tokens, and active character tokens. For all practical purposes of this description, active character tokens will behave exactly as control sequence tokens.

Character tokens

A character token is characterized by its character code, and its meaning is the category code (`\catcode`). When a command is defined, the meaning of the character token is fixed into the definition of the command and cannot change. A command will correctly see an argument delimiter if the open delimiter has the same character and category codes as at the time of the definition. For example in:

```
\NewDocumentCommand { \foobar } { D<>{default} } {(#1)}
\foobar <hello> \par
\char_set_catcode_letter:N <
\foobar <hello>
```

the output would be:

```
(hello)
(default)<hello>
```

as the open-delimiter `<` changed in meaning between the two calls to `\foobar`, so the second one doesn't see the `<` as a valid delimiter. Commands assume that if a valid open-delimiter was found, a matching close-delimiter will also be there. If it is not (either by being omitted or by changing in meaning), a low-level TEX error is raised and the command call is aborted.

Control sequence tokens

A control sequence (or control character) token is characterized by its name, and its meaning is its definition. A token cannot have two different meanings at the same time. When a control sequence is defined as delimiter in a command, it will be detected as delimiter whenever the control sequence name is found in the document regardless of its current definition. For example in:

```
\cs_set:Npn \x { abc }
\NewDocumentCommand { \foobar } { D\x\y{default} } {(#1)}
\foobar \x hello\y \par
\cs_set:Npn \x { def }
\foobar \x hello\y
```

the output would be:

```
(hello)
(hello)
```

with both calls to the command seeing the delimiter `\x`.

0.1.19 Creating new argument processors

`\ProcessedArgument`

Argument processors allow manipulation of a grabbed argument before it is passed to the underlying code. New processor implementations may be created as functions which take one trailing argument, and which leave their result in the `\ProcessedArgument` variable. For example, `\ReverseBoolean` is defined as

```
\ExplSyntaxOn
\cs_new_protected:Npn \ReverseBoolean #1
  {
    \bool_if:NTF #1
      { \tl_set:Nn \ProcessedArgument { \c_false_bool } }
      { \tl_set:Nn \ProcessedArgument { \c_true_bool } }
  }
\ExplSyntaxOff
```

[As an aside: the code is written in expl3, so we don't have to worry about spaces creeping into the definition.]

Part II
Hooks

Chapter 1

LaTeX's hook management

1.1 Introduction

Hooks are points in the code of commands or environments where it is possible to add processing code into existing commands. This can be done by different packages that do not know about each other, and to allow for hopefully safe processing it is necessary to sort different chunks of code added by different packages into a suitable processing order.

This is done by the packages adding chunks of code (via `\AddToHook`) and labeling their code with some label by default using the package name as a label.

At `\begin{document}` all code for a hook is then sorted according to some rules (given by `\DeclareHookRule`) for fast execution without processing overhead. If the hook code is modified afterwards (or the rules are changed), a new version for fast processing is generated.

Some hooks are used already in the preamble of the document. If that happens then the hook is prepared for execution (and sorted) already at that point.

1.2 Package writer interface

The hook management system is offered as a set of CamelCase commands for traditional LaTeX 2_ε packages (and for use in the document preamble if needed) as well as `expl3` commands for modern packages, that use the L3 programming layer of LaTeX. Behind the scenes, a single set of data structures is accessed so that packages from both worlds can coexist and access hooks in other packages.

1.2.1 LaTeX 2_ε interfaces

Declaring hooks

With a few exceptions, hooks have to be declared before they can be used. The exceptions are the generic hooks for commands and environments (executed at `\begin` and `\end`), and the generic hooks run when loading files (see section 1.3.1).

`\NewHook` `\NewHook {⟨hook⟩}`

Creates a new ⟨*hook*⟩. If this hook is declared within a package it is suggested that its name is always structured as follows: ⟨*package-name*⟩`/`⟨*hook-name*⟩. If necessary you can further subdivide the name by adding more `/` parts. If a hook name is already taken, an error is raised and the hook is not created.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1. The string `??` can't be used as a hook name because it has a special significance as a placeholder in hook rules.

`\NewReversedHook` `\NewReversedHook {⟨hook⟩}`

Like `\NewHook` declares a new ⟨*hook*⟩. the difference is that the code chunks for this hook are in reverse order by default (those added last are executed first). Any rules for the hook are applied after the default ordering. See sections 1.2.3 and 1.2.4 for further details.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

`\NewMirroredHookPair` `\NewMirroredHookPair {⟨hook-1⟩} {⟨hook-2⟩}`

A shorthand for `\NewHook{`⟨*hook-1*⟩`}\NewReversedHook{`⟨*hook-2*⟩`}`.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

`\NewHookWithArguments` `\NewHookWithArguments {⟨hook⟩} {⟨number⟩}`

Creates a new ⟨*hook*⟩ whose code takes ⟨*number*⟩ arguments, and otherwise works exactly like `\NewHook`. Section 1.2.7 explains hooks with arguments.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

`\NewReversedHookWithArguments` `\NewReversedHookWithArguments {⟨hook⟩} {⟨number⟩}`

Like `\NewReversedHook`, but creates a hook whose code takes ⟨*number*⟩ arguments. Section 1.2.7 explains hooks with arguments.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

`\NewMirroredHookPairWithArguments` `\NewMirroredHookPairWithArguments {⟨hook-1⟩} {⟨hook-2⟩} {⟨number⟩}`

A shorthand for `\NewHookWithArguments{`⟨*hook-1*⟩`}{`⟨*number*⟩`}`
`\NewReversedHookWithArguments{`⟨*hook-2*⟩`}{`⟨*number*⟩`}`. Section 1.2.7 explains hooks with arguments.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

Special declarations for generic hooks

The declarations here should normally not be used. They are available to provide support for special use cases mainly involving generic command hooks.

\DisableGenericHook `\DisableGenericHook {⟨hook⟩}`

After this declaration[1] the ⟨*hook*⟩ is no longer usable: Any further attempt to add code to it will result in an error and any use, e.g., via `\UseHook`, will simply do nothing.

This is intended to be used with generic command hooks (see `ltcmdhooks-doc`) as depending on the definition of the command such generic hooks may be unusable. If that is known, a package developer can disable such hooks up front.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

\ActivateGenericHook `\ActivateGenericHook {⟨hook⟩}`

This declaration activates a generic hook provided by a package/class (e.g., one used in code with `\UseHook` or `\UseOneTimeHook`) without it being explicitly declared with `\NewHook`). If the hook is already activated, this command does nothing.

Note that this command does not undo the effect of `\DisableGenericHook`. See section 1.2.6 for a discussion of when this declaration is appropriate.

Using hooks in code

Using a hook that is executing the code that has been associated with it is only allowed if the hook has been previously declared with `\NewHook`. For performance reason there are no runtime checks for this and it is the responsibility of the programmer of a package to ensure that all hooks that are used in a package (with one of the commands in this section) are declared first.

\UseHook `\UseHook {⟨hook⟩}`

Execute the code stored in the ⟨*hook*⟩.

Before `\begin{document}` the fast execution code for a hook is not set up, so in order to use a hook there it is explicitly initialized first. As that involves assignments using a hook at those times is not 100% the same as using it after `\begin{document}`.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

\UseHookWithArguments `\UseHookWithArguments {⟨hook⟩} {⟨number⟩} {⟨arg1⟩} ... {⟨argn⟩}`

Execute the code stored in the ⟨*hook*⟩ and pass the arguments {⟨arg_1⟩} through {⟨arg_n⟩} to the ⟨*hook*⟩. Otherwise, it works exactly like `\UseHook`. The ⟨*number*⟩ should be the number of arguments declared for the hook. If the hook is not declared, this command does nothing and it will remove ⟨*number*⟩ items from the input. Section 1.2.7 explains hooks with arguments.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

[1] In the 2020/06 release this command was called `\DisableHook`, but that name was misleading as it shouldn't be used to disable non-generic hooks.

`\UseOneTimeHook` `\UseOneTimeHook {⟨hook⟩}`

Some hooks are only used (and can be only used) in one place, for example, those in `\begin{document}` or `\end{document}`. From that point onwards, adding to the hook through a defined `\⟨addto-cmd⟩` command (e.g., `\AddToHook` or `\AtBeginDocument`, etc.) would have no effect (as would the use of such a command inside the hook code itself). It is therefore customary to redefine `\⟨addto-cmd⟩` to simply process its argument, i.e., essentially make it behave like `\@firstofone`.

`\UseOneTimeHook` does that: it records that the hook has been consumed and any further attempt to add to it will result in executing the code to be added immediately.

Using `\UseOneTimeHook` several times with the same `{⟨hook⟩}` means that it only executes the first time it is used. For example, if it is used in a command that can be called several times then the hook executes during only the *first* invocation of that command; this allows its use as an "initialization hook".

Mixing `\UseHook` and `\UseOneTimeHook` for the same `{⟨hook⟩}` should be avoided, but if this is done then neither will execute after the first `\UseOneTimeHook`.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally. See section 1.2.1 for details.

`\UseOneTimeHookWithArguments` `\UseOneTimeHookWithArguments {⟨hook⟩} {⟨number⟩} {⟨arg1⟩} ... {⟨argn⟩}`

Works exactly like `\UseOneTimeHook`, but passes arguments `{⟨arg1⟩}` through `{⟨argn⟩}` to the ⟨*hook*⟩. The ⟨*number*⟩ should be the number of arguments declared for the hook. If the hook is not declared, this command does nothing and it will remove ⟨*number*⟩ items from the input.

It should be noted that after a one-time hook is used, it is no longer possible to use `\AddToHookWithArguments` or similar with that hook. `\AddToHook` continues to work as normal. Section 1.2.7 explains hooks with arguments.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally. See section 1.2.1 for details.

Updating code for hooks

In contrast to the commands from the previous section, declarations such as `\AddToHook` or `\DeclareHookRule` can be used even when the hook is not yet declared. The rationale is that the hook declaration may be in some package that is loaded later, or perhaps not loaded at all.

A side effect of this design is that misspellings do not raise an error but are simply regarded as declarations for hooks with a different name.

`\AddToHook` `\AddToHook {⟨hook⟩} [⟨label⟩] {⟨code⟩}`

Adds ⟨*code*⟩ to the ⟨*hook*⟩ labeled by ⟨*label*⟩. When the optional argument ⟨*label*⟩ is not provided, the ⟨*default label*⟩ is used (see section 1.2.1). If `\AddToHook` is used in a package/class, the ⟨*default label*⟩ is the package/class name, otherwise it is `top-level` (the `top-level` label is treated differently: see section 1.2.1).

If there already exists code under the ⟨*label*⟩ then the new ⟨*code*⟩ is appended to the existing one (even if this is a reversed hook). If you want to replace existing code under the ⟨*label*⟩, first apply `\RemoveFromHook`.

The hook doesn't have to exist for code to be added to it. However, if it is not declared, then obviously the added ⟨*code*⟩ will never be executed. This allows for hooks to work regardless of package loading order and enables packages to add to hooks from other packages without worrying whether they are actually used in the current document. See section 1.2.1.

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

`\AddToHookWithArguments` `\AddToHookWithArguments {⟨hook⟩} [⟨label⟩] {⟨code⟩}`

Works exactly like `\AddToHook`, except that the ⟨*code*⟩ can access the arguments passed to the hook using `#1`, `#2`, ..., `#n` (up to the number of arguments declared for the hook). If the ⟨*code*⟩ should contain *parameter tokens* (`#`) that are not supposed to be understood as the arguments of the hook, such tokens should be doubled. For example, with `\AddToHook` one can write:

```
\AddToHook{myhook}{\def\foo#1{Hello, #1!}}
```

but to achieve the same with `\AddToHookWithArguments`, one should write:

```
\AddToHookWithArguments{myhook}{\def\foo##1{Hello, ##1!}}
```

because in the latter case, `#1` refers to the first argument of the hook `myhook`. Section 1.2.7 explains hooks with arguments.

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

`\RemoveFromHook` `\RemoveFromHook {⟨hook⟩} [⟨label⟩]`

Removes any code labeled by ⟨*label*⟩ from the ⟨*hook*⟩. When the optional argument ⟨*label*⟩ is not provided, the ⟨*default label*⟩ is used (see section 1.2.1).

If there is no code under the ⟨*label*⟩ in the ⟨*hook*⟩, or if the ⟨*hook*⟩ does not exist, a warning is issued when you attempt to `\RemoveFromHook`, and the command is ignored. `\RemoveFromHook` should be used only when you know exactly what labels are in a hook. Typically this will be when some code gets added to a hook by a package, then later this code is removed by that same package. If you want to prevent the execution of code from another package, use the `voids` rule instead (see section 1.2.1).

If the optional ⟨*label*⟩ argument is `*`, then all code chunks are removed. This is rather dangerous as it may well drop code from other packages (that one may not know about); it should therefore not be used in packages but only in document preambles!

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

Important: *The* `\RemoveFromHook` *command should be only used if one has full control over the code chunk to be removed. In particular it should not be used to remove code chunks from other packages! For this the* `voids` *relation is provided.*

In contrast to the `voids` relationship between two labels in a `\DeclareHookRule` this is a destructive operation as the labeled code is removed from the hook data structure, whereas the relationship setting can be undone by providing a different relationship later.

A useful application for this declaration inside the document body is when one wants to temporarily add code to hooks and later remove it again, e.g.,

```
\AddToHook{env/quote/begin}{\small}
\begin{quote}
  A quote set in a smaller typeface
\end{quote}
...
\RemoveFromHook{env/quote/begin}
... now back to normal for further quotes
```

Note that you can't cancel the setting with

```
\AddToHook{env/quote/begin}{}
```

because that only "adds" a further empty chunk of code to the hook. Adding `\normalsize` would work but that means the hook then contained `\small\normalsize` which means two font size changes for no good reason.

The above is only needed if one wants to typeset several quotes in a smaller typeface. If the hook is only needed once then `\AddToHookNext` is simpler, because it resets itself after one use.

`\AddToHookNext` `\AddToHookNext {⟨hook⟩} {⟨code⟩}`

Adds ⟨*code*⟩ to the next invocation of the ⟨*hook*⟩. The code is executed after the normal hook code has finished and it is executed only once, i.e. it is deleted after it was used.

Using this declaration is a global operation, i.e., the code is not lost even if the declaration is used inside a group and the next invocation of the hook happens after the end of that group. If the declaration is used several times before the hook is executed then all code is executed in the order in which it was declared.[2]

If this declaration is used with a one-time hook then the code is only ever used if the declaration comes before the hook's invocation. This is because, in contrast to `\AddToHook`, the code in this declaration is not executed immediately in the case when the invocation of the hook has already happened—in other words, this code will truly execute only on the next invocation of the hook (and in the case of a one-time hook there is no such "next invocation"). This gives you a choice: should my code execute always, or should it execute only at the point where the one-time hook is used (and not at all if this is impossible)? For both of these possibilities there are use cases.

It is possible to nest this declaration using the same hook (or different hooks): e.g.,

`\AddToHookNext{`⟨*hook*⟩`}{`⟨*code-1*⟩`\AddToHookNext{`⟨*hook*⟩`}{`⟨*code-2*⟩`}}`

will execute ⟨*code-1*⟩ next time the ⟨*hook*⟩ is used and at that point puts ⟨*code-2*⟩ into the ⟨*hook*⟩ so that it gets executed on following time the hook is run.

A hook doesn't have to exist for code to be added to it. This allows for hooks to work regardless of package loading order. See section 1.2.1.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

[2] There is no mechanism to reorder such code chunks (or delete them).

\AddToHookNextWithArguments `\AddToHookNextWithArguments {⟨hook⟩} {⟨code⟩}`

Works exactly like `\AddToHookNext`, but the ⟨*code*⟩ can contain references to the arguments of the ⟨*hook*⟩ as described for `\AddToHookWithArguments` above. Section 1.2.7 explains hooks with arguments.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

\ClearHookNext `\ClearHookNext {⟨hook⟩}`

Normally `\AddToHookNext` is only used when you know precisely where it will apply and why you want some extra code at that point. However, there are a few use cases in which such a declaration needs to be canceled, for example, when discarding a page with `\DiscardShipoutBox` (but even then not always), and in such situations `\ClearHookNext` can be used.

Hook names and default labels

It is best practice to use `\AddToHook` in packages or classes *without specifying a* ⟨*label*⟩ because then the package or class name is automatically used, which is helpful if rules are needed, and avoids mistyping the ⟨*label*⟩.

Using an explicit ⟨*label*⟩ is only necessary in very specific situations, e.g., if you want to add several chunks of code into a single hook and have them placed in different parts of the hook (by providing some rules).

The other case is when you develop a larger package with several sub-packages. In that case you may want to use the same ⟨*label*⟩ throughout the sub-packages in order to avoid that the labels change if you internally reorganize your code.

Except for `\UseHook`, `\UseOneTimeHook` and `\IfHookEmptyTF` (and their expl3 interfaces `\hook_use:n`, `\hook_use_once:n` and `\hook_if_empty:nTF`), all ⟨*hook*⟩ and ⟨*label*⟩ arguments are processed in the same way: first, spaces are trimmed around the argument, then it is fully expanded until only character tokens remain. If the full expansion of the ⟨*hook*⟩ or ⟨*label*⟩ contains a non-expandable non-character token, a low-level TEX error is raised (namely, the ⟨*hook*⟩ is expanded using TEX's `\csname...\endcsname`, as such, Unicode characters are allowed in ⟨*hook*⟩ and ⟨*label*⟩ arguments). The arguments of `\UseHook`, `\UseOneTimeHook`, and `\IfHookEmptyTF` are processed much in the same way except that spaces are not trimmed around the argument, for better performance.

It is not enforced, but highly recommended that the hooks defined by a package, and the ⟨*labels*⟩ used to add code to other hooks contain the package name to easily identify the source of the code chunk and to prevent clashes. This should be the standard practice, so this hook management code provides a shortcut to refer to the current package in the name of a ⟨*hook*⟩ and in a ⟨*label*⟩. If the ⟨*hook*⟩ name or the ⟨*label*⟩ consist just of a single dot (`.`), or starts with a dot followed by a slash (`./`) then the dot denotes the ⟨*default label*⟩ (usually the current package or class name—see `\SetDefaultHookLabel`). A "`.`" or "`./`" anywhere else in a ⟨*hook*⟩ or in ⟨*label*⟩ is treated literally and is not replaced.

For example, inside the package `mypackage.sty`, the default label is `mypackage`, so the instructions:

```
\NewHook   {./hook}
\AddToHook {./hook}[.]{code}     % Same as \AddToHook{./hook}{code}
```

```
\AddToHook {./hook}[./sub]{code}
\DeclareHookRule{begindocument}{.}{before}{babel}
\AddToHook {file/foo.tex/after}{code}
```

are equivalent to:

```
\NewHook   {mypackage/hook}
\AddToHook {mypackage/hook}[mypackage]{code}
\AddToHook {mypackage/hook}[mypackage/sub]{code}
\DeclareHookRule{begindocument}{mypackage}{before}{babel}
\AddToHook {file/foo.tex/after}{code}                  % unchanged
```

The ⟨*default label*⟩ is automatically set equal to the name of the current package or class at the time the package is loaded. If the hook command is used outside of a package, or the current file wasn't loaded with `\usepackage` or `\documentclass`, then the `top-level` is used as the ⟨*default label*⟩. This may have exceptions—see `\PushDefaultHookLabel`.

This syntax is available in all ⟨*label*⟩ arguments and most ⟨*hook*⟩ arguments, both in the LaTeX 2_ε interface, and the LaTeX3 interface described in section 1.2.2.

Important:
The dot-syntax is ***not*** *available with* `\UseHook` *and some other commands that are typically used within code!*

Note, however, that the replacement of `.` by the ⟨*default label*⟩ takes place when the hook command is executed, so actions that are somehow executed after the package ends will have the wrong ⟨*default label*⟩ if the dot-syntax is used. For that reason, this syntax is not available in `\UseHook` (and `\hook_use:n`) because the hook is most of the time used outside of the package file in which it was defined. This syntax is also not available in the hook conditionals `\IfHookEmptyTF` (and `\hook_if_empty:nTF`), because these conditionals are used in some performance-critical parts of the hook management code, and because they are usually used to refer to other package's hooks, so the dot-syntax doesn't make much sense.

In some cases, for example in large packages, one may want to separate the code in logical parts, but still use the main package name as the ⟨*label*⟩, then the ⟨*default label*⟩ can be set using `\PushDefaultHookLabel{...}` ... `\PopDefaultHookLabel` or `\SetDefaultHookLabel{...}`.

\PushDefaultHookLabel
\PopDefaultHookLabel

```
\PushDefaultHookLabel {⟨default label⟩}
  ⟨code⟩
\PopDefaultHookLabel
```

`\PushDefaultHookLabel` sets the current ⟨*default label*⟩ to be used in ⟨*label*⟩ arguments, or when replacing a leading "`.`" (see above). `\PopDefaultHookLabel` reverts the ⟨*default label*⟩ to its previous value.

Inside a package or class, the ⟨*default label*⟩ is equal to the package or class name, unless explicitly changed. Everywhere else, the ⟨*default label*⟩ is `top-level` (see section 1.2.1) unless explicitly changed.

The effect of `\PushDefaultHookLabel` holds until the next `\PopDefaultHookLabel`. `\usepackage` (and `\RequirePackage` and `\documentclass`) internally use

```
\PushDefaultHookLabel{⟨package name⟩}
  ⟨package code⟩
\PopDefaultHookLabel
```

to set the ⟨*default label*⟩ for the package or class file. Inside the ⟨*package code*⟩ the ⟨*default label*⟩ can also be changed with `\SetDefaultHookLabel`. `\input` and other file input-related commands from the LaTeX kernel do not use `\PushDefaultHookLabel`, so code within files loaded by these commands does *not* get a dedicated ⟨*label*⟩! (that is, the ⟨*default label*⟩ is the current active one when the file was loaded.)

Packages that provide their own package-like interfaces (Ti*k*Z's `\usetikzlibrary`, for example) can use `\PushDefaultHookLabel` and `\PopDefaultHookLabel` to set dedicated labels and to emulate `\usepackage`-like hook behavior within those contexts.

The `top-level` label is treated differently, and is reserved to the user document, so it is not allowed to change the ⟨*default label*⟩ to `top-level`.

\SetDefaultHookLabel

```
\SetDefaultHookLabel {⟨default label⟩}
```

Similarly to `\PushDefaultHookLabel`, sets the current ⟨*default label*⟩ to be used in ⟨*label*⟩ arguments, or when replacing a leading "`.`". The effect holds until the label is changed again or until the next `\PopDefaultHookLabel`. The difference between `\PushDefaultHookLabel` and `\SetDefaultHookLabel` is that the latter does not save the current ⟨*default label*⟩.

This command is useful when a large package is composed of several smaller packages, but all should have the same ⟨*label*⟩, so `\SetDefaultHookLabel` can be used at the beginning of each package file to set the correct label.

`\SetDefaultHookLabel` is not allowed in the main document, where the ⟨*default label*⟩ is `top-level` and there is no `\PopDefaultHookLabel` to end its effect. It is also not allowed to change the ⟨*default label*⟩ to `top-level`.

The `top-level` label

The `top-level` label, assigned to code added from the main document, is different from other labels. Code added to hooks (usually `\AtBeginDocument`) in the preamble is almost always to change something defined by a package, so it should go at the very end of the hook.

Therefore, code added in the `top-level` is always executed at the end of the hook, regardless of where it was declared. If the hook is reversed (see `\NewReversedHook`), the `top-level` chunk is executed at the very beginning instead.

Rules regarding `top-level` have no effect: if a user wants to have a specific set of rules for a code chunk, they should use a different label to said code chunk, and provide a rule for that label instead.

The `top-level` label is exclusive for the user, so trying to add code with that label from a package results in an error.

Defining relations between hook code

The default assumption is that code added to hooks by different packages are independent and the order in which they are executed is irrelevant. While this is true in many cases it is obviously false in others.

Before the hook management system was introduced packages had to take elaborate precautions to determine whether some other package had also been loaded (before or after) and then to find some ways to alter its behavior accordingly. In addition is was often the user's responsibility to load packages in the right order so that alterations made by packages were done in thsat same order; and in some cases even altering the loading order wouldn't resolve the conflicts.

With the new hook management system it is now possible to define rules (i.e., relationships) between code chunks added by different packages and to specify explicitly the order in which they should be processed.

The rules can be declared for hooks before the hook has been declared with `\NewHook` and they are allowed to refer to code labels that do not yet exist, e.g., because a package defining the code chunk with that label has not yet been loaded. When the hook code is finally sorted for fast execution, all rules that apply are acted on and the others are ignored.

This offers the flexibility needed to handle complicated relationships between code from different packages and to set this up beforehand in a way that is independent of whether or not the packages are actually loaded in a specific document. The downside of this is that misspellings of hook names or code labels will not raise any error, instead the rule will simply never apply!

`\DeclareHookRule` `\DeclareHookRule {⟨hook⟩} {⟨label1⟩} {⟨relation⟩} {⟨label2⟩}`

Defines a relation between ⟨*label1*⟩ and ⟨*label2*⟩ for a given ⟨*hook*⟩. If ⟨*hook*⟩ is `??` this defines a default relation for all hooks that use the two labels, i.e., that have chunks of code labeled with ⟨*label1*⟩ and ⟨*label2*⟩.

Currently, the supported relations are the following:

`before` or `<` Code for ⟨*label1*⟩ comes before code for ⟨*label2*⟩.

`after` or `>` Code for ⟨*label1*⟩ comes after code for ⟨*label2*⟩.

`incompatible-warning` Only code for either ⟨*label1*⟩ or ⟨*label2*⟩ can appear for that hook (a way to say that two packages—or parts of them—are incompatible). A warning is raised if both labels appear in the same hook.

`incompatible-error` Like `incompatible-error` but instead of a warning a LaTeX error is raised, and the code for both labels are dropped from that hook until the conflict is resolved.

`voids` Code for ⟨*label1*⟩ overwrites code for ⟨*label2*⟩. More precisely, code for ⟨*label2*⟩ is dropped for that hook. This can be used, for example if one package is a superset in functionality of another one and therefore wants to undo code in some hook and replace it with its own version.

`unrelated` The order of code for ⟨*label1*⟩ and ⟨*label2*⟩ is irrelevant. This rule is there to undo an incorrect rule specified earlier.

There can only be a single relation between two labels for a given hook, i.e., a later `\DeclareHookRule` overwrites any previous declaration. In all cases rules specific to a given hook take precedence over default rules that use `??` as the ⟨*hook*⟩.

If a default rule is applied, it is done before reversing the label order in a reversed hook, e.g., `before` in a default rule effectively becomes `after` in such a hook. In contrast, a rule for a specific hook is always applied to the state after any reversal (i.e., the state you see when using `\ShowHook` on that hook).

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

`\ClearHookRule` `\ClearHookRule {⟨hook⟩} {⟨label1⟩} {⟨label2⟩}`

Syntactic sugar for saying that ⟨*label1*⟩ and ⟨*label2*⟩ are unrelated for the given ⟨*hook*⟩.

\DeclareDefaultHookRule

`\DeclareDefaultHookRule {⟨label1⟩} {⟨relation⟩} {⟨label2⟩}`

This sets up a relation between ⟨*label1*⟩ and ⟨*label2*⟩ for all hooks unless overwritten by a specific rule for a hook. Useful for cases where one package has a specific relation to some other package, e.g., is `incompatible` or always needs a special ordering `before` or `after`. (Technically it is just a shorthand for using `\DeclareHookRule` with `??` as the hook name.)

If such a rule is applied to a reversed hook it behaves as if the rule is reversed (e.g., `after` becomes `before`) because those rules are applied first and then the order is reversed. The rationale is that in hook pairs (in which the ordering in one is reversed) default rules have to be reversed too in nearly all scenarios. If this is not the case, a default rule can't be used or has to be overwritten with an explicit `\DeclareHookRule` for that specific hook.

Declaring default rules is only supported in the document preamble.[3]

The ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

Querying hooks

Simpler data types, like token lists, have three possible states; they can:

- exist and be empty;
- exist and be non-empty; and
- not exist (in which case emptiness doesn't apply);

Hooks are a bit more complicated: a hook may exist or not, and independently it may or may not be empty. This means that even a hook that doesn't exist may be non-empty and it can also be disabled.

This seemingly strange state may happen when, for example, package A defines hook `A/foo`, and package B adds some code to that hook. However, a document may load package B before package A, or may not load package A at all. In both cases some code is added to hook `A/foo` without that hook being defined yet, thus that hook is said to be non-empty, whereas it doesn't exist. Therefore, querying the existence of a hook doesn't imply its emptiness, neither does the other way around.

Given that code or rules can be added to a hook even if it doesn't physically exist yet, means that a querying its existence has no real use case (in contrast to other variables that can only be update if they have already been declared). For that reason only the test for emptiness has a public interface.

A hook is said to be empty when no code was added to it, either to its permanent code pool, or to its "next" token list. The hook doesn't need to be declared to have code added to its code pool. A hook is said to exist when it was declared with `\NewHook` or some variant thereof. Generic hooks such as `file` and `env` hooks are automatically declared when code is added to them.

[3] Trying to do so, e.g., via `\DeclareHookRule` with `??` has bad side-effects and is not supported (though not explicitly caught for performance reasons).

\IfHookEmptyTF ⋆ \IfHookEmptyTF {⟨*hook*⟩} {⟨*true code*⟩} {⟨*false code*⟩}
\IfHookEmptyT ⋆
\IfHookEmptyF ⋆

Tests if the ⟨*hook*⟩ is empty (*i.e.*, no code was added to it using either \AddToHook or \AddToHookNext) or such code was removed again (via \RemoveFromHook), and branches to either ⟨*true code*⟩ or ⟨*false code*⟩ depending on the result.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading . is treated literally.

Displaying hook code

If one has to adjust the code execution in a hook using a hook rule it is helpful to get some information about the code associated with a hook, its current order and the existing rules.

\ShowHook \ShowHook {⟨*hook*⟩}
\LogHook \LogHook {⟨*hook*⟩}

Displays information about the ⟨*hook*⟩ such as

- the code chunks (and their labels) added to it,
- any rules set up to order them,
- the computed order in which the chunks are executed,
- any code executed on the next invocation only.

\LogHook prints the information to the .log file, and \ShowHook prints them to the terminal/command window and starts TEX's prompt (only in \errorstopmode) to wait for user action.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

Suppose a hook example-hook whose output of \ShowHook{example-hook} is:

```
1  -> The hook 'example-hook':
2  > Code chunks:
3  >     foo -> [code from package 'foo']
4  >     bar -> [from package 'bar']
5  >     baz -> [package 'baz' is here]
6  > Document-level (top-level) code (executed last):
7  >     -> [code from 'top-level']
8  > Extra code for next invocation:
9  >     -> [one-time code]
10 > Rules:
11 >     foo|baz with relation >
12 >     baz|bar with default relation <
13 > Execution order (after applying rules):
14 >     baz, foo, bar.
```

In the listing above, lines 3 to 5 show the three code chunks added to the hook and their respective labels in the format

⟨*label*⟩ -> ⟨*code*⟩

Line 7 shows the code chunk added by the user in the main document (labeled `top-level`) in the format

```
Document-level (top-level) code (executed ⟨first|last⟩):
  -> ⟨top-level code⟩
```

This code will be either the first or last code executed by the hook (`last` if the hook is normal, `first` if it is reversed). This chunk is not affected by rules and does not take part in sorting.

Line 9 shows the code chunk for the next execution of the hook in the format

```
-> ⟨next-code⟩
```

This code will be used and disappear at the next `\UseHook{example-hook}`, in contrast to the chunks mentioned earlier, which can only be removed from that hook by doing `\RemoveFromHook{⟨label⟩}[example-hook]`.

Lines 11 and 12 show the rules declared that affect this hook in the format

```
⟨label-1⟩|⟨label-2⟩ with ⟨default?⟩ relation ⟨relation⟩
```

which means that the ⟨*relation*⟩ applies to ⟨*label-1*⟩ and ⟨*label-2*⟩, in that order, as detailed in `\DeclareHookRule`. If the relation is `default` it means that this rule applies to ⟨*label-1*⟩ and ⟨*label-2*⟩ in *all* hooks, (unless overridden by a non-default relation).

Finally, line 14 lists the labels in the hook after sorting; that is, in the order they will be executed when the hook is used.

Debugging hook code

`\DebugHooksOn`
`\DebugHooksOff`

`\DebugHooksOn ... \DebugHooksOff`

Turn the debugging of hook code on or off. This displays most changes made to the hook data structures. The output is rather coarse and not really intended for normal use, but it can be helpful in case hooks do not work as expected. See also 1.2.1 for commands to inspect individual hooks.

1.2.2 L3 programming layer (`expl3`) interfaces

This is a quick summary of the LaTeX3 programming interfaces for use with packages written in `expl3`. In contrast to the LaTeX 2ε interfaces they always use mandatory arguments only, e.g., you always have to specify the ⟨*label*⟩ for a code chunk. We therefore suggest to use the declarations discussed in the previous section even in `expl3` packages, but the choice is yours.

`\hook_new:n`
`\hook_new_reversed:n`
`\hook_new_pair:nn`

`\hook_new:n {⟨hook⟩}`
`\hook_new_reversed:n {⟨hook⟩}`
`\hook_new_pair:nn {⟨hook-1⟩} {⟨hook-2⟩}`

Creates a new ⟨*hook*⟩ with normal or reverse ordering of code chunks. `\hook_new_pair:nn` creates a pair of such hooks with `{`⟨*hook-2*⟩`}` being a reversed hook. If a hook name is already taken, an error is raised and the hook is not created.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

`\hook_new_with_args:nn` `\hook_new_with_args:nn {⟨hook⟩} {⟨number⟩}`
`\hook_new_reversed_with_args:nn` `\hook_new_reversed_with_args:nn {⟨hook⟩} {⟨number⟩}`
`\hook_new_pair_with_args:nnn` `\hook_new_pair_with_args:nnn {⟨hook-1⟩} {⟨hook-2⟩} {⟨number⟩}`

Creates a new ⟨*hook*⟩ with normal or reverse ordering of code chunks, that takes ⟨*number*⟩ arguments from the input stream when it is used. `\hook_new_pair_with_args:nn` creates a pair of such hooks with {⟨*hook-2*⟩} being a reversed hook. If a hook name is already taken, an error is raised and the hook is not created.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

`\hook_disable_generic:n` `\hook_disable_generic:n {⟨hook⟩}`

Marks {⟨*hook*⟩} as disabled. Any further attempt to add code to it or declare it, will result in an error and any call to `\hook_use:n` will simply do nothing.

This declaration is intended for use with generic hooks that are known not to work (see `ltcmdhooks-doc`) if they receive code.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

`\hook_activate_generic:n` `\hook_activate_generic:n {⟨hook⟩}`

This is like `\hook_new:n` but it does nothing if the hook was previously declared with `\hook_new:n`. This declaration should be used only in special situations, e.g., when a command from another package needs to be altered and it is not clear whether a generic `cmd` hook (for that command) has been previously explicitly declared.

Normally `\hook_new:n` should be used instead of this.

`\hook_use:n` `\hook_use:n {⟨hook⟩}`
`\hook_use:nnw` `\hook_use:nnw {⟨hook⟩} {⟨number⟩} {⟨arg1⟩} ... {⟨argn⟩}`

Executes the {⟨*hook*⟩} code followed (if set up) by the code for next invocation only, then empties that next invocation code. `\hook_use:nnw` should be used for hooks declared with arguments, and should be followed by as many brace groups as the declared number of arguments. The ⟨*number*⟩ should be the number of arguments declared for the hook. If the hook is not declared, this command does nothing and it will remove ⟨*number*⟩ items from the input.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

`\hook_use_once:n` `\hook_use_once:n {⟨hook⟩}`
`\hook_use_once:nnw` `\hook_use_once:nnw {⟨hook⟩} {⟨number⟩} {⟨arg1⟩} ... {⟨argn⟩}`

Changes the {⟨*hook*⟩} status so that from now on any addition to the hook code is executed immediately. Then execute any {⟨*hook*⟩} code already set up. `\hook_use_once:nnw` should be used for hooks declared with arguments, and should be followed by as many brace groups as the declared number of arguments. The ⟨*number*⟩ should be the number of arguments declared for the hook. If the hook is not declared, this command does nothing and it will remove ⟨*number*⟩ items from the input.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

\hook_gput_code:nnn \hook_gput_code:nnn {⟨hook⟩} {⟨label⟩} {⟨code⟩}
\hook_gput_code_with_args:nnn \hook_gput_code_with_args:nnn {⟨hook⟩} {⟨label⟩} {⟨code⟩}

Adds a chunk of ⟨*code*⟩ to the ⟨*hook*⟩ labeled ⟨*label*⟩. If the label already exists the ⟨*code*⟩ is appended to the already existing code.

If `\hook_gput_code_with_args:nnn` is used, the ⟨*code*⟩ can access the arguments passed to `\hook_use:nnw` (or `\hook_use_once:nnw`) with `#1`, `#2`, ..., `#n` (up to the number of arguments declared for the hook). In that case, if an actual parameter token should be added to the code, it should be doubled.

If code is added to an external ⟨*hook*⟩ (of the kernel or another package) then the convention is to use the package name as the ⟨*label*⟩ not some internal module name or some other arbitrary string.

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

\hook_gput_next_code:nn \hook_gput_next_code:nn {⟨hook⟩} {⟨code⟩}
\hook_gput_next_code_with_args:nn \hook_gput_next_code_with_args:nn {⟨hook⟩} {⟨code⟩}

Adds a chunk of ⟨*code*⟩ for use only in the next invocation of the ⟨*hook*⟩. Once used it is gone.

If `\hook_gput_next_code_with_args:nn` is used, the ⟨*code*⟩ can access the arguments passed to `\hook_use:nnw` (or `\hook_use_once:nnw`) with `#1`, `#2`, ..., `#n` (up to the number of arguments declared for the hook). In that case, if an actual parameter token should be added to the code, it should be doubled.

This is simpler than `\hook_gput_code:nnn`, the code is simply appended to the hook in the order of declaration at the very end, i.e., after all standard code for the hook got executed. Thus if one needs to undo what the standard does one has to do that as part of ⟨*code*⟩.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

\hook_gclear_next_code:n \hook_gclear_next_code:n {⟨hook⟩}

Undo any earlier `\hook_gput_next_code:nn`.

\hook_gremove_code:nn \hook_gremove_code:nn {⟨hook⟩} {⟨label⟩}

Removes any code for ⟨*hook*⟩ labeled ⟨*label*⟩.

If there is no code under the ⟨*label*⟩ in the ⟨*hook*⟩, or if the ⟨*hook*⟩ does not exist, a warning is issued when you attempt to use `\hook_gremove_code:nn`, and the command is ignored.

If the second argument is `*`, then all code chunks are removed. This is rather dangerous as it drops code from other packages one may not know about, so think twice before using that!

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

`\hook_gset_rule:nnnn` `\hook_gset_rule:nnnn {⟨hook⟩} {⟨label1⟩} {⟨relation⟩} {⟨label2⟩}`

Relate ⟨*label1*⟩ with ⟨*label2*⟩ when used in ⟨*hook*⟩. See `\DeclareHookRule` for the allowed ⟨*relation*⟩s. If ⟨*hook*⟩ is `??` a default rule is specified.

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1. The dot-syntax is parsed in both ⟨*label*⟩ arguments, but it usually makes sense to be used in only one of them.

`\hook_if_empty_p:n` ⋆ `\hook_if_empty:nTF {⟨hook⟩} {⟨true code⟩} {⟨false code⟩}`
`\hook_if_empty:nTF` ⋆

Tests if the ⟨*hook*⟩ is empty (*i.e.*, no code was added to it using either `\AddToHook` or `\AddToHookNext`), and branches to either ⟨*true code*⟩ or ⟨*false code*⟩ depending on the result.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

`\hook_show:n` `\hook_show:n {⟨hook⟩}`
`\hook_log:n` `\hook_log:n {⟨hook⟩}`

Displays information about the ⟨*hook*⟩ such as

- the code chunks (and their labels) added to it,
- any rules set up to order them,
- the computed order in which the chunks are executed,
- any code executed on the next invocation only.

`\hook_log:n` prints the information to the `.log` file, and `\hook_show:n` prints them to the terminal/command window and starts TEX's prompt (only if `\errorstopmode`) to wait for user action.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 1.2.1.

`\hook_debug_on:` `\hook_debug_on:`
`\hook_debug_off:`

Turns the debugging of hook code on or off. This displays changes to the hook data.

1.2.3 On the order of hook code execution

Chunks of code for a ⟨*hook*⟩ under different labels are supposed to be independent if there are no special rules set up that define a relation between the chunks. This means that you can't make assumptions about the order of execution!

Suppose you have the following declarations:

```
\NewHook{myhook}
\AddToHook{myhook}[packageA]{\typeout{A}}
\AddToHook{myhook}[packageB]{\typeout{B}}
\AddToHook{myhook}[packageC]{\typeout{C}}
```

then executing the hook with `\UseHook` will produce the typeout `A B C` in that order. In other words, the execution order is computed to be `packageA`, `packageB`, `packageC` which you can verify with `\ShowHook{myhook}`:

```
-> The hook 'myhook':
> Code chunks:
>     packageA -> \typeout {A}
>     packageB -> \typeout {B}
>     packageC -> \typeout {C}
> Document-level (top-level) code (executed last):
>     ---
> Extra code for next invocation:
>     ---
> Rules:
>     ---
> Execution order:
>     packageA, packageB, packageC.
```

The reason is that the code chunks are internally saved in a property list and the initial order of such a property list is the order in which key-value pairs got added. However, that is only true if nothing other than adding happens!

Suppose, for example, you want to replace the code chunk for `packageA`, e.g.,

```
\RemoveFromHook{myhook}[packageA]
\AddToHook{myhook}[packageA]{\typeout{A alt}}
```

then your order becomes `packageB`, `packageC`, `packageA` because the label got removed from the property list and then re-added (at its end).

While that may not be too surprising, the execution order is also sometimes altered if you add a redundant rule, e.g. if you specify

```
\DeclareHookRule{myhook}{packageA}{before}{packageB}
```

instead of the previous lines we get

```
-> The hook 'myhook':
> Code chunks:
>     packageA -> \typeout {A}
>     packageB -> \typeout {B}
>     packageC -> \typeout {C}
> Document-level (top-level) code (executed last):
>     ---
> Extra code for next invocation:
>     ---
> Rules:
>     packageB|packageA with relation >
> Execution order (after applying rules):
>     packageA, packageC, packageB.
```

As you can see the code chunks are still in the same order, but in the execution order for the labels `packageB` and `packageC` have swapped places. The reason is that, with the rule there are two orders that satisfy it, and the algorithm for sorting happened to pick a different one compared to the case without rules (where it doesn't run at all as there is nothing to resolve). Incidentally, if we had instead specified the redundant rule

```
\DeclareHookRule{myhook}{packageB}{before}{packageC}
```

the execution order would not have changed.

In summary: it is not possible to rely on the order of execution unless there are rules that partially or fully define the order (in which you can rely on them being fulfilled).

1.2.4 The use of “reversed” hooks

You may have wondered why you can declare a “reversed” hook with `\NewReversedHook` and what that does exactly.

In short: the execution order of a reversed hook (without any rules!) is exactly reversed to the order you would have gotten for a hook declared with `\NewHook`.

This is helpful if you have a pair of hooks where you expect to see code added that involves grouping, e.g., starting an environment in the first and closing that environment in the second hook. To give a somewhat contrived example[4], suppose there is a package adding the following:

```
\AddToHook{env/quote/before}[package-1]{\begin{itshape}}
\AddToHook{env/quote/after} [package-1]{\end{itshape}}
```

As a result, all quotes will be in italics. Now suppose further that another `package-too` makes the quotes also in blue and therefore adds:

```
\usepackage{color}
\AddToHook{env/quote/before}[package-too]{\begin{color}{blue}}
\AddToHook{env/quote/after} [package-too]{\end{color}}
```

Now if the `env/quote/after` hook would be a normal hook we would get the same execution order in both hooks, namely:

```
package-1, package-too
```

(or vice versa) and as a result, would get:

```
\begin{itshape}\begin{color}{blue} ...
\end{itshape}\end{color}
```

and an error message saying that `\begin{color}` was ended by `\end{itshape}`. With `env/quote/after` declared as a reversed hook the execution order is reversed and so all environments are closed in the correct sequence and `\ShowHook` would give us the following output:

```
-> The hook 'env/quote/after':
> Code chunks:
>     package-1 -> \end {itshape}
>     package-too -> \end {color}
> Document-level (top-level) code (executed first):
>     ---
> Extra code for next invocation:
>     ---
> Rules:
>     ---
> Execution order (after reversal):
>     package-too, package-1.
```

If there is a matching default rule (done with `\DeclareDefaultHookRule` or with `??` for the hook name) then this default rule is applied before the reversal so that the order in the reversed hook mirrors the one in the normal hook. However, all rules specific to a hook happen always after the reversal of the execution order, so if you alter the order you will probably have to alter it in both hooks, not just in one, but that depends on the use case.

[4] There are simpler ways to achieve the same effect.

1.2.5 Difference between "normal" and "one-time" hooks

When executing a hook a developer has the choice of using either `\UseHook` or `\UseOneTimeHook` (or their `expl3` equivalents `\hook_use:n` and `\hook_use_once:n`). This choice affects how `\AddToHook` is handled after the hook has been executed for the first time.

With normal hooks adding code via `\AddToHook` means that the code chunk is added to the hook data structure and then used each time `\UseHook` is called.

With one-time hooks it this is handled slightly differently: After `\UseOneTimeHook` has been called, any further attempts to add code to the hook via `\AddToHook` will simply execute the ⟨*code*⟩ immediately.

This has some consequences one needs to be aware of:

- If ⟨*code*⟩ is added to a normal hook after the hook was executed and it is never executed again for one or the other reason, then this new ⟨*code*⟩ will never be executed.
- In contrast if that happens with a one-time hook the ⟨*code*⟩ is executed immediately.

In particular this means that construct such as

```
\AddToHook{myhook}
          { ⟨code-1⟩ \AddToHook{myhook}{⟨code-2⟩} ⟨code-3⟩ }
```

works for one-time hooks[5] (all three code chunks are executed one after another), but it makes little sense with a normal hook, because with a normal hook the first time `\UseHook{myhook}` is executed it would

- execute ⟨*code-1*⟩,
- then execute `\AddToHook{myhook}{code-2}` which adds the code chunk ⟨*code-2*⟩ to the hook for use on the next invocation,
- and finally execute ⟨*code-3*⟩.

The second time `\UseHook` is called it would execute the above and in addition ⟨*code-2*⟩ as that was added as a code chunk to the hook in the meantime. So each time the hook is used another copy of ⟨*code-2*⟩ is added and so that code chunk is executed ⟨*# of invocations*⟩ − 1 times.

1.2.6 Generic hooks provided by packages

The hook management system also implements a category of hooks that are called "Generic Hooks". Normally a hook has to be explicitly declared before it can be used in code. This ensures that different packages are not using the same hook name for unrelated purposes—something that would result in absolute chaos. However, there are a number of "standard" hooks where it is unreasonable to declare them beforehand, e.g, each and every command has (in theory) an associated `before` and `after` hook. In such cases, i.e., for command, environment or file hooks, they can be used simply by adding code to them with `\AddToHook`. For more specialized generic hooks, e.g., those provided

[5] This is sometimes used with `\AtBeginDocument` which is why it is supported.

by babel, you have to additionally enable them with `\ActivateGenericHook` as explained below.

The generic hooks provided by LaTeX are those for `cmd`, `env`, `file`, `include`, `package`, and `class`, and all these are available out of the box: you only have to use `\AddToHook` to add code to them, but you don't have to add `\UseHook` or `\UseOneTimeHook` to your code, because this is already done for you (or, in the case of `cmd` hooks, the command's code is patched at `\begin{document}`, if necessary).

However, if you want to provide further generic hooks in your own code, the situation is slightly different. To do this you should use `\UseHook` or `\UseOneTimeHook`, but *without declaring the hook* with `\NewHook`. As mentioned earlier, a call to `\UseHook` with an undeclared hook name does nothing. So as an additional setup step, you need to explicitly activate your generic hook. Note that a generic hook produced in this way is always a normal hook.

For a truly generic hook, with a variable part in the hook name, such upfront activation would be difficult or impossible, because you typically do not know what kind of variable parts may come up in real documents.

For example, babel provides hooks such as `babel/`⟨*language*⟩`/afterextras`. However, language support in babel is often done through external language packages. Thus doing the activation for all languages inside the core babel code is not a viable approach. Instead it needs to be done by each language package (or by the user who wants to use a particular hook).

Because the hooks are not declared with `\NewHook` their names should be carefully chosen to ensure that they are (likely to be) unique. Best practice is to include the package or command name, as was done in the babel example above.

Generic hooks defined in this way are always normal hooks (i.e., you can't implement reversed hooks this way). This is a deliberate limitation, because it speeds up the processing considerably.

1.2.7 Hooks with arguments

Sometimes it is necessary to pass contextual information to a hook, and, for one reason or another, it is not feasible to store such information in macros. To serve this purpose, hooks can be declared with arguments, so that the programmer can pass along the data necessary for the code in the hook to function properly.

A hook with arguments works mostly like a regular hook, and most commands that work for regular hooks, also work for hooks that take arguments. The differences are when the hook is declared (`\NewHookWithArguments` is used instead of `\NewHook`), then code can be added with both `\AddToHook` and `\AddToHookWithArguments`, and when the hook is used (`\UseHookWithArguments` instead of `\UseHook`).

A hook with arguments must be declared as such (before it is first used, as all regular hooks) using `\NewHookWithArguments{`⟨*hook*⟩`}{`⟨*number*⟩`}`. All code added to that hook can then use `#1` to access the first argument, `#2` to access the second, and so forth up to the number of arguments declared. However, it is still possible to add code with references to the arguments of a hook that was not yet declared (we will discuss that later). At their core, hooks are macros, so TeX's limit of 9 arguments applies, and a low-level TeX error is raised if you try to reference an argument number that doesn't exist.

To use a hook with arguments, just write `\UseHookWithArguments{`⟨*hook*⟩`}{`⟨*number*⟩`}` followed by a braced list of the arguments. For example, if the hook `test` takes three arguments, write:

```
\UseHookWithArguments{test}{3}{arg-1}{arg-2}{arg-3}
```

then, in the ⟨*code*⟩ of the hook, all instances of `#1` will be replaced by `arg-1`, `#2` by `arg-2` and so on. If, at the point of usage, the programmer provides more arguments than the hook is declared to take, the excess arguments are simply ignored by the hook. Behavior is unpredictable[6] if too few arguments are provided. If the hook isn't declared, ⟨*number*⟩ arguments are removed from the input stream.

Adding code to a hook with arguments can be done with `\AddToHookWithArguments` as well as with the regular `\AddToHook`, to achieve different outcomes. The main difference when it comes to adding code to a hook, in this case, is firstly the possibility of accessing a hook's arguments, of course, and second, how parameter tokens (`#`$_6$) are treated.

Using `\AddToHook` in a hook that takes arguments will work as it does for all other hooks. This allows a package developer to add arguments to a hook that otherwise had none without having to worry about compatibility. This means that, for example:

```
\AddToHook{test}{\def\foo#1{Hello, #1!}}
```

will define the same macro `\foo` regardless if the hook `test` takes arguments or not.

Using `\AddToHookWithArguments` allows the ⟨*code*⟩ added to access the arguments of the hook with `#1`, `#2`, and so forth, up to the number of the arguments declared in the hook. This means that if one wants to add a `#`$_6$ to the ⟨*code*⟩ that token must be doubled in the input. The same definition from above, using `\AddToHookWithArguments`, needs to be rewritten:

```
\AddToHookWithArguments{test}{\def\foo##1{Hello, ##1!}}
```

Extending the above example to use the hook arguments, we could rewrite something like (now from declaration to usage, to get the whole picture):

```
\NewHookWithArguments{test}{1}
\AddToHookWithArguments{test}{%
  \typeout{Defining foo with "#1"}
  \def\foo##1{Hello, ##1! Some text after: #1}%
}
\UseHook{test}{Howdy!}
\ShowCommand\foo
```

Running the code above prints in the terminal:

```
Defining foo with "Howdy!"
> \foo=macro:
#1->Hello, #1! Some text after: Howdy!.
```

[6] The hook *will* take the declared number of arguments, and what will happen depends on what was grabbed, and what the hook code does with its arguments.

Note how `##1` in the call to `\AddToHookWithArguments` became `#1`, and the `#1` was replaced by the argument passed to the hook. Should the hook be used again, with a different argument, the definition would naturally change.

It is possible to add code referencing a hook's arguments before such hook is declared and the number of hooks is fixed. However, if some code is added to the hook, that references more arguments than will be declared for the hook, there will be a low-level TEX error about an "Illegal parameter number" at the time the hook is declared, which will be hard to track down because at that point TEX can't know whence the offending code came from. Thus it is important that package writers explicitly document how many arguments (if any) each hook can take, so users of those packages know how many arguments can be referenced, and equally important, what each argument means.

1.2.8 Private LATEX kernel hooks

There are a few places where it is absolutely essential for LATEX to function correctly that code is executed in a precisely defined order. Even that could have been implemented with the hook management (by adding various rules to ensure the appropriate ordering with respect to other code added by packages). However, this makes every document unnecessary slow, because there has to be sorting even though the result is predetermined. Furthermore it forces package writers to unnecessarily add such rules if they add further code to the hook (or break LATEX).

For that reason such code is not using the hook management, but instead private kernel commands directly before or after a public hook with the following naming convention: `\@kernel@before@`⟨*hook*⟩ or `\@kernel@after@`⟨*hook*⟩. For example, in `\enddocument` you find

```
\UseHook{enddocument}%
\@kernel@after@enddocument
```

which means first the user/package-accessible `enddocument` hook is executed and then the internal kernel hook. As their name indicates these kernel commands should not be altered by third-party packages, so please refrain from that in the interest of stability and instead use the public hook next to it.[7]

1.2.9 Legacy LATEX 2ε interfaces

LATEX 2ε offered a small number of hooks together with commands to add to them. They are listed here and are retained for backwards compatibility.

With the new hook management, several additional hooks have been added to LATEX and more will follow. See the next section for what is already available.

[7] As with everything in TEX there is not enforcement of this rule, and by looking at the code it is easy to find out how the kernel adds to them. The main reason of this section is therefore to say "please don't do that, this is unconfigurable code!"

\AtBeginDocument `\AtBeginDocument [⟨label⟩] {⟨code⟩}`

If used without the optional argument ⟨*label*⟩, it works essentially like before, i.e., it is adding ⟨*code*⟩ to the hook `begindocument` (which is executed inside `\begin{document}`). However, all code added this way is labeled with the label `top-level` (see section 1.2.1) if done outside of a package or class or with the package/class name if called inside such a file (see section 1.2.1).

This way one can add code to the hook using `\AddToHook` or `\AtBeginDocument` using a different label and explicitly order the code chunks as necessary, e.g., run some code before or after another package's code. When using the optional argument the call is equivalent to running `\AddToHook {begindocument} [`⟨*label*⟩`] {`⟨*code*⟩`}`.

`\AtBeginDocument` is a wrapper around the `begindocument` hook (see section 1.3.2), which is a one-time hook. As such, after the `begindocument` hook is executed at `\begin{document}` any attempt to add ⟨*code*⟩ to this hook with `\AtBeginDocument` or with `\AddToHook` will cause that ⟨*code*⟩ to execute immediately instead. See section 1.2.5 for more on one-time hooks.

For important packages with known order requirement we may over time add rules to the kernel (or to those packages) so that they work regardless of the loading-order in the document.

\AtEndDocument `\AtEndDocument [⟨label⟩] {⟨code⟩}`

Like `\AtBeginDocument` but for the `enddocument` hook.

The few hooks that existed previously in LaTeX 2_ε used internally commands such as `\@begindocumenthook` and packages sometimes augmented them directly rather than working through `\AtBeginDocument`. For that reason there is currently support for this, that is, if the system detects that such an internal legacy hook command contains code it adds it to the new hook system under the label `legacy` so that it doesn't get lost.

However, over time the remaining cases of direct usage need updating because in one of the future release of LaTeX we will turn this legacy support off, as it does unnecessary slow down the processing.

1.3 LaTeX 2_ε commands and environments augmented by hooks

In this section we describe the standard hooks that are now offered by LaTeX, or give pointers to other documents in which they are described. This section will grow over time (and perhaps eventually move to usrguide3).

1.3.1 Generic hooks

As stated earlier, with the exception of generic hooks, all hooks must be declared with `\NewHook` before they can be used. All generic hooks have names of the form "⟨*type*⟩/⟨*name*⟩/⟨*position*⟩", where ⟨*type*⟩ is from the predefined list shown below, and ⟨*name*⟩ is the variable part whose meaning will depend on the ⟨*type*⟩. The last component, ⟨*position*⟩, has more complex possibilities: it can always be `before` or `after`; for `env` hooks, it can also be `begin` or `end`; and for `include` hooks it can also be `end`. Each specific hook is documented below, or in `ltcmdhooks-doc.pdf` or `ltfilehook-doc.pdf`.

The generic hooks provided by LaTeX belong to one of the six types:

env Hooks executed before and after environments – ⟨*name*⟩ is the name of the environment, and available values for ⟨*position*⟩ are `before`, `begin`, `end`, and `after`;

cmd Hooks added to and executed before and after commands – ⟨*name*⟩ is the name of the command, and available values for ⟨*position*⟩ are `before` and `after`;

file Hooks executed before and after reading a file – ⟨*name*⟩ is the name of the file (with extension), and available values for ⟨*position*⟩ are `before` and `after`;

package Hooks executed before and after loading packages – ⟨*name*⟩ is the name of the package, and available values for ⟨*position*⟩ are `before` and `after`;

class Hooks executed before and after loading classes – ⟨*name*⟩ is the name of the class, and available values for ⟨*position*⟩ are `before` and `after`;

include Hooks executed before and after `\included` files – ⟨*name*⟩ is the name of the included file (without the `.tex` extension), and available values for ⟨*position*⟩ are `before`, `end`, and `after`.

Each of the hooks above are detailed in the following sections and in linked documentation.

Generic hooks for all environments

Every environment ⟨*env*⟩ has now four associated hooks coming with it:

`env/`⟨*env*⟩`/before` This hook is executed as part of `\begin` as the very first action, in particular prior to starting the environment group. Its scope is therefore not restricted by the environment.

`env/`⟨*env*⟩`/begin` This hook is executed as part of `\begin` directly in front of the code specific to the environment start (e.g., the third argument of `\NewDocumentEnvironment` and the second argument of `\newenvironment`). Its scope is the environment body.

`env/`⟨*env*⟩`/end` This hook is executed as part of `\end` directly in front of the code specific to the end of the environment (e.g., the forth argument of `\NewDocumentEnvironment` and the third argument of `\newenvironment`).

`env/`⟨*env*⟩`/after` This hook is executed as part of `\end` after the code specific to the environment end and after the environment group has ended. Its scope is therefore not restricted by the environment.

The hook is implemented as a reversed hook so if two packages add code to `env/`⟨*env*⟩`/before` and to `env/`⟨*env*⟩`/after` they can add surrounding environments and the order of closing them happens in the right sequence.

Given that these generic hook names involve `/` as part of their name they would not work if one tries to define an environment using a name that involves a `/`.[8]

Generic environment hooks are never one-time hooks even with environments that are supposed to appear only once in a document.[9] In contrast to other hooks there is also no need to declare them using `\NewHook`.

[8] Officially, LaTeX names for environments should only consist of a sequence of letters, numbers, and the character `*`, i.e., this is not a new restriction.

[9] Thus if one adds code to such hooks after the environment has been processed, it will only be executed if the environment appears again and if that doesn't happen the code will never get executed.

The hooks are only executed if `\begin{`⟨*env*⟩`}` and `\end{`⟨*env*⟩`}` is used. If the environment code is executed via low-level calls to `\`⟨*env*⟩ and `\end`⟨*env*⟩ (e.g., to avoid the environment grouping) they are not available. If you want them available in code using this method, you would need to add them yourself, i.e., write something like

```
\UseHook{env/quote/before}\quote
    ...
\endquote\UseHook{env/quote/after}
```

to add the outer hooks, etc.

Largely for compatibility with existing packages, the following four commands are also available to set the environment hooks; but for new packages we recommend directly using the hook names and `\AddToHook`.

`\BeforeBeginEnvironment` `\BeforeBeginEnvironment [`⟨*label*⟩`] {`⟨*env*⟩`} {`⟨*code*⟩`}`

This declaration adds to the `env/`⟨*env*⟩`/before` hook using the ⟨*label*⟩. If ⟨*label*⟩ is not given, the ⟨*default label*⟩ is used (see section 1.2.1).

`\AtBeginEnvironment` `\AtBeginEnvironment [`⟨*label*⟩`] {`⟨*env*⟩`} {`⟨*code*⟩`}`

This is like `\BeforeBeginEnvironment` but it adds to the `env/`⟨*env*⟩`/begin` hook.

`\AtEndEnvironment` `\AtEndEnvironment [`⟨*label*⟩`] {`⟨*env*⟩`} {`⟨*code*⟩`}`

This is like `\BeforeBeginEnvironment` but it adds to the `env/`⟨*env*⟩`/end` hook.

`\AfterEndEnvironment` `\AfterEndEnvironment [`⟨*label*⟩`] {`⟨*env*⟩`} {`⟨*code*⟩`}`

This is like `\BeforeBeginEnvironment` but it adds to the `env/`⟨*env*⟩`/after` hook.

Generic hooks for commands

Similar to environments there are now (at least in theory) two generic hooks available for any LaTeX command. These are

`cmd/`⟨*name*⟩`/before` This hook is executed at the very start of the command execution.

`cmd/`⟨*name*⟩`/after` This hook is executed at the very end of the command body. It is implemented as a reversed hook.

In practice there are restrictions and especially the `after` hook works only with a subset of commands. Details about these restrictions are documented in `ltcmdhooks-doc.pdf` or with code in `ltcmdhooks-code.pdf`.

Generic hooks provided by file loading operations

There are several hooks added to LaTeX's process of loading file via its high-level interfaces such as `\input`, `\include`, `\usepackage`, `\RequirePackage`, etc. These are documented in `ltfilehook-doc.pdf` or with code in `ltfilehook-code.pdf`.

1.3.2 Hooks provided by \begin{document}

Until 2020 \begin{document} offered exactly one hook that one could add to using \AtBeginDocument. Experiences over the years have shown that this single hook in one place was not enough and as part of adding the general hook management system a number of additional hooks have been added at this point. The places for these hooks have been chosen to provide the same support as offered by external packages, such as etoolbox and others that augmented \document to gain better control.

Supported are now the following hooks (all of them one-time hooks):

begindocument/before This hook is executed at the very start of \document, one can think of it as a hook for code at the end of the preamble section and this is how it is used by etoolbox's \AtEndPreamble.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 1.2.5).

begindocument This hook is added to by using \AddToHook{begindocument} or by using \AtBeginDocument and it is executed after the .aux file has been read and most initialization are done, so they can be altered and inspected by the hook code. It is followed by a small number of further initializations that shouldn't be altered and are therefore coming later.

The hook should not be used to add material for typesetting as we are still in LaTeX's initialization phase and not in the document body. If such material needs to be added to the document body use the next hook instead.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 1.2.5).

begindocument/end This hook is executed at the end of the \document code in other words at the beginning of the document body. The only command that follows it is \ignorespaces.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 1.2.5).

The generic hooks executed by \begin also exist, i.e., env/document/before and env/document/begin, but with this special environment it is better use the dedicated one-time hooks above.

1.3.3 Hooks provided by \end{document}

LaTeX 2$_\varepsilon$ has always provided \AtEndDocument to add code to the \end{document}, just in front of the code that is normally executed there. While this was a big improvement over the situation in LaTeX 2.09, it was not flexible enough for a number of use cases and so packages, such as etoolbox, atveryend and others patched \enddocument to add additional points where code could be hooked into.

Patching using packages is always problematical as leads to conflicts (code availability, ordering of patches, incompatible patches, etc.). For this reason a number of additional hooks have been added to the \enddocument code to allow packages to add code in various places in a controlled way without the need for overwriting or patching the core code.

Supported are now the following hooks (all of them one-time hooks):

enddocument The hook associated with `\AtEndDocument`. It is immediately called at the beginning of `\enddocument`.

When this hook is executed there may be still unprocessed material (e.g., floats on the deferlist) and the hook may add further material to be typeset. After it, `\clearpage` is called to ensure that all such material gets typeset. If there is nothing waiting the `\clearpage` has no effect.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 1.2.5).

enddocument/afterlastpage As the name indicates this hook should not receive code that generates material for further pages. It is the right place to do some final housekeeping and possibly write out some information to the `.aux` file (which is still open at this point to receive data). Just before this hook `\write` is redefined and is now always `\immediate\write`. This means that writing e.g. a label or something to the `.toc` works despite the fact that no more pages are output. It is also the correct place to set up any testing code to be run when the `.aux` file is re-read in the next step.

After this hook has been executed the `.aux` file is closed for writing and then read back in to do some tests (e.g., looking for missing references or duplicated labels, etc.).

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 1.2.5).

enddocument/afteraux At this point, the `.aux` file has been reprocessed and so this is a possible place for final checks and display of information to the user. However, for the latter you might prefer the next hook, so that your information is displayed after the (possibly longish) list of files if that got requested via `\listfiles`.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 1.2.5).

enddocument/info This hook is meant to receive code that write final information messages to the terminal. It follows immediately after the previous hook (so both could have been combined, but then packages adding further code would always need to also supply an explicit rule to specify where it should go.

This hook already contains some code added by the kernel (under the labels `kernel/filelist` and `kernel/warnings`), namely the list of files when `\listfiles` has been used and the warnings for duplicate labels, missing references, font substitutions etc.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 1.2.5).

enddocument/end Finally, this hook is executed just in front of the final call to `\@@end`.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 1.2.5).is it even possible to add code after this one?

There is also the hook `shipout/lastpage`. This hook is executed as part of the last `\shipout` in the document to allow package to add final `\special`'s to that page. Where this hook is executed in relation to those from the above list can vary from document to

document. Furthermore to determine correctly which of the `\shipout`s is the last one, LaTeX needs to be run several times, so initially it might get executed on the wrong page. See section 1.3.4 for where to find the details.

It is in also possible to use the generic `env/document/end` hook which is executed by `\end`, i.e., just in front of the first hook above. Note however that the other generic `\end` environment hook, i.e., `env/document/after` will never get executed, because by that time LaTeX has finished the document processing.

1.3.4 Hooks provided by `\shipout` operations

There are several hooks and mechanisms added to LaTeX's process of generating pages. These are documented in `ltshipout-doc.pdf` or with code in `ltshipout-code.pdf`.

1.3.5 Hooks provided for paragraphs

The paragraph processing has been augmented to include a number of internal and public hooks. These are documented in `ltpara-doc.pdf` or with code in `ltpara-code.pdf`.

1.3.6 Hooks provided in NFSS commands

In languages that need to support for more than one script in parallel (and thus several sets of fonts, e.g., supporting both Latin and Japanese fonts), NFSS font commands such as `\sffamily` need to switch both the Latin family to "Sans Serif" and in addition alter a second set of fonts.

To support this, several NFSS commands have hooks to which such support can be added.

`rmfamily` After `\rmfamily` has done its initial checks and prepared a font series update, this hook is executed before `\selectfont`.

`sffamily` This is like the `rmfamily` hook, but for the `\sffamily` command.

`ttfamily` This is like the `rmfamily` hook, but for the `\ttfamily` command.

`normalfont` The `\normalfont` command resets the font encoding, family, series and shape to their document defaults. It then executes this hook and finally calls `\selectfont`.

`expand@font@defaults` The internal `\expand@font@defaults` command expands and saves the current defaults for the metafamilies (rm/sf/tt) and the metaseries (bf/md). If the NFSS machinery has been augmented, e.g., for Chinese or Japanese fonts, then further defaults may need to be set at this point. This can be done in this hook which is executed at the end of this macro.

`bfseries/defaults`, `bfseries` If the `\bfdefault` was explicitly changed by the user, its new value is used to set the bf series defaults for the metafamilies (rm/sf/tt) when `\bfseries` is called. The `bfseries/defaults` hook allows further adjustments to be made in this case. This hook is only executed if such a change is detected. In contrast, the `bfseries` hook is always executed just before `\selectfont` is called to change to the new series.

`mdseries/defaults`, `mdseries` These two hooks are like the previous ones but they are in the `\mdseries` command.

selectfont This hook is executed inside `\selectfont`, after the current values for *encoding*, *family*, *series*, *shape*, and *size* are evaluated and the new font is selected (and if necessary loaded). After the hook has executed, NFSS will still do any updates necessary for a new *size* (such as changing the size of `\strut`) and any updates necessary to a change in *encoding*.

This hook is intended for use cases where, in parallel to a change in the main font, some other fonts need to be altered (e.g., in CJK processing where you may need to deal with several different alphabets).

1.3.7 Hook provided by the mark mechanism

See `ltmarks-doc.pdf` for details.

insertmark This hook allows for a special setup while `\InsertMark` inserts a mark. It is executed in group so local changes only apply to the mark being inserted.

Chapter 2

LaTeX's hook management for files

2.1 Introduction

2.1.1 Provided hooks

The code offers a number of hooks into which packages (or the user) can add code to support different use cases. Many hooks are offered as pairs (i.e., the second hook is reversed. Also important to know is that these pairs are properly nested with respect to other pairs of hooks.

There are hooks that are executed for all files of a certain type (if they contain code), e.g., for all "include files" or all "packages", and there are also hooks that are specific to a single file, e.g., do something after the package `foo.sty` has been loaded.

2.1.2 General hooks for file reading

There are four hooks that are called for each file that is read using document-level commands such as `\input`, `\include`, `\usepackage`, etc. They are not called for files read using internal low-level methods, such as `\@input` or `\openin`.

`file/before`
`file/.../before`
`file/.../after`
`file/after`

These are:

`file/before`, `file/`⟨*file-name*⟩`/before` These hooks are executed in that order just before the file is loaded for reading. The code of the first hook is used with every file, while the second is executed only for the file with matching ⟨*file-name*⟩ allowing you to specify code that only applies to one file.

`file/`⟨*file-name*⟩`/after`, `file/after` These hooks are executed after the file with name ⟨*file-name*⟩ has been fully consumed. The order is swapped (the specific one comes first) so that the `/before` and `/after` hooks nest properly, which is important if any of them involve grouping (e.g., contain environments, for example). Furthermore both hooks are reversed hooks to support correct nesting of different packages adding code to both `/before` and `/after` hooks.

So the overall sequence of hook processing for any file read through the user interface commands of LaTeX is:

```
\UseHook{file/before}
\UseHook{file/⟨file name⟩/before}
   ⟨file contents⟩
\UseHook{file/⟨file name⟩/after}
\UseHook{file/after}
```

The file hooks only refer to the file by its name and extension, so the ⟨*file name*⟩ should be the file name as it is on the filesystem with extension (if any) and without paths. Different from `\input` and similar commands, the `.tex` extension is not assumed in hook ⟨*file name*⟩, so `.tex` files must be specified with their extension to be recognized. Files within subfolders should also be addressed by their name and extension only.

Extensionless files also work, and should then be given without extension. Note however that TeX prioritizes `.tex` files, so if two files `foo` and `foo.tex` exist in the search path, only the latter will be seen.

When a file is input, the ⟨*file name*⟩ is available in `\CurrentFile`, which is then used when accessing the `file/`⟨*file name*⟩`/before` and `file/`⟨*file name*⟩`/after`.

`\CurrentFile` The name of the file about to be read (or just finished) is available to the hooks through `\CurrentFile` (there is no `expl3` name for it for now). The file is always provided with its extension, i.e., how it appears on your hard drive, but without any specified path to it. For example, `\input{sample}` and `\input{app/sample.tex}` would both have `\CurrentFile` being `sample.tex`.

`\CurrentFilePath` The path to the current file (complement to `\CurrentFile`) is available in `\CurrentFilePath` if needed. The paths returned in `\CurrentFilePath` are only user paths, given through `\input@path` (or `expl3`'s equivalent `\l_file_search_path_seq`) or by directly typing in the path in the `\input` command or equivalent. Files located by `kpsewhich` get the path added internally by the TeX implementation, so at the macro level it looks as if the file were in the current folder, so the path in `\CurrentFilePath` is empty in these cases (package and class files, mostly).

`\CurrentFileUsed` `\CurrentFilePathUsed` In normal circumstances these are identical to `\CurrentFile` and `\CurrentFilePath`. They will differ when a file substitution has occurred for `\CurrentFile`. In that case, `\CurrentFileUsed` and `\CurrentFilePathUsed` will hold the actual file name and path loaded by LaTeX, while `\CurrentFile` and `\CurrentFilePath` will hold the names that were *asked for*. Unless doing very specific work on the file being read, `\CurrentFile` and `\CurrentFilePath` should be enough.

2.1.3 Hooks for package and class files

Commands to load package and class files (e.g., `\usepackage`, `\RequirePackage`, `\LoadPackageWithOptions`, etc.) offer the hooks from section 2.1.2 when they are used to load a package or class file, e.g., `file/array.sty/after` would be called after the `array` package got loaded. But as packages and classes form as special group of files, there are some additional hooks available that only apply when a package or class is loaded.

package/before
package/after
package/.../before
package/.../after
class/before
class/after
class/.../before
class/.../after

These are:

package/before, package/after These hooks are called for each package being loaded.

package/⟨*name*⟩/before, package/⟨*name*⟩/after These hooks are additionally called if the package name is ⟨*name*⟩ (without extension).

class/before, class/after These hooks are called for each class being loaded.

class/⟨*name*⟩/before, class/⟨*name*⟩/after These hooks are additionally called if the class name is ⟨*name*⟩ (without extension).

All `/after` hooks are implemented as reversed hooks.

The overall sequence of execution for `\usepackage` and friends is:

```
\UseHook{package/before}
\UseOneTimeHook{package/⟨package name⟩/before}
   \UseHook{file/before}
   \UseHook{file/⟨package name⟩.sty/before}
      ⟨package contents⟩
   \UseHook{file/⟨package name⟩.sty/after}
   \UseHook{file/after}
   code from \AtEndOfPackage if used inside the package
\UseOneTimeHook{package/⟨package name⟩/after}
\UseHook{package/after}
```

and similar for class file loading, except that `package/` is replaced by `class/` and `\AtEndOfPackage` by `\AtEndOfClass`.

If a package or class is not loaded none of the hooks are executed!

All class or package hooks involving the name of the class or package are implemented as one-time hooks, whereas all other such hooks are normal hooks. This allows for the following use case

```
\AddToHook{package/varioref/after}
      { ... apply my customizations if the package gets
        loaded (or was loaded already) ... }
```

without the need to first test if the package is already loaded.

2.1.4 Hooks for `\include` files

To manage `\include` files, LaTeX issues a `\clearpage` before and after loading such a file. Depending on the use case one may want to execute code before or after these `\clearpage`s especially for the one that is issued at the end.

Executing code before the final `\clearpage`, means that the code is processed while the last page of the included material is still under construction. Executing code after it means that all floats from inside the include file are placed (which might have added further pages) and the final page has finished.

Because of these different scenarios we offer hooks in three places.[10] None of the hooks are executed when an `\include` file is bypassed because of an `\includeonly` declaration. They are, however, all executed if LaTeX makes an attempt to load the `\include` file (even if it doesn't exist and all that happens is "`No file ⟨filename⟩.tex`").

```
include/before
include/.../before
include/end
include/.../end
include/after
include/.../after
```

These are:

include/before, include/⟨name⟩/before These hooks are executed (in that order) after the initial `\clearpage` and after `.aux` file is changed to use ⟨*name*⟩`.aux`, but before the ⟨*name*⟩`.tex` file is loaded. In other words they are executed at the very beginning of the first page of the `\include` file.

include/⟨name⟩/end, include/end These hooks are executed (in that order) after LaTeX has stopped reading from the `\include` file, but before it has issued a `\clearpage` to output any deferred floats.

include/⟨name⟩/after, include/after These hooks are executed (in that order) after LaTeX has issued the `\clearpage` but before is has switched back writing to the main `.aux` file. Thus technically we are still inside the `\include` and if the hooks generate any further typeset material including anything that writes to the `.aux` file, then it would be considered part of the included material and bypassed if it is not loaded because of some `\includeonly` statement.[11]

include/excluded, include/⟨name⟩/excluded The above hooks for `\include` files are only executed when the file is loaded (or more exactly the load is attempted). If, however, the `\include` file is explicitly excluded (through an `\includeonly` statement) the above hooks are bypassed and instead the `include/excluded` hook followed by the `include/⟨name⟩/excluded` hook are executed. This happens after LaTeX has loaded the `.aux` file for this include file, i.e., after LaTeX has updated its counters to pretend that the file was seen.

All `include` hooks involving the name of the included file are implemented as one-time hooks (whereas all other such hooks are normal hooks).

If you want to execute code that is run for every `\include` regardless of whether or not it is excluded, use the `cmd/include/before` or `cmd/include/after` hooks.

2.1.5 High-level interfaces for LaTeX

We do not provide any additional wrappers around the hooks (like filehook or scrlfile do) because we believe that for package writers the high-level commands from the hook management, e.g., `\AddToHook`, etc. are sufficient and in fact easier to work with, given that the hooks have consistent naming conventions.

[10] If you want to execute code before the first `\clearpage` there is no need to use a hook—you can write it directly in front of the `\include`.

[11] For that reason another `\clearpage` is executed after these hooks which normally does nothing, but starts a new page if further material got added this way.

2.1.6 Kernel, class, and package interfaces for LaTeX

\declare@file@substitution `\declare@file@substitution {⟨file⟩} {⟨replacement-file⟩}`
\undeclare@file@substitution `\undeclare@file@substitution {⟨file⟩}`

If ⟨*file*⟩ is requested for loading replace it with ⟨*replacement-file*⟩. `\CurrentFile` remains pointing to ⟨*file*⟩ but `\CurrentFileUsed` will show the file actually loaded.

The main use case for this declaration is to provide a corrected version of a package that can't be changed (due to its license) but no longer functions because of LaTeX kernel changes, for example, or to provide a version that makes use of new kernel functionality while the original package remains available for use with older releases. As such it is mainly meant for use in the LaTeX kernel but other use cases are conceivable.

The `\undeclare@file@substitution` declaration undoes a substitution made earlier.

> *Please do not misuse this functionality and replace a file with another unless if really needed and only if the new version is implementing the same functionality as the original one!*

\disable@package@load `\disable@package@load {⟨package⟩} {⟨alternate-code⟩}`
\reenable@package@load `\reenable@package@load {⟨package⟩}`

If ⟨*package*⟩ is requested, do not load it but instead run ⟨*alternate-code*⟩ which could issue a warning, error or any other code.

The main use case is for classes that want to restrict the set of supported packages or contain code that make the use of some packages impossible. So rather than waiting until the document breaks they can set up informative messages why certain packages are not available.

The function is only implemented for packages not for arbitrary files and again it should only be applied if there are good reasons for doing this.[12]

2.1.7 A sample package for structuring the log output

As an application we provide the package structuredlog that adds lines to the `.log` when a file is opened and closed for reading keeping track of nesting level es well. For example, for the current document it adds the lines

```
= (LEVEL 1 START) t1lmr.fd
= (LEVEL 1 STOP) t1lmr.fd
= (LEVEL 1 START) supp-pdf.mkii
= (LEVEL 1 STOP) supp-pdf.mkii
= (LEVEL 1 START) nameref.sty
== (LEVEL 2 START) refcount.sty
== (LEVEL 2 STOP) refcount.sty
== (LEVEL 2 START) gettitlestring.sty
== (LEVEL 2 STOP) gettitlestring.sty
= (LEVEL 1 STOP) nameref.sty
= (LEVEL 1 START) ltfilehook-doc.out
```

[12] Just to be sure: "I don't like this package by somebody else" is not a good one :-)

```
= (LEVEL 1 STOP) ltfilehook-doc.out
= (LEVEL 1 START) ltfilehook-doc.out
= (LEVEL 1 STOP) ltfilehook-doc.out
= (LEVEL 1 START) ltfilehook-doc.hd
= (LEVEL 1 STOP) ltfilehook-doc.hd
= (LEVEL 1 START) ltfilehook.dtx
== (LEVEL 2 START) ot1lmr.fd
== (LEVEL 2 STOP) ot1lmr.fd
== (LEVEL 2 START) omllmm.fd
== (LEVEL 2 STOP) omllmm.fd
== (LEVEL 2 START) omslmsy.fd
== (LEVEL 2 STOP) omslmsy.fd
== (LEVEL 2 START) omxlmex.fd
== (LEVEL 2 STOP) omxlmex.fd
== (LEVEL 2 START) umsa.fd
== (LEVEL 2 STOP) umsa.fd
== (LEVEL 2 START) umsb.fd
== (LEVEL 2 STOP) umsb.fd
== (LEVEL 2 START) ts1lmr.fd
== (LEVEL 2 STOP) ts1lmr.fd
== (LEVEL 2 START) t1lmss.fd
== (LEVEL 2 STOP) t1lmss.fd
= (LEVEL 1 STOP) ltfilehook.dtx
```

Thus if you inspect an issue in the `.log` it is easy to figure out in which file it occurred, simply by searching back for `LEVEL` and if it is a `STOP` then remove 1 from the level value and search further for `LEVEL` with that value which should then be the `START` level of the file you are in.

Chapter 3

Hook management for commands

3.1 Introduction

This file implements generic hooks for (arbitrary) commands. In theory every command \⟨*name*⟩ offers now two associated hooks to which code can be added using `\AddToHook`,[13] `\AddToHookNext`, `\AddToHookWithArguments`, and `\AddToHookNextWithArguments`.[14]

However, this is only true "in theory". In practice there are a number of restrictions that makes it impossible to use such generic command hooks in a number of cases, so please read all of section 3.2 to understand what may prevent you from using them successfully.

The generic command hooks are:

`cmd/`⟨*name*⟩`/before` This hook is executed at the very start of the command, right after its arguments (if any) are parsed. The hook ⟨*code*⟩ runs in the command inside a call to `\UseHookWithArguments`. Any code added to this hook using `\AddToHookWithArguments` or `\AddToHookNextWithArguments` can access the command's arguments using `#1`, `#2`, etc., up to the number of arguments of the command. If `\AddToHook` or `\AddToHookNext` are used, the arguments cannot be accessed (see the lthooks documentation[15] on hooks with arguments).

`cmd/`⟨*name*⟩`/after` This hook is similar to `cmd/`⟨*name*⟩`/before`, but it is executed at the very end of the command body. This hook is implemented as a reversed hook.

The hooks are not physically present before `\begin{document}`[16] (i.e., using a command in the preamble will never execute the hook) and if nobody has declared any code for them, then they are not added to the command code ever. For example, if we have the following definition

[13] In this documentation, when something is being said about `\AddToHook`, the same will be valid for `\AddToHookWithArguments`, unless that particular paragraph is highlighting the differences between both. The same is true for the other hook-related functions and their `...WithArguments` counterparts.

[14] In practice this is not supported for all types of commands, see section 3.2.4 for the restrictions that apply and what happens if one tries to use this with commands for which this is not supported.

[15] `texdoc lthooks-doc`

[16] More specifically, they are inserted in the commands after the `begindocument` hook, so they are also not present while LaTeX is reading the `.aux` file.

```
\newcommand\foo[2]{Code #1 for #2!}
```

then executing `\foo{A}{B}` will simply run `Code␣A␣for␣B!` as it was always the case. However, if somebody, somewhere (e.g., in a package) adds

```
\AddToHook{cmd/foo/before}{<before code>}
```

then, after `\begin{document}` the definition of `\foo` will be:

```
\renewcommand\foo[2]{%
  \UseHookWithArguments{cmd/foo/before}{2}{#1}{#2}%
  Code #1 for #2!}
```

and similarly `\AddToHook{cmd/foo/after}{<after code>}` alters the definition to

```
\renewcommand\foo[2]{%
  Code #1 for #2!%
  \UseHookWithArguments{cmd/foo/after}{2}{#1}{#2}}
```

In other words, the mechanism is similar to what etoolbox offers with `\pretocmd` and `\apptocmd` with the important differences

- that code can be prepended or appended (i.e., added to the hooks) even if the command itself is not (yet) defined, because the defining package has not been loaded at this point;
- and that by using the hook management interface it is now possible to define how the code chunks added in these places are ordered, if different packages want to add code at these points.

3.2 Restrictions and operational details

Adding arbitrary material to commands is tricky because most of the time we do not know what the macro expects as arguments when expanding and TEX doesn't have a reliable way to see that, so some guesswork has to be employed.

We can do this in most cases when commands are defined using `\NewDocumentCommand` or `\newcommand` (with a few exceptions). For commands defined with `\def` the situation is less good. Common cases where the command hooks will not work are:

- Commands that use special catcode settings within their definition. In that case it is usually not possible to augment the definition (see 3.2.1).
- If a command is defined while `\ExplSyntaxOn` is in force **and** the command contains `~` characters to represent spaces, then it can't be patched to include the command hooks. In fact in some very special circumstances you might even get a low-level error rather than the information that the command can't be patched (see, for example, https://github.com/latex3/latex2e/issues/1430.
- Commands that have arguments as far as the user is concerned (e.g., `\section` or `\caption`), but are defined in a way that these arguments are not read by the user level command but only later during the processing. In that case the `after` hook doesn't work at all. The before hook only works with `\AddToHook` but not with `\AddToHookWithArguments` because the arguments haven't been read at that point where the hook is patched in. See section 3.2.4.

- Adding a specific generic command hook is only attempted once per command, thus after redefining a command such hooks will no longer be there and will also not being re-added, see section 3.2.1.

All this means that you have to have a good understanding of how commands are defined when you attempt to make use of such hooks and something goes wrong. What can help in that case is to turn on `\DebugHooksOn` in which case you get much more (low-level) details on why something fails and what was tried to enable the hooks.

3.2.1 Patching

The code here tries to find out if a command was defined with `\newcommand` or `\DeclareRobustCommand` or `\NewDocumentCommand`, and if so it *assumes* that the argument specification of the command is as expected (which is not fail-proof, if someone redefines the internals of these commands in devious ways, but is a reasonable assumption).

If the command is one of the defined types, the code here does a sandboxed expansion of the command such that it can be redefined again exactly as before, but with the hook code added.

If however the command is not a known type (it was defined with `\def`, for example), then the code uses an approach similar to etoolbox's `\patchcmd` to retokenize the command with the hook code in place. This procedure, however, is more likely to fail if the catcode settings are not the same as the ones at the time of command's definition, so not always adding a hook to a command will work.

Timing

When `\AddToHook` (or its expl3 equivalent) is called with a generic `cmd` hook, say, `cmd/foo/before`, for the first time (that is, no code was added to that same hook before), in the preamble of a document, it will store a patch instruction for that command until `\begin{document}`, and only then all the commands which had hooks added will be patched in one go. That means that no command in the preamble will have hooks patched into them.

At `\begin{document}` all the delayed patches will be executed, and if the command doesn't exist the code is still added to the hook, but it will not be executed. After `\begin{document}`, when `\AddToHook` is called with a generic `cmd` hook the first time, the command will be immediately patched to include the hook, and if it doesn't exist or if it can't be patched for any reason, an error is thrown; if `\AddToHook` was already used in the preamble no new patching is attempted.

This has the consequence that a command defined or redefined after `\begin{document}` only uses generic `cmd` hook code if `\AddToHook` is called for the first time after the definition is made, or if the command explicitly uses the generic hook in its definition by declaring it with `\NewHookPair` adding `\UseHook` as part of the code.[17]

3.2.2 Command copies

Once a hook has been added to a command, it will be present in any copies of that command which are made. For example, if in the preamble we have

[17] We might change this behavior in the main document slightly after gaining some usage experience.

```
\NewDocumentCommand\foo{}{}
\AddToHook{cmd/foo/after}{1}%
```

then after `\begin{document}`, any use of `\NewCommandCopy` will include the hook in the copy, for example

```
\NewCommandCopy\baz\foo
```

would include the hook in this copied command.

3.2.3 Grouping

Command hooks are intended to be added to document commands, which are typically defined globally. As such, adding a hook is a global operation, even if the command was previously only locally-defined. Addition of material to hooks is also global.

3.2.4 Commands that look ahead

Some commands are defined in different "steps" and they look ahead in the input stream to find more arguments. If you try to add some code to the `cmd/`⟨*name*⟩`/after` hook of such command, it will not work, and it is not possible to detect that programmatically, so the user has to know (or find out) which commands can or cannot have hooks attached to them.

One good example is the `\section` command. You can add something to the `cmd/section/before` hook (but only with `\AddToHook` not `\AddToHookWithArguments`), but if you try to add anything to the `cmd/section/after` hook, `\section` will no longer work at all. That happens because the `\section` macro takes no argument, but instead calls a few internal LaTeX macros to look for the optional and mandatory arguments. By adding code to the `cmd/section/after` hook, you get in the way of that scanning.

In such a case, where it is known that a specific generic command hook does not work if code is added to it, the package author can add a `\DisableGenericHook`[18] declaration to prevent this from happening in user documents and thereby avoiding obscure errors.

3.3 Package author interface

The `cmd` hooks are, by default, available for all commands that can be patched to add the hooks. For some commands, however, the very beginning or the very end of the code is not the best place to put the hooks, for example, if the command looks ahead for arguments (see section 3.2.4).

If you are a package author and you want to add the hooks to your own commands in the proper position you can define the command and manually add the `\UseHookWithArguments` calls inside the command in the proper positions, and manually define the hooks with `\NewHookWithArguments` or `\NewReversedHookWithArguments`. When the hooks are explicitly defined, patching is not attempted so you can make sure your command works properly. For example, an (admittedly not really useful) command that typesets its contents in a framed box with width optionally given in parentheses:

```
\newcommand\fancybox{\@ifnextchar({\@fancybox}{\@fancybox(5cm)}}
\def\@fancybox(#1)#2{\fbox{\parbox{#1}{#2}}}
```

[18] Please use `\DisableGenericHook` if at all, only on hooks that you "own", i.e., for commands your package or class defines and not second guess whether or not hooks of other packages should get disabled!

If you try that definition, then add some code after it with

```
\AddToHook{cmd/fancybox/after}{<code>}
```

and then use the `\fancybox` command you will see that it will be completely broken, because the hook will get executed in the middle of parsing for optional `(...)` argument.

If, on the other hand, you want to add hooks to your command you can do something like:

```
\newcommand\fancybox{\@ifnextchar({\@fancybox}{\@fancybox(5cm)}}
\def\@fancybox(#1)#2{\fbox{%
               \UseHookWithArguments{cmd/fancybox/before}{2}{#1}{#2}%
               \parbox{#1}{#2}%
               \UseHookWithArguments{cmd/fancybox/after}{2}{#1}{#2}}}
\NewHookWithArguments{cmd/fancybox/before}{2}
\NewReversedHookWithArguments{cmd/fancybox/after}{2}
```

then the hooks will be executed where they should and no patching will be attempted. It is important that the hooks are declared with `\NewHookWithArguments` or `\NewReversedHookWithArguments`, otherwise the command hook code will try to patch the command. Note also that the call to `\UseHookWithArguments{cmd/fancybox/before}` does not need to be in the definition of `\fancybox`, but anywhere it makes sense to insert it (in this case in the internal `\@fancybox`).

Alternatively, if for whatever reason your command does not support the generic hooks provided here, you can disable a hook with `\DisableGenericHook`[19], so that when someone tries to add code to it they will get an error. Or if you don't want the error, you can simply declare the hook with `\NewHook` and never use it.

The above approach is useful for really complex commands where for one or the other reason the hooks can't be placed at the very beginning and end of the command body and some hand-crafting is needed. However, in the example above the real (and in fact only) issue is the cascading argument parsing in the style developed long ago in LaTeX 2.09. Thus, a much simpler solution for this case is to replace it with the modern `\NewDocumentCommand` syntax and define the command as follows:

```
\DeclareDocumentCommand\fancybox{D(){5cm}m}{\fbox{\parbox{#1}{#2}}}
```

If you do that then both hooks automatically work and are patched into the right places.

3.3.1 Arguments and redefining commands

The code in ltcmdhooks does its best to find out how many arguments a given command has, and to insert the appropriate call to `\UseHookWithArguments`, so that the arguments seen by the hook are exactly those grabbed by the command (the hook, after all, is a macro call, so the arguments have to be placed in the right order, or they won't match).

When using the package writer interface, as discussed in section 3.3, to change the position of the hooks in your commands, you are also free to change how the hook code in your command sees its arguments. When a `cmd` hook is declared with `\NewHook` (or `\NewHookWithArguments` or other variations of that), it loses its "generic" nature and works as a regular hook. This means that you may choose to declare it without arguments

[19] Please use `\DisableGenericHook` if at all, only on hooks that you "own", i.e., for commands your package or class defines and not second guess whether or not hooks of other packages should get disabled!

regardless if the command takes arguments or not, or declare it with arguments, even if the command takes none.

However, this flexibility should not be abused. When using a nonstandard configuration for the hook arguments, think reasonably: a user will expect that the argument `#1` in the hook corresponds to the argument's first argument, and so on. Any other configuration is likely to cause confusion and, if used, will have to be well documented.

This flexibility, however, allows you to "correct" the arguments for the hooks. For example, LaTeX's `\refstepcounter` has a single argument, the name of the counter. The cleveref package adds an optional argument to `\refstepcounter`, making the name of the counter argument `#2`. If the author of cleveref wanted, for whatever reason, to add hooks to `\refstepcounter`, to preserve compatibility he could write something along the lines of:

```
\NewHookWithArguments{cmd/refstepcounter/before}{1}
\renewcommand\refstepcounter[2][<default>]{%
  \UseHookWithArguments{cmd/refstepcounter/before}{1}{#2}%
  <code for \refstepcounter>}
```

so that the mandatory argument, which is arg `#2` in the definition, would still be seen as `#1` in the hook code.

Another possibility would be to place the optional argument as the second argument for the hook, so that people looking for it would be able to use it. In either case, it would have to be well documented to cause as little confusion as possible.

Chapter 4

Paragraph building and hooks

4.1 Introduction

The building of paragraphs in the TEX engine(s) has a number of peculiarities that makes it on one hand fairly flexible but on the other hand somewhat awkward to control or reliably to extend. Thus to better understand the code below we start with a brief introduction of the mechanism; for more details refer to the TEXbook [**?**, chap. 14] (for the full truth you may even have to study the program code).

4.1.1 The default processing done by the engine

TEX automatically starts building a paragraph when it is currently in vertical mode and encounters anything that can only live in horizontal mode. Most often this is a character, but there are also many commands that can be used only in horizontal mode. If any of them is encountered, TEX will immediately back up (i.e., the character or command is read later again), adds a `\parskip` glue to the current vertical list unless the list is empty, switches to horizontal mode, starts its special "start of paragraph processing" and only then rereads the character or command that caused the mode change.[20]

This "start of paragraph processing" first adds an empty box at the start of the horizontal list of width `\parindent` (which represents the paragraph indentation) unless the paragraph was started with `\noindent` in which case no such box is added[21]. It then reads and processes all tokens stored in the special engine token register `\everypar`. After that it reads and processes whatever has caused the paragraph to start.

Thus out of the box, TEX offers the possibility to put some special code into `\everypar` to gain control at (more or less) the start of the paragraph. For example, in LaTeX and a number of packages, special code like the following is sometimes used:

```
\everypar{{\setbox\z@\lastbox}\everypar{} ...}
```

This removes the paragraph indentation box again (that was already placed by TEX), then resets `\everypar` so that it doesn't do anything on the next paragraph start and then does whatever it wants to do, e.g., in an `\item` of a list it will typeset the label in front of the paragraph text. However, there is only one such `\everypar` token register and if

[20] Already not quite true: the command `\noindent` starts the paragraph but influences the special processing by suppressing the paragraph indentation box normally inserted by it.

[21] That's a bit different from placing a zero-sized box!

different packages and/or the kernel all attempt to add their own code here, coordination is very difficult if not impossible.

The process when the paragraph ends has different mechanisms and interfaces. A paragraph ends when the engine primitive `\par` is called while TEX is in unrestricted horizontal mode, i.e., is building a paragraph. At other times this primitive does nothing or generates as an error depending on the mode TEX is in, e.g., the `\par` in `\hbox{a\par b}` is ignored, but `$a\par b$` would complain.

If this primitive ends the paragraph it does some special "end of horizontal list" processing, then calls TEX's paragraph builder; this breaks the horizontal list into lines and then these lines are added as boxes to the enclosing vertical list and TEX returns to vertical mode.

This `\par` command can be given explicitly, but there are also situations in which TEX is generating it on the fly. Most often this happens when TEX encounters a blank line which is automatically changed to a `\par` command which is then executed. The other possibility is that TEX encounters a command which is incompatible with horizontal processing, e.g., `\vskip` (a request for adding vertical space). In such cases it silently backs up, and inserts a `\par` in the hope that this gets it out of horizontal mode and makes the vertical command acceptable.

The important point to note here is that TEX really inserts the command with the name `\par`, which can be redefined. Thus, it may not have its original "primitive" meaning and therefore may not end the horizontal list and call the paragraph builder. This approach offers some flexibility but also allows you to easily produce a TEX document that loops forever, for example, the simple line

```
A \let\par\relax \vskip
```

will start a horizontal list at `A`, redefines `\par`, then sees `\vskip` and inserts `\par` to end the paragraph. But this now only runs `\relax` so nothing changes and `\vskip` is read again, issues a `\par` which .... In short, it only takes a plain TEX document with five tokens to run forever (since no memory is consumed and therefore eventually exhausted).

There is no way other than changing `\par` to gain control at the end of a paragraph, i.e., there is no token list like `\everypar` that is inserted. Hence the only way to change the default behavior is to modify the action that `\par` executes, with similar issues as outlined before: different processes need to ensure that they do not overwrite their modifications or worse, think that the `\par` in front of them is the engine primitive while in fact it has already been changed by other code.

To make matters slightly worse there are a few places where TEX handles the situation differently (most likely for speed reasons back when computers were much slower). If TEX finds itself in unrestricted horizontal mode at the end of building a vertical box (for an `\insert`, `\vadjust` or executing the output routine code), it will finish the horizontal list not by issuing a `\par` command (which would be consistent with all other places) but by simply executing the primitive meaning of `\par`, regardless of the actual definition that `\par` has at the time.

Thus, if you have carefully crafted a redefined `\par` to execute some special actions at the end of a paragraph and you write something like

```
\vbox{Some paragraph ... text.}
```

you will find that your code does not get run for the last paragraph in that box. LATEX avoids this problem, by making sure that its boxes (such as `\parbox` or the `minipage` environment, etc.) all internally add an explicit `\par` at the end so that such code is run

and TEX finds itself in vertical mode already without the need to start up the paragraph builder internally. But, of course, this only works for boxes under direct control of the LATEX kernel; if some package uses low-level `\vbox`es without adding this precaution the TEX optimization kicks in and no special `\par` code is executed.

And there is another optimization that is painful: if a paragraph is interrupted by a mathematical display, e.g., `\[...\]` in LATEX or `$$...$$` in plain TEX, then TEX will resume horizontal mode afterward, i.e., it will start to build a new horizontal list without inserting an indentation box or `\everypar` at that point. However, if that list immediately ends with an explicit or implicit `\par` then TEX will simply throw away this "null" paragraph and not do its usual "end of horizontal list" processing, so this special case also needs to be accounted for when introducing any extended processing.

4.2 The new mechanism implemented for LATEX

To improve the situation (and also to support automatic tagging of PDF documents) we now offer public as well as private hooks at the start and end of the paragraph processing. The public hooks can be used by packages (or by the user in the preamble or within the document) and using the hook mechanisms it is possible to reorder or arrange code from different packages in such a way that these can safely coexist.

To make that happen we have to make use of the basic functionality that is offered by TEX, e.g., we install special code inside `\everypar` to provide hooks at the beginning and we redefine `\par` to do some special processing when appropriate to install hooks at the end of the paragraph.

In order to make this work, we have to ensure that package use of `\everypar` is not overwriting our code. This is done through a trick: we basically hide the real `\everypar` from the packages and offer them a new token register (with the same name). So if they install their own code it doesn't overwrite ours. Our code then inserts the new `\everypar` at the right place inside the process so that it looks as if it was the primitive `\everypar`.[22]

At the end of the paragraph it would be great if we could use a similar trick. However, due to the fact that TEX inserts the token `\par` (that doesn't have a defined meaning) we can't hide "the real thing™" and offer the package an indistinguishable alternate.

Fortunately, LATEX has already redefined `\par` for its own purposes. As a result there aren't many packages that attempt to change `\par`, because without a lot of extra care that would fail miserably. But the bottom line is that, if you load a package that alters `\par` then the end of paragraph hooks are most likely not executing while that redefinition is active.[23]

[22] Ideally, `\everypar` wouldn't be used at all by packages and instead they would simply write their code into the hooks now offered by the kernel. However, while this is the longterm goal and clearly an improvement (because then the packages do no longer need to worry about getting their code overwritten or needing to account for already existing code in `\everypar`), this will not happen overnight. For that reason support for this legacy method is retained.

[23] Similarly to the `\everypar` situation, the remedy is that such packages stop doing this and instead add their alterations into the paragraph hooks now provided.

4.2.1 The provided hooks

para/before
para/begin
para/end
para/after

The following four public hooks are defined and executed for each paragraph:

para/before This hook is executed after the kernel hook `\@kernel@before@para@before` (discussed below) in vertical mode immediately after TEX has contributed `\parskip` to the vertical list and before the actual paragraph processing in horizontal mode starts.

This hook should either not produce any typeset material or add only vertical material. If it starts a paragraph an error is generated. The reason is that we are in the starting process of processing a paragraph and so this would lead to endless recursion.[24]

para/begin This hook is executed after the kernel hook `\@kernel@before@para@begin` (discussed below) in horizontal mode immediately before the indentation box is placed (if there is any, i.e., if the paragraph hasn't been started with `\noindent`).

The indentation box to be typeset is available to the hook as `\IndentBox` and its automatic placement (after the hook is executed) can be prevented through `\OmitIndent`. More precisely `\OmitIndent` voids the box.

The indentation box is then typeset directly after the hook execution by something equivalent to `\box\IndentBox` followed by the current content of the token register `\everypar` that it is available to the kernel or to packages (that run some legacy code).

One has to be careful not to add any code to the hook that starts its own paragraph (e.g., by adding a `\parbox` or a `\marginpar` inside) because that would call the hook inside again (as a new paragraph is started there) and thus lead to an endless recursion ending only after exhausting the available memory. This can only be done by making sure that is not executed for the inner paragraphs (or at least not recursively forever).

para/end This hook is executed at the end of a paragraph when TEX is ready to return to vertical mode and after it has removed the last horizontal glue (but not any kerns) placed on the horizontal list. The code is still executed in horizontal mode so it is possible to add further horizontal material at this point, but it should not alter the mode (even a temporary exit from horizontal mode would create chaos—any attempt will cause an error message)! After the hook has ended the kernel hook `\@kernel@after@para@end` is executed and then TEX returns to vertical mode.

The hook is offered as public hook, but because of the requirement to stay within horizontal mode one needs to be careful in what is placed into the hook.[25]

This hook is implemented as a reversed hook.

para/after This hook is executed directly after TEX has returned to vertical mode and after any material that migrated out of the horizontal list (e.g., from a `\vadjust`) has processed.

[24] One could allow it but only if the newly started paragraph is processed without any hooks. Furthermore correct spacing would be a bit of a nightmare so for now this is forbidden.

[25] Maybe we should guard against that, but it would be rather tricky to implement as mode changes can happen across group boundaries so one would need to keep a private stack just for that. Well, something to ponder.

This hook should either not produce any typeset material or add only vertical material. However, for this hook starting a new paragraph is not a disaster so that it isn't prevented.

This hook is implemented as a reversed hook.

Once that hook code has been processed the kernel hook `\@kernel@after@para@after` is executed as the final action of the paragraph processing.

`\@kernel@before@para@before`
`\@kernel@after@para@after`
`\@kernel@before@para@begin`
`\@kernel@after@para@end`

As already mentioned above there are also four kernel hooks that are executed at the start and end of the processing.

`\@kernel@before@para@before` For future extensions, not currently used by the kernel.

`\@kernel@after@para@after` For future extensions, not currently used by the kernel.

`\@kernel@before@para@begin` Used by the kernel to implement tagging. This hook is executed at the very beginning of a paragraph after TEX has switched to horizontal mode but before any indentation box got added or any `\everypar` was run.

It should not generate typeset material that could alter the position. Note that it should never leave hmode, otherwise you will end with a loop! We could guard against this, but since it is an internal kernel hook that shouldn't be touched this isn't checked.

`\@kernel@after@para@end` Used by the kernel to implement tagging. It is executed directly after the public `para/end` hook. After it there is a quick check that we are still in horizontal mode, i.e., that the public hook has not mistakenly ended horizontal mode prematurely (this is an incomplete check just testing the mode and could perhaps be improved (at the cost of speed)).

4.2.2 Altered and newly provided commands

`\par`
`\endgraf`
`\para_end:`

An explicit request for ending a paragraph is provided in plain TEX under the name `\endgraf`, which simply uses the primitive meaning (regardless of what `\par` may have as its current definition). In LATEX `\endgraf` (with that behavior) was originally also available.

With the new paragraph handling in LATEX, ending a paragraph means a bit more than just calling the engine's paragraph builder: the process also has to add any hook code for the end of a paragraph. Thus `\endgraf` was changed to provide this additional functionality (along with `\par` remaining subject to its current meaning).

The expl3 name for this functionality is `\para_end:`.

Note: *The next two commands are still under discussion and may slightly change their semantics (as described in the document) and/or their names between now and the 2021 Spring release!*

\OmitIndent
\para_omit_indent:

Inside the para/begin hook one can use this command to suppress the indentation box at the start of the paragraph. (Technically it is possible to use this command outside the hook as well, but this should not be relied upon.) The box itself remains available for use.

The expl3 name for the function is \para_omit_indent:.

\IndentBox
\g_para_indent_box

The box register holding the indentation box for the paragraph is available for inspection (or changes) inside hooks. It remains available even if the \OmitIndent command was used; in that case it will just not be automatically placed.

The expl3 name for the box register is \g_para_indent_box.

\RawIndent
\para_raw_indent:
\RawNoindent
\para_raw_noindent:
\RawParEnd
\para_raw_end:

\RawIndent *hmode material* \RawParEnd
\RawNoindent *hmode material* \RawParEnd

The commands \RawIndent and \RawNoindent are not meant for normal paragraph building (where the result is a textual paragraph in the traditional meaning of the word), but for special cases where TEX's low-level algorithm is used to achieve special effects, but where the result is not a "paragraph".

They are called "raw", because they bypass LATEX's hook mechanism for paragraphs and simply invoke the low-level TEX algorithm. I.e., they are like the original TEX primitives \indent and \noindent (that is they execute no hooks other than \everypar) except that they can only be used in vertical mode and generate an error if found elsewhere.

To avoid issues a paragraph started by them should always be ended by \RawParEnd[26] and not by \par (or a blank line), because the latter will execute hooks which then have no counterpart at the beginning of the paragraph. It is the responsibility of the programmer to make sure that they are properly paired. This also means that one should not put arbitrary user content between these commands if that content could contain stray \pars.

The expl3 names for the functions are \para_raw_indent:, \para_raw_indent: and \para_raw_end:.

4.2.3 Examples

None of the examples in this section are meant for real use as they are far too simple-minded but they should give some ideas of what could be possible if a bit more care is applied.

Testing the mechanism

The idea is to output for each paragraph encountered some information: a paragraph sequence number, a level number in roman numerals, the environment in which this paragraph appears, and the line number where the start or end of the paragraph is, e.g., something like

[26] Technical note for those who know their *TEXbook*: the \RawParEnd command invokes the original TEX engine definition of \par that (solely) triggers the paragraph builder in TEX when found inside unrestricted horizontal mode and does nothing in other processing modes.

```
PARA: 1-i start (document env. on input line 38)
PARA: 1-i end   (document env. on input line 38)
PARA: 2-i start (document env. on input line 40)
PARA: 3-ii start (minipage env. on input line 40)
PARA: 3-ii end   (minipage env. on input line 40)
PARA: 2-i end   (document env. on input line 41)
```

As you can see paragraph 2 starts on line 40 and ends on 41 and inside a minipage started paragraph 3 (start and end on line 40). If you run this on some document you will find that LaTeX considers more things "a paragraph" than you have probably thought.

This was generated by the following hook code:

```
\newcounter{paracnt}          % sequence counter
\newcounter{paralevel}        % level counter
```

To support paragraph nesting we need to maintain a stack of the sequence numbers. This is most easily done using expl3 functions, so we switch over. This is not a very general implementation, just enough for what we need and a bit of LaTeX 2_ε thrown in as well. When popping, the result gets stored in `\paracntvalue` and the `\ERROR` should never happen because it means we have tried to pop from an empty stack.

```
\ExplSyntaxOn
\seq_new:N \g_para_seq
\cs_new:Npn \ParaPush
  {\seq_gpush:No  \g_para_seq {\the\value{paracnt}}}
\cs_new:Npn \ParaPop  {\seq_gpop:NNF  \g_para_seq \paracntvalue \ERROR }
\ExplSyntaxOff
```

At the start of the paragraph increment both sequence counter and level and also save the then current sequence number on our stack.

```
\makeatletter % because we use a few internal 2e commands
\AddToHook{para/begin}{%
  \stepcounter{paracnt}\stepcounter{paralevel}%
  \ParaPush
```

To display the sequence number we `\typeout` the current sequence and level number. The command `\@currenvir` gives us the current environment and `\on@line` produces a space and the current input line number.

```
  \typeout{PARA: \arabic{paracnt}-\roman{paralevel} start
    (\@currenvir\space env.\on@line)}%
```

We also typeset the sequence number as a tiny red number in a box that takes up no horizontal space. This helps us seeing where LaTeX sees the start and end of the paragraphs in the document.

```
  \llap{\color{red}\tiny\arabic{paracnt}\ }%
}
```

At the end of the paragraph we display sequence number and level again. The level counter has the correct value but we need to retrieve the right sequence value by popping it off the stack after which it is available in `\paracntvalue` the way we have set this up above.

```
\AddToHook{para/end}{%
  \ParaPop
  \typeout{PARA: \paracntvalue-\roman{paralevel} end \space\space
    (\@currenvir\space env.\on@line)}%
```

We also typeset again a tiny red number with that value, this time sticking out to the right.[27] We also decrement the level counter since our level has finished.

```
  \rlap{\color{red}\tiny\ \paracntvalue}%
  \addtocounter{paralevel}{-1}%
}
\makeatother
```

Mark the first paragraph of each `itemize`

The code for this is rather simple. We supply some code that is executed only once inside a hook at the start of each `itemize`. We explicitly change the color back and forth so that we don't introduce grouping around the paragraph.

```
\AddToHook{env/itemize/begin}{%
  \AddToHookNext{para/begin}{\color{blue}}%
  \AddToHookNext{para/end}{\color{black}}%
}
```

As a result the first paragraph of each `itemize` will appear in blue.

4.2.4 Some technical notes

The code tries hard to be transparent for package code, but of course any change means that there is a potential for breaking other code. So in section we collect a few cases that may be of importance if low-level code is dealing with paragraphs that are now behaving slightly differently. The notes are from issues we observed and will probably grow over time.

Glue items between paragraphs (found with fancypar)

In the past LaTeX placed two glue items between two consecutive paragraphs, e.g.,

```
text1 \par text2 \par
```

would show something like

```
\glue(\parskip) 0.0 plus 1.0
\glue(\baselineskip) 5.16669
```

but now there is another `\parskip` glue (that is always 0pt):

```
\glue(\parskip) 0.0 plus 1.0
\glue(\parskip) 0.0
\glue(\baselineskip) 5.16669
```

[27] Note that this can alter the document pagination, because a paragraph ending in a display (e.g., an equation) will get an extra line—in that case our tiny number has an effect even though it doesn't take up any space, because it paragraph is no longer empty and thus isn't dropped!

The reason is that we generate a “fake” paragraph to gain control and safely add the early hooks, but this generates an additional glue item. That item doesn't contribute anything vertically but if somebody writes code that unravels a constructed list using `\lastbox`, `\unskip` and `\unpenalty` then the code has to remove one additional glue item or else it will fail.

Chapter 5

The shipout routine: hooks and interfaces

5.1 Introduction

The code provides an interface to the `\shipout` primitive of TEX which is called when a finished pages is finally "shipped out" to the target output file, e.g., the `.dvi` or `.pdf` file. A good portion of the code is based on ideas by Heiko Oberdiek implemented in his packages atbegshi and atenddvi even though the interfaces are somewhat different.[28]

5.1.1 Overloading the `\shipout` primitive

`\shipout`

With this implementation TEX's shipout primitive is no longer available for direct use. Instead `\shipout` is running some (complicated) code that picks up the box to be shipped out regardless of how that is done, i.e., as a constructed `\vbox` or `\hbox` or as a box register.

It then stores it in a named box register. This box can then be manipulated through a set of hooks after which it is shipped out for real.

Each shipout that actually happens (i.e., where the material is not discarded for one or the other reason) is recorded and the total number is available in a readonly variable and in a LATEX counter.

[28] Heiko's interfaces are emulated by the kernel code, if a document requests his packages, so older documents will continue to work.

`\RawShipout` This command implements a simplified shipout that bypasses the foreground and background hooks, e.g., only `shipout/firstpage` and `shipout/lastpage` are executed and the total shipout counters are incremented.

The command doesn't use `\ShipoutBox` but its own private box register so that it can be used inside of shipout hooks to do some additional shipouts while already in the output routine with the current page being stored in `\ShipoutBox`. It does have access to `\ShipoutBox` if it is used in `shipout/before` (or `shipout/after`) and can use its content.

It is safe to use it in `shipout/before` or `shipout/after` but not necessarily in the other `shipout/...` hooks as they are intended for special processing.

`\ShipoutBox`
`\l_shipout_box` This box register is called `\ShipoutBox` (alternatively available via the L3 name `\l_shipout_box`).

This box is a "local" box and assignments to it should be done only locally. Global assignments (as done by some packages with older code where this is box is known as 255) may work but they are conceptually wrong and may result in errors under certain circumstances.

During the execution of `shipout/before` this box contains the accumulated material for the page, but not yet any material added by other shipout hooks. During execution of `shipout/after`, i.e., after the shipout has happened, the box also contains any background or foreground material.

Material from the hooks `shipout/firstpage` or `shipout/lastpage` is not included (but only used during the actual shipout) to facilitate reuse of the box data (e.g., `shipout/firstpage` material should never be added to a later page of the output).

`\l_shipout_box_ht_dim`
`\l_shipout_box_dp_dim`
`\l_shipout_box_wd_dim`
`\l_shipout_box_ht_plus_dp_dim`

The shipout box dimensions are available in the L3 registers `\l_shipout_box_ht_dim`, etc. (there are no LaTeX 2_ε names).[29] These variables can be used inside the hook code for `shipout/before`, `shipout/foreground` and `shipout/background` if needed.

[29] Might need changing, but HO's version as strings is not really helpful I think).

5.1.2 Provided hooks

shipout/before
shipout/after
shipout/foreground
shipout/background
shipout/firstpage
shipout/lastpage

The code for `\shipout` offers a number of hooks into which packages (or the user) can add code to support different use cases. These are:

shipout/before This hook is executed after the finished page has been stored in `\ShipoutBox` / `\l_shipout_box`). It can be used to alter that box content or to discard it completely (see `\DiscardShipoutBox` below).

You can use `\RawShipout` inside this hook for special use cases. It can make use of `\ShipoutBox` (which doesn't yet include the background and foreground material).

Note: It is not possible (or say advisable) to try and use this hook to typeset material with the intention to return it to main vertical list, it will go wrong and give unexpected results in many cases—for starters it will appear after the current page not before or it will vanish or the vertical spacing will be wrong!

shipout/background This hook adds a picture environment into the background of the page with the `(0,0)` coordinate in the top-left corner using a `\unitlength` of `1pt`.

It should therefore only receive `\put` commands or other commands suitable in a `picture` environment and the vertical coordinate values would normally be negative.

Technically this is implemented by adding a zero-sized `\hbox` as the very first item into the `\ShipoutBox` containing that `picture` environment. Thus the rest of the box content will overprint what ever is typeset by that hook.

shipout/foreground This hook adds a picture environment into the foreground of the page with the `(0,0)` coordinate in the top-left corner using a `\unitlength` of `1pt`.

Technically this is implemented by adding a zero-sized `\hbox` as the very last item into the `\ShipoutBox` and raising it up so that it still has its `(0,0)` point in the top-left corner. But being placed after the main box content it will be typeset later and thus overprints it (i.e., is in the foreground).

shipout This hook is executed after foreground and/or background material has been added, i.e., just in front of the actual shipout operation. Its purpose is to allow manipulation of the finalized box (stored in `\ShipoutBox`) with the extra material also in place (which is not yet the case in `shipout/before`).

It cannot be used to cancel the shipout operation via `\DiscardShipoutBox` (that has to happen in `shipout/before`, if desired!

shipout/firstpage The material from this hook is executed only once at the very beginning of the first output page that is shipped out (i.e., not discarded at the last minute). It should only contain `\special` or similar commands needed to direct post processors handling the `.dvi` or `.pdf` output.[30]

This hook is added to the very first page regardless of how it is shipped out (i.e., with `\shipout` or `\RawShipout`).

[30] In LaTeX 2_ε that was already existing, but implemented using a box register with the name `\@begindvibox`.

shipout/lastpage The corresponding hook to add \specials at the very end of the output file. It is only executed on the very last page of the output file — or rather on the page that LaTeX believes is the last one. Again it is executed regardless of the shipout method.

It may not be possible for LaTeX to correctly determine which page is the last one without several reruns. If this happens and the hook is non-empty then LaTeX will add an extra page to place the material and also request a rerun to get the correct placement sorted out.

shipout/after This hook is executed after a shipout has happened. If the shipout box is discarded this hook is not looked at.

You can use \RawShipout inside this hook for special use cases and the main \ShipoutBox is still available at this point (but in contrast to shipout/before it now includes the background and foreground material).

Note: Just like shipout/before this hook is not meant to be used for adding typeset material back to the main vertical list—it might vanish or the vertical spacing will be wrong!

As mentioned above the hook shipout/before is executed first and can manipulate the prepared shipout box stored in \ShipoutBox or set things up for use in \write during the actual shipout. It is even run if there was a \DiscardShipoutBox request in the document.

The other hooks (except shipout and shipout/after) are added inside hboxes to the box being shipped out in the following order:

shipout/firstpage	only on the first page
shipout/background	
⟨*boxed content of* \ShipoutBox⟩	
shipout/foreground	
shipout/lastpage	only on the last page

If any of the hooks has no code then the corresponding box is added at that point.

Once the (page) box has got the above extra content it can again be manipulated using the shipout hook and then is shipped out for real.

Once the (page) box has been shipped out the shipout/after hook is called (while you are still inside the output routine). It is not called if the shipout box was discarded.

In a document that doesn't produce pages, e.g., only makes \typeouts, none of the hooks are ever executed (as there is no \shipout) not even the shipout/lastpage hook.

If \RawShipout is used instead of \shipout then only the hooks shipout/firstpage and shipout/lastpage are executed (on the first or last page), all others are bypassed.

5.1.3 Legacy LaTeX commands

\AtBeginDvi
\AtEndDvi

\AtBeginDvi {⟨*code*⟩}

\AtBeginDvi is the existing LaTeX 2ε interface to fill the shipout/firstpage hook. This is not really a good name as it is not just supporting .dvi but also .pdf output or .xdv.

\AtEndDvi is the counterpart that was not available in the kernel but only through the package atenddvi. It fills the shipout/lastpage hook.

Neither interface can set a code label but uses the current default label.

As these two wrappers have been available for a long time we continue offering them (but not enhancing them, e.g., by providing support for code labels).

For new code we strongly suggest using the high-level hook management commands directly instead of "randomly-named" wrappers. This will lead to code that is easier to understand and to maintain and it also allows you to set code labels if needed.

For this reason we do not provide any other "new" wrapper commands for the above hooks in the kernel, but only keep the existing ones for backward compatibility.

5.1.4 Special commands for use inside the hooks

`\DiscardShipoutBox`
`\shipout_discard:`

```
\AddToHookNext {shipout/before} {...\DiscardShipoutBox...}
```

The `\DiscardShipoutBox` declaration (L3 name `\shipout_discard:`) requests that on the next shipout the page box is thrown away instead of being shipped to the `.dvi` or `.pdf` file.

Typical applications wouldn't do this unconditionally, but have some processing logic that decides to use or not to use the page.

Note that if this declaration is used directly in the document it may depend on the placement to which page it applies, given that LaTeX output routine is called in an asynchronous manner! Thus normally one would use this only as part of the `shipout/before` code.

> *Todo: Once we have a new mark mechanism available we can improve on that and make sure that the declaration applies to the page that contains it — not done (yet)*

`\DiscardShipoutBox` cannot be used in any of the `shipout/...` hooks other than `shipout/before`.

In the atbegshi package there are a number of additional commands for use inside the `shipout/before` hook. They should normally not be needed any more as one can instead simply add code to the hooks `shipout/before`, `shipout`, `shipout/background` or `shipout/foreground`.[31] If atbegshi gets loaded then those commands become available as public functions with their original names as given below.

5.1.5 Provided LuaTeX callbacks

`pre_shipout_filter`

Under LuaTeX the `pre_shipout_filter` Lua callback is provided which gets called directly after the `shipout` hook, immediately before the shipout primitive gets invoked. The signature is

```
function(<node> head)
  return true
end
```

The `head` is the list node corresponding to the box to be shipped out. The return value should always be `true`.

[31] If that assumption turns out to be wrong it would be trivial to change them to public functions (right now they are private).

5.1.6 Information counters

`\ReadonlyShipoutCounter` `\ifnum\ReadonlyShipoutCounter=...`
`\g_shipout_readonly_int` `\int_use:N \g_shipout_readonly_int % expl3 usage`

This integer holds the number of pages shipped out up to now (including the one to be shipped out when inside the output routine). More precisely, it is incremented only after it is clear that a page will be shipped out, i.e., after the `shipout/before` hook (because that might discard the page)! In contrast `shipout/after` sees the incremented value.

Just like with the `page` counter its value is only accurate within the output routine. In the body of the document it may be off by one as the output routine is called asynchronously!

Also important: it *must not* be set, only read. There are no provisions to prevent that restriction, but if you manipulate it, chaos will be the result. To emphasize this fact it is not provided as a LaTeX counter but as a TeX counter (i.e., a command), so `\Alph{\ReadonlyShipoutCounter}` etc, would not work.

`totalpages` `\arabic{totalpages}`
`\g_shipout_totalpages_int` `\int_use:N \g_shipout_totalpage_int % expl3 usage`

In contrast to `\ReadonlyShipoutCounter`, the `totalpages` counter is a LaTeX counter and incremented for each shipout attempt including those pages that are discarded for one or the other reason. Again `shipout/before` sees the counter before it is incremented. In contrast `shipout/after` sees the incremented value.

Furthermore, while it is incremented for each page, its value is never used by LaTeX. It can therefore be freely reset or changed by user code, for example, to additionally count a number of pages that are not build by LaTeX but are added in a later part of the process, e.g., cover pages or picture pages made externally.

Important: as this is a page-related counter its value is only reliable inside the output routine!

`\PreviousTotalPages` `\PreviousTotalPages`

Command that expands to the number of total pages from the previous run. If there was no previous run or if used in the preamble it expands to `0`. Note that this is a command and not a counter, so in order to display the number in, say, Roman numerals you have to assign its value to a counter and then use `\Roman` on that counter.

5.1.7 Debugging shipout code

`\DebugShipoutsOn` `\DebugShipoutsOn`
`\DebugShipoutsOff`
`\shipout_debug_on:`
`\shipout_debug_off:`

Turn the debugging of shipout code on or off. This displays changes made to the shipout data structures.

Todo: This needs some rationalizing and may not stay this way.

5.2 Emulating commands from other packages

The packages in this section are no longer necessary, but as they are used by other packages, they are emulated when they are explicitly loaded with `\usepackage` or `\RequirePackage`.

Please note that the emulation only happens if the package is explicitly requested, i.e., the commands documented below are not automatically available in the LaTeX kernel! If you write a new package we suggest to use the appropriate kernel hooks directly instead of loading the emulation.

5.2.1 Emulating atbegshi

`\AtBeginShipoutUpperLeft` `\AddToHook {shipout/before} {...\AtBeginShipoutUpperLeft{⟨code⟩}...}`
`\AtBeginShipoutUpperLeftForeground`

This adds a `picture` environment into the background of the shipout box expecting ⟨*code*⟩ to contain `picture` commands. The same effect can be obtained by simply using kernel features as follows:

`\AddToHook{shipout/background}{`⟨*code*⟩`}`

There is one technical difference: if `\AtBeginShipoutUpperLeft` is used several times each invocation is put into its own box inside the shipout box whereas all ⟨*code*⟩ going into `shipout/background` ends up all in the same box in the order it is added or sorted based on the rules for the hook chunks.

`\AtBeginShipoutUpperLeftForeground` is similar with the difference that the `picture` environment is placed in the foreground. To model it with the kernel functions use the hook `shipout/foreground` instead.

`\AtBeginShipoutAddToBox` `\AddToHook {shipout/before} {...\AtBeginShipoutAddToBox{⟨code⟩}...}`
`\AtBeginShipoutAddToBoxForeground`

These work like `\AtBeginShipoutUpperLeft` and `\AtBeginShipoutUpperLeftForeground` with the difference that ⟨*code*⟩ is directly placed into an `\hbox` inside the shipout box and not surrounded by a `picture` environment.

To emulate them using `shipout/background` or `shipout/foreground` you may have to wrap ⟨*code*⟩ into a `\put` statement but if the code is not doing any typesetting just adding it to the hook should be sufficient.

`\AtBeginShipoutBox` This is the name of the shipout box as atbegshi knows it.

`\AtBeginShipoutOriginalShipout`

This is the name of the `\shipout` primitive as atbegshi knows it. This bypasses all the mechanisms set up by the LaTeX kernel and there are various scenarios in which it can therefore fail. It should only be used to run existing legacy atbegshi code but not in newly developed applications.

The kernel alternative is `\RawShipout` which is integrated with the LaTeX mechanisms and updates, for example, the `\ReadonlyShipoutCounter` counter. Please use `\RawShipout` for new code if you want to bypass the before, foreground and background hooks.

`\AtBeginShipoutInit` By default atbegshi delayed its action until `\begin{document}`. This command was forcing it in an earlier place. With the new concept it does nothing.

`\AtBeginShipout` `\AtBeginShipout{`⟨*code*⟩`}` ≡ `\AddToHook{shipout/before}{`⟨*code*⟩`}`
`\AtBeginShipoutNext` `\AtBeginShipoutNext{`⟨*code*⟩`}` ≡ `\AddToHookNext{shipout/before}{`⟨*code*⟩`}`

This is equivalent to filling the `shipout/before` hook by either using `\AddToHook` or `\AddToHookNext`, respectively.

`\AtBeginShipoutFirst` `\AtBeginShipoutDiscard` The atbegshi names for `\AtBeginDvi` and `\DiscardShipoutBox`.

5.2.2 Emulating everyshi

The everyshi package is providing commands to run arbitrary code just before the shipout starts. One point of difference: in the new shipout hooks the page is available as `\ShipoutBox` for inspection of change, one should not manipulate box 255 directly inside `shipout/before`, so old code doing this would change to use `\ShipoutBox` instead of 255 or `\@cclv`.

`\EveryShipout` `\EveryShipout{`⟨*code*⟩`}` ≡ `\AddToHook{shipout/before}{`⟨*code*⟩`}`

`\AtNextShipout` `\AtNextShipout{`⟨*code*⟩`}` ≡ `\AddToHookNext{shipout/before}{`⟨*code*⟩`}`

However, most use cases for everyshi are attempts to put some picture or text into the background or foreground of the page and that can be done today simply by using the `shipout/background` and `shipout/foreground` hooks without any need to coding.

5.2.3 Emulating atenddvi

The atenddvi package implemented only a single command: `\AtEndDvi` and that is now available out of the box so the emulation makes the package a no-op.

5.2.4 Emulating everypage

This package patched the original `\@begindvi` hook and replaced it with its own version. Its functionality is now covered by the hooks offered by the kernel so that there is no need for such patching any longer.

`\AddEverypageHook` `\AddEverypageHook{`⟨*code*⟩`}` ≡
`\AddToHook{shipout/background}{\put(1in,-1in){`⟨*code*⟩`}}`

`\AddEverypageHook` is adding something into the background of every page at a position of 1in to the right and 1in down from the top left corner of the page. By using the kernel hook directly you can put your material directly to the right place, i.e., use other coordinates in the `\put` statement above.

`\AddThispageHook` `\AddThispageHook{`⟨*code*⟩`}` ≡
`\AddToHookNext{shipout/background}{\put(1in,-1in){`⟨*code*⟩`}}`

The `\AddThispageHook` wrapper is similar but uses `\AddToHookNext`.

Part III
Run data and page design

Chapter 6

The marks mechanism

6.1 Introduction

The TEX engines offer a low-level mark mechanism to communicate information about the content of the current page to the asynchronous operating output routine. It works by placing `\mark` commands into the source document. When the material for the current page is assembled in box 255, TEX scans for such marks and sets the commands `\topmark`, `\firstmark` and `\botmark`. The `\firstmark` receives the content of the first `\mark` seen in box 255 and `\botmark` the content of the last mark seen. The `\topmark` holds the content of the last mark seen on the previous page or more exactly the value of `\botmark` from the previous page. If there are no marks on the current page then all three are made equal to the `\botmark` from the previous page.

This mechanism works well for simple formats (such as plain TEX) whose output routines are only called to generate pages. It fails, however, in LATEX (and other more complex formats), because here the output routine is sometimes called without producing a page, e.g., when encountering a float and placing it into one of the float regions. In that case the output routine is called, determines where to place the float, alters the goal for assembling text material (if the float was added to the top or bottom region) and then it resumes collecting textual material.

As a result the `\botmark` gets updated and so `\topmark` no longer reflects the situation at the top of the next page when that page is finally boxed.

Another problem for LATEX was that it wanted to use several "independent" marks and in the early implementations of TEX there was only a single `\mark` command available. For that reason LATEX implemented its own mark mechanism where the marks always contained two parts with their own interfaces: `\markboth` and `\markright` to set marks and `\leftmark` and `\rightmark` to retrieve them.

However, this extended mechanism (while supporting scenarios such as chapter/section marks) was far from general. The mark situation at the top of a page (i.e., `\topmark`) remained unusable and the two marks offered were not really independent of each other because `\markboth` (as the name indicates) was always setting both.

The new mechanism overcomes both issues:

- It provides arbitrarily many, fully independent named marks, that can be allocated and, from that point onwards, used.
- It offers access for each such marks to retrieve its top, first, and bottom values separately.

- Furthermore, the mechanism is augmented to give access to marks in different "regions" which may not be just full pages.

6.2 Design-level and code-level interfaces

The interfaces are mainly meant for package developers, but they are usable (with appropriate care) also in the document preamble, for example, when setting up special running headers with fancyhdr, etc. They are therefore available both as CamelCase commands as well as commands for use in the L3 programming layer. Both are described together below.

`\NewMarkClass` `\NewMarkClass {⟨class⟩}`
`\mark_new_class:n` `\mark_new_class:n {⟨class⟩}`

Declares a new ⟨*class*⟩ of marks to be tracked by LaTeX. Each ⟨*class*⟩ must be declared before it is used.

Mark classes can only be declared before `\begin{document}`.

`\InsertMark` `\InsertMark {⟨class⟩} {⟨text⟩}`
`\mark_insert:nn` `\mark_insert:nn {⟨class⟩} {⟨text⟩}`

Adds a mark to the current galley for the ⟨*class*⟩, containing the ⟨*text*⟩.

It has no effect in places in which you can't place floats, e.g., a mark inside a box or inside a footnote never shows up anywhere.

If used in vertical mode it obeys LaTeX's internal `@nobreak` switch, i.e., it does not introduce a breakpoint if used after a heading. If used in horizontal mode it doesn't handle spacing (like, for example, `\index` or `\label` does, so it should be attached to material that is typeset.

`insertmark` `\AddToHook {insertmark} {⟨code⟩}`

When marks are inserted, the mark content may need some special treatment, e.g., by default `\label`, `\index`, and `\glossary` do not expand at this time (but only later if and when the mark content is actually used. In order to allow packages to augment or alter this setup there is a public hook `insertmark` that is executed at this point. It runs in a group so local modification to commands are only applied to the ⟨*text*⟩ argument of `\InsertMark` or `\mark_insert:nn`.

\TopMark ⋆ \TopMark [⟨region⟩] {⟨class⟩}
\FirstMark ⋆ \FirstMark [⟨region⟩] {⟨class⟩}
\LastMark ⋆ \LastMark [⟨region⟩] {⟨class⟩}
\mark_use_top:nn ⋆ \mark_use_top:nn {⟨region⟩} {⟨class⟩}
\mark_use_first:nn ⋆ \mark_use_first:nn {⟨region⟩} {⟨class⟩}
\mark_use_last:nn ⋆ \mark_use_last:nn {⟨region⟩} {⟨class⟩}

These functions expand to the appropriate mark ⟨*text*⟩ for the given ⟨*class*⟩ in the specified ⟨*region*⟩. The default ⟨*region*⟩ in the design-level commands is `page`. Note that with the L3 layer commands there are no optional arguments, i.e., both arguments have to be provided.

TEXhackers note: The result is returned within the `\unexpanded` primitive (`\exp_not:n`), which means that the ⟨*text*⟩ does not expand further when appearing in an `x`-type or `e`-type argument expansion.

The "first" and "last" marks are those seen first and last in the current region/page, respectively. The "top" mark is the last mark of the ⟨*class*⟩ seen in an earlier region, i.e., the ⟨*text*⟩ what would be "current" at the very top of the region.

Important! The commands are only meaningful inside the output routine, in other places their result is (while not random) unpredictable due to the way LaTeX cuts text material into pages. There is, however, one exception: if you produce multiple columns using the multicol package, it is possible to retrieve mark values from the regions `first-column`, `last-column`, `mcol-1`, `mcol-2`,... directly after the environment has ended. This can, for example, be useful if a `multicols` has been be used inside a box.

Currently, ⟨*region*⟩ is one of `page`, `previous-page`, `column`, `previous-column`, `first-column`, `last-column`, and `mcol-1` (first column in a `multicols`), `mcol-2` (second column in a `multicols`), up to `mcol-20` (twentieth column in a `multicols`). See section 6.2.2 for discussion of how these regions behave and how one can make use of them.

\IfMarksEqualTF ⋆ \IfMarksEqualTF [⟨region⟩] {⟨class⟩} {⟨pos$_1$⟩} {⟨pos$_2$⟩} {⟨true⟩} {⟨false⟩}
\IfMarksEqualT ⋆ \mark_if_eq:nnnnTF {⟨region⟩} {⟨class⟩} {⟨pos$_1$⟩} {⟨pos$_2$⟩} {⟨true⟩} {⟨false⟩}
\IfMarksEqualF ⋆ \mark_if_eq:nnnnnnTF {⟨region$_1$⟩} {⟨class$_1$⟩} {⟨pos$_1$⟩}
\mark_if_eq:nnnnTF ⋆ {⟨region$_2$⟩} {⟨class$_2$⟩} {⟨pos$_2$⟩} {⟨true⟩} {⟨false⟩}
\mark_if_eq:nnnnnnTF ⋆

These conditionals allow you to compare the content of two marks and act based on the result. The commands work in an expansion context, if necessary.

It is quite common when programming with marks to need to interrogate conditions such as whether marks have appeared on a previous page, or if there are multiple marks present on the current page, and so on. The tests above allow for the construction of a variety of typical test scenarios, with three examples presented below.

The first two conditionals cover only the common scenarios. Both marks are picked up from the same ⟨*region*⟩ (by default `page`) and they have to be of the same ⟨*class*⟩.[32] The ⟨*pos$_i$*⟩ argument can be either `top`, `first`, or `last`.

Important to note is that the comparison is not with respect to the textual content of the marks but whether or not they originated from the same `\InsertMark` command (or the L3 layer version `\mark_insert:nn`).

If you wish to compare marks across different regions or across different classes, you have to do it using the generic test only available in the L3 programming layer or do it

[32] If an undeclared mark class is used the tests return *true* (not an error).

manually, i.e., get the marks and then compare the values yourself.[33]

6.2.1 Use cases for conditionals

However, the basic version is enough for the following typical use cases:

Test for at most one mark of class `myclass` on current page: If the first and last mark in a region are the same then either there was no mark at all, or there was at most one. To test this on the current page:

```
\NewMarkClass{myclass}
\IfMarksEqualTF{myclass}{first}{last}
               { <zero or one mark> }{ <two or more marks> }
```

Test for no mark of class `myclass` in the previous page: If the top mark is the same as the first mark, there is no mark in the region at all. If we wanted to do this test for the previous page:

```
\IfMarksEqualTF[previous-page]{myclass}{top}{first}
               { <no marks> }{ <at least one mark> }
```

Comparing `top` and `last` would give you the same result.

Test for zero, one, or more than one: Combining the two tests from above you can test for zero, one or more than one mark.

```
\IfMarksEqualTF{myclass}{top}{first}
               { <no marks> }
               {\IfMarksEqualTF{myclass}{first}{last}
                 { <exactly one mark> }{ <more than one mark> }}
```

If you need one of such tests more often (or if you want a separate command for it for readability), then consider defining:

```
\providecommand\IfNoMarkTF[2][page]{\IfMarksEqualTF[#1]{#2}{first}{last}}
```

6.2.2 Understanding regions

If a page has just been finished then the region `page` refers to the current page and `previous-page`, as the name indicates, refers to the page before the current page. This means you are able to access mark information for the current page as well as for the page before (as long as you are inside the output routine) without the need to explicitly save that information beforehand. The `page` region is the region that is most often queried, which is why commands like `\FirstMark` use that region by default.

In single column documents the `column` is the same as the `page` region, but in two-column documents (if not produced by `multicols`), `column` refers to the current column that just got finished and previous-column to the one previously finished. Code for running headers is (in standard LaTeX) evaluated only after both columns have been assembled, which is another way of saying that in that case `previous-column` refers to the left column and `column` to the right column. However, to make these somewhat

[33] If two undeclared mark classes are compared the result is always *true*; if a declared and an undeclared mark class is used it is always *false*.

easier to use, there are also aliased names for these two regions: `first-column` and `last-column`.[34]

Note that you can only look backwards at already processed regions, e.g., in a `twoside` document finishing a recto (odd, right-hand) page you can access the data from the facing verso (left-hand) page, but if you are finishing a left-hand page you can't integrate data from the upcoming right-hand page. If such a scenario needs to be realized then it is necessary to save the left-hand page temporarily instead of finalizing it, process material for the right-hand page and once both are ready, attach running headers and footers and shipout out both in one go.[35]

The situation starts getting rather complex if you allow for multiple columns in the way they are supported by the multicol package. In this case you might have a variable number of "columns" on a single page to be shipped out. And even if not, then a `multicols` might start or end in the middle of the page; in either case, the regions `column` and `previous-column` become rather meaningless and you should therefore not use them.[36] Instead, the algorithm offers `mcol-1`, `mcol-2`, `mcol-3`, etc., to represent the columns in the `multicols` on the current page to be shipped out. If there is more than one `multicols` on the current page then in the output routine only the columns of the last one will be accessible.

These provisions cover, out of the box, a number of layouts and use cases, but obviously not all. However, more cases can be supported by storing away mark information during the processing. Here is the full algorithm:

- The `column` region is used by the "current column" that is being built (moving through all columns with `previous-column` trailing behind (to handle top marks properly).
- When the `multicols` starts, the `column` region is cleared, i.e., from that point on it looks as if there have not been any marks so far. This will make sure that the top mark in the first column is always empty.
- If the `multicols` extends beyond the current page, then the material designated for the current page is split into columns. The `column` region is used to represent each column in turn.
  - First we copy the current data from `column` to `previous-column`. Then the mark data from the current column is placed into the `column` region. Then we alias `column` to `mcol-1`.
  - These steps are repeated for all columns of the `multicols` environment.
  - Finally, the first and the last column of that page is also made available as `first-column` and `last-column`, respectively.
- All those marks inside any of the columns are also available in the `page` region. Thus, if you are interested in the top, first, or last mark of a specific class on the whole page you simply need to query for it in the `page` region.

[34] The region is called "last-column" not "second-column" in anticipation of extending the mechanism to multiple columns, where first and last would still make sense. There aren't any `previous-first-column` and `previous-last-column` regions to access the corresponding columns from the previous page.

[35] As of now that scenario is not (yet) officially supported but it would be possible to achieve this using the shipout hooks to store the verso page and then on the next shipout use the hook to shipout both with running headers and footers attached.

[36] They return something, because they represent the last two columns of the `multicols` when you are inside the output routine, but that is obviously of little use.

- If the `multicols` continues across several pages then this algorithm above is repeated for each page, except that the `column` region is not cleared again. This means that the top mark of the first column of the next page will be the last mark of the last column from the previous page.
- When the `multicols` finishes the remaining material for the current page is balanced to produce columns of roughly equal height.
- Again `column` and `previous-column` are used while this balancing happens and `mcol-1`, `mcol-2`, etc., are used to represent the column regions and `first-column` and `last-column` are set appropriately.
- Then the balanced set of columns is returned back to the page (since there may be space for further material). In addition, all marks inside that material are reinserted so that they become available in the `page` region.
- As a side effect, it is possible (and useful in certain circumstances) to query for mark classes directly after the `multicols` has ended without the need to be inside the output routine. The regions that can be queried this way are `mcol-1`, `mcol-2`, etc. (up to the number of columns the multicol had) and `first-column` and last-column.

6.2.3 Debugging mark code

`\DebugMarksOn`
`\DebugMarksOff`
`\mark_debug_on:`
`\mark_debug_off:`

`\DebugMarksOn ... \DebugMarksOff`

Commands to turn the debugging of mark code on or off. The debugging output is rather coarse and not really intended for normal use at this point in time.

6.3 Application examples

If you want to figure out if a break was taken at a specific point, e.g., whether a heading appears at the top of the page, you can do something like this:

```
\newcounter{breakcounter}
\NewMarkClass{break}
\newcommand\markedbreak[1]{\stepcounter{breakcounter}%
                           \InsertMark{break}{\arabic{breakcounter}%
                           \penalty #1\relax
                           \InsertMark{break}{-\arabic{breakcounter}}
```

To test if the break was taken you can test if `\TopMark{break}` is positive (taken) or negative (not taken) or zero (there was never any marked break so far). The absolute value can be used to keep track of which break it was (with some further coding).

to be extended with additional application examples

6.4 Legacy LaTeX 2ε interface

Here we describe the interfaces that LaTeX 2ε offered since the early nineties and some minor extensions.

6.4.1 Legacy design-level and document-level interfaces

\markboth \markboth {⟨*left*⟩} {⟨*right*⟩}
\markright \markright {⟨*right*⟩}

LaTeX 2ε uses two marks which aren't fully independent. A "left" mark generated by the first argument of \markboth and a "right" mark generated by the second argument of \markboth or by the only argument of \markright. The command \markboth and \markright are in turn called from heading commands such as \chaptermark or \sectionmark and their behavior is controlled by the document class.

For example, in the article class with twoside in force the \sectionmark will issue \markboth with an empty second argument and \subsectionmark will issue \markright. As a result the left mark will contain chapter titles and the right mark subsection titles.

Note, however, that in one-sided documents the standard behavior is that only \markright is used, i.e., there will only be right-marks but no left marks!

\leftmark ⋆ \leftmark
\rightmark ⋆ \rightmark

These functions return the appropriate mark value from the current page and work as before, that is \leftmark will get the last (!) left mark from the page and \rightmark the first (!) right mark.

In other words they work reasonably well if you want to show the section title that is current when you are about to turn the page and also show the first subsection title on the current page (or the last from the previous page if there wasn't one). Other combinations can't be shown using this interface.

The commands are fully expandable, because this is how they have been always defined in LaTeX. However, this is of course only true if the content of the mark they return is itself expandable and does not contain any fragile material. Given that this can't be guaranteed for arbitrary content, a programmer using them in this way should use \protected@edef and *not* \edef to avoid bad surprises as far as this is possible, or use the new interfaces (\TopMark, \FirstMark, and \LastMark) which return the ⟨*text*⟩ in \exp_not:n to prevent uncontrolled expansion.

6.4.2 Legacy interface extensions

The new implementation adds three mark classes: 2e-left, 2e-right and 2e-right-nonempty and patches \markboth and \markright slightly so that they also update these new mark classes, so that the new classes work with existing document classes.

As a result you can use \LastMark{2e-left} and \FirstMark{2e-right} instead of \leftmark and \rightmark. But more importantly, you can use any of the other retrieval commands to get a different status value from those marks, e.g., \LastMark{2e-right} would return the last subsection on the page (instead of the first as returned by \rightmark).

The difference between 2e-right and 2e-right-nonempty is that the latter will only be updated if the material for the mark is not empty. Thus \markboth{title}{} as issued by, say, \sectionmark, sets a 2e-left mark with title and a 2e-right mark with the empty string but does not add a 2e-right-nonempty mark.

Thus, if you have a section at the start of a page and you would ask for \FirstMark{2e-right} you would get an empty string even if there are subsections on that page. But 2e-right-nonempty would then give you the first or last subsection

on that page. Of course, nothing is simple. If there are no subsections it would tell you the last subsection from an earlier page. We therefore need comparison tools, e.g., if top and first are identical you know that the value is bogus, i.e., a suitable implementation would be

```
\IfMarksEqualTF{2e-right-nonempty}{top}{first}
               { <appropriate action if there was no real mark> }
               {\FirstMark{2e-right-nonempty}}
```

6.5 Notes on the mechanism

In contrast to vanilla TEX, ε-TEX extends the mark system to allow multiple independent marks. However, it does not solve the `\topmark` problem which means that LATEX still needs to manage marks almost independently of TEX. The reason for this is that the more complex output routine used by LATEX to handle floats (and related structures) means that `\topmark(s)` remain unreliable. Each time the output routine is fired up, TEX moves `\botmark` to `\topmark`, and while ε-TEX extends this to multiple registers the fundamental concept remains the same. That means that the state of marks needs to be tracked by LATEX itself. An early implementation of this package used TEX's `\botmark` only to ensure the correct interaction with the output routine (this was before the ε-TEX mechanism was even available). However, other than in a prototype implementation for LATEX3, this package was never made public.

The new implementation now uses ε-TEX's marks as they have some advantages, because with them we can leave the mark text within the galley and only extract the marks during the output routine when we are finally shipping out a page or storing away a column for use in the next page. That means we do not have to maintain a global data structure that we have to keep in sync with informational marks in the galley but can rely on everything being in one place and thus manipulations (e.g. reordering of material) will take the marks with them without a need for updating a fragile linkage.

To allow for completely independent marks we use the following procedure:

- For every type of marks we allocate a mark class so that in the output routine TEX can calculate for each class the current top, first, and bottom mark independently. For this we use `\newmarks`, i.e., one marks register per class.
- As already mentioned firing up an output routine without shipping out a page means that TEX's top marks get wrong so it is impossible to rely on TEX's approach directly. What we do instead is to keep track of the real marks (for the last page or more generally last region) in some global variables.
- These variables are updated in the output routine at defined places, i.e., when we do real output processing but not if we use special output routines to do internal housekeeping.
- The trick we use to get correctly updated variables is the following: the material that contains new marks (for example the page to be shipped out) is stored in a box. We then use TEX primitive box splitting functions by splitting off the largest amount possible (which should be the whole box if nothing goes really wrong). While that seems a rather pointless thing to do, it has one important side effect: TEX sets up first and bottom marks for each mark class from the material it has split off. This way we get the first and last marks (if there have been any) from the material in the box.

- The top marks are simply the last marks from the previous page or region. And if there hasn't been a first or bottom mark in the box then the new top mark also becomes new first and last mark for that class.
- That mark data is then stored in global token lists for use during the output routine and legacy commands such as `\leftmark` or new commands such as `\TopMark` simply access the data stored in these token lists.

That's about it in a nutshell. Of course, there are some details to be taken care of—those are discussed in the implementation sections.

6.6 Public interfaces for packages such as multicol

The functions in this section are public so that packages can make use of them. However, this must be done with great care, e.g., `\mark_update_structure_from_material:nn` updates the global mark structure and can therefore be used only in places where such an update is meaningful, e.g., in special output routines. Elsewhere, a change to the mark structure would break the whole mechanism and querying the marks would return incorrect data.

`\mark_update_structure_from_material:nn` `\mark_update_structure_from_material:nn {⟨region⟩} {⟨material with marks⟩}`

Helper function that inspects the marks inside the second argument and assigns new mark values based on that to the ⟨*region*⟩ given in the first argument. For this it first copies the mark structure from ⟨*region*⟩ to `previous-`⟨*region*⟩ and then takes all last mark values currently in the region and makes them the new top mark values. Finally it assigns new first and last values for all mark classes based on what was found in the second argument.

As a consequence, the allowed values for ⟨*region*⟩ are `page` and `column` because only they have `previous-...` counterparts.

Another important aspect to keep in mind is that marks are recognized only if they appear on the top level, e.g., if we want to process material stored in boxes we need to put it unboxed (using `\unvcopy` etc.) into the second argument.

`\mark_copy_structure:nn` `\mark_copy_structure:nn {⟨alias⟩} {⟨source⟩}`

Helper function that copies all mark values in the ⟨*source*⟩ region to ⟨*alias*⟩, i.e., make the structures identical. Used to update the `previous-...` structures inside `\mark_update_structure_from_material:nn` and `first-column` and `last-column` structures inside the internal commands `\__mark_update_singlecol_structures:` or `\__mark__update_dblcol_structures:`.

`\mark_set_structure_to_err:n` `\mark_set_structure_to_err:n {⟨region⟩}`

Helper function that sets all mark values in the ⟨*region*⟩ to an error message. This is currently used for `last-column` at times where using marks from it would be questionable/wrong, i.e., when we have just processed the first column in a two-column document.

\mark_clear_structure:n \mark_clear_structure:n {⟨*region*⟩}

Helper function that sets all mark values in the ⟨*region*⟩ to empty. This is currently used for `column` when a multicol environment starts; this is because it wouldn't make sense if the top mark in the first column returned the last mark from a previous multicol (which may have been much earlier, with intermediate material).

\mark_get_marks_for_reinsertion:nNN \mark_get_marks_for_reinsertion:nNN {⟨*source*⟩}
⟨*token-list-var for collecting first marks*⟩
⟨*token-list-var for collecting last marks*⟩

Helper function for extracting marks that would otherwise get lost, for example when they are hidden inside a box. This helper does not update mark structures and can therefore be used outside the output routine as well.

It collects all the top-level marks from inside the ⟨*source*⟩ and then adds suitable `\mark_insert:nn` commands to each of the two token lists. These token lists can then be executed at the right place to reinsert the marks, e.g., directly after the box. This is, for example, going to be used[37] by multicol when a short balanced `multicols` is returned to the galley for typesetting.

If the ⟨*source*⟩ consists of a single vertical box (plus possibly followed by some glue but nothing else) then the box is unpacked and the top-level marks are collected from its content. However, if it is not a vertical box or there are other data then nothing is unpacked and you have to do the unpacking yourself to get at the marks inside.

It is quite likely that one only needs a single token list for returning the `\mark_insert:nn` statements. If that is the case this command may change to take only two arguments.

6.7 Internal functions for the standard output routine of LaTeX

The functions in this section are tied to the output routine and used in the interface to LaTeX 2ε and perhaps at some later time within a new output routine for LaTeX. They are not (yet) meant for general use and are therefore made internal, even though we already use them in multicol. Internal means that `@@` automatically gets replaced in the code (and in the documentation) so we have to give it a suitable value.

```
1 ⟨@@=mark⟩
```

__mark_update_singlecol_structures: __mark_update_singlecol_structures:

LaTeX 2ε integration function in case we are doing single column layouts. It assumes that the page content is already stored in `\@outputbox` and processes the marks inside that box. It is called as part of `\@opcol`.

__mark_update_dblcol_structures: __mark_update_singlecol_structures:

LaTeX 2ε integration function mark used when we are doing double column documents. It assumes that the page content is already stored in `\@outputbox` and processes the marks inside that box. It then does different post-processing depending on the start of the switch `\if@firstcolumn`. If we are in the second column it also has to update page marks, otherwise it only updates column marks. It too is called as part of `\@opcol`.

[37] Probably not before 2025, though.

Chapter 7

Recording and cross-referencing document properties

7.1 Introduction

The module allows to record the "current state" of various document properties (typically the content of macros and values of counters) and to access them in other places through a label. The list of properties that can be recorded and retrieved are not fix and can be extended by the user. The values of the properties are recorded in the `.aux` file and can be retrieved at the second compilation.

The module uses the ideas of properties and labels. A label is a document reference point: a name for the user. An property is something that LaTeX can track, such as a page number, section number or name. The names of labels and properties may be arbitrary. Note that there is a single namespace for each.

7.2 Design discussion

The design here largely follows ideas from zref. In particular, there are two independent concepts: properties that can be recorded between runs, and labels which consist of lists of these properties. The reason for the split is that individual labels will want to record some but not all properties. For examples, a label concerned with position would track the x and y coordinates of the current point, but not for example the page number.

In the current implementation, properties share a single namespace. This allows multiple lists to re-use the same properties, for example page number, absolute page number, etc. This does mean that *changing* a standard property is an issue. However, some properties have complex definitions (again, see zref at present): having them in a single shared space avoids the need to copy code.

Labels could be implemented as `prop` data. That is not done at present as there is no obvious need to map to or copy the data. As such, faster performance is available using a hash table approach as in a "classical" set up. Data written to the `.aux` file uses simple paired *balanced text* not keyvals: this avoids any restrictions on names and again offers increased performance.

The expl3 versions of the label command do not use `\@bsphack`/`\@esphack` to avoid double spaces, but the LaTeX 2_ε command does as it lives at the document command level.

The reference commands are expandable.

Currently the code has nearly no impact on the main `\label` and `\ref` commands as too many external packages rely on the concrete implementation. There is one exception: the label names share the same namespace. That means that if both `\label{ABC}` and `\RecordProperties{ABC}{page}` are used there is a warning `Label 'ABC' multiply defined`.

7.3 Handling unknown labels and properties

With the standard `\label`/`\ref` commands the requested label is either in the `.aux`-file (and so known) or not. In the first case the stored value can be used, in the second case the reference commands print two question marks.

With flexible property lists a reference commands asks for the value of a specific property stored under a label name and we have to consider more variants:

- If the requested property is unknown (not declared) the system is not correctly set up and an error is issued.
- If the label is unknown, the default of the property is used.
- If the label is known, but doesn't provide a value for the property then again the default of the property is used.
- The command `\property_ref:nnn` allows to give a local default which is used instead of the property default in the two cases before.

7.4 Rerun messages

As the reference commands are expandable they can neither issue a message that the label or the label-property combination is unknown, nor can they trigger the rerun message at the end of the LaTeX run.

Where needed such messages must therefore be triggered manually. For this two commands are provided: `\property_ref_undefined_warn:` and `\property_ref_undefined_warn:nn`. See below for a description.

7.5 Open points

- The `xpos` and `ypos` properties require that the position is stored first but there is no (public) engine independent interface yet. Code must use `\tex_savepos:D`.

7.6 Code interfaces

\property_new:nnnn
\property_gset:nnnn

```
\property_new:nnnn  {⟨property⟩} {⟨setpoint⟩} {⟨default⟩} {⟨code⟩}
\property_gset:nnnn {⟨property⟩} {⟨setpoint⟩} {⟨default⟩} {⟨code⟩}
```

LaTeX 2ε-interface: see \NewProperty, \SetProperty.
Sets the ⟨*property*⟩ to have the ⟨*default*⟩ specified, and at the ⟨*setpoint*⟩ (either now or shipout) to write the result of the ⟨*code*⟩ as part of a label. The ⟨*code*⟩ should be expandable. The expansion of ⟨*code*⟩ (the value of the property) is written to the .aux file and read back from there at the next compilation. Values should assume that the standard LaTeX catcode régime with @ a letter is active then.

If the property is declared within a package it is suggested that its name is build from letters, hyphens and slashes, and is always structured as follows: ⟨*package-name*⟩/⟨*property-name*⟩.

\property_record:nN
\property_record:nn
\property_record:(nV|ee)

```
\property_record:nN {⟨label⟩} ⟨clist var⟩
\property_record:nn {⟨label⟩} {⟨clist⟩}
```

LaTeX 2ε-interface: see \RecordProperties.
Writes the list of properties given by the ⟨*clist*⟩ to the .aux file with the ⟨*label*⟩ specified.

\property_ref:nn ⋆
\property_ref:ee ⋆

```
\property_ref:nn {⟨label⟩} {⟨property⟩}
```

LaTeX 2ε-interface: see \RefProperty.
Expands to the value of the ⟨*property*⟩ for the ⟨*label*⟩, if available, and the default value of the property otherwise. If ⟨*property*⟩ has not been declared with \property_new:nnnn an error is issued. The command raises an internal, expandable, local flag if the reference can not be resolved.

\property_item:nn ⋆
\property_item:ee ⋆

New: 2025-11-20

```
\property_item:nn {⟨label⟩} {⟨property⟩}
```

Retrieves the value of the ⟨*property*⟩ for the ⟨*label*⟩ like \property_ref:nn but the result is returned within the \unexpanded primitive (\exp_not:n), which means that the ⟨*value*⟩ does not expand further when appearing in an e-type or x-type argument expansion. This allows for example to handle values containing user commands which are not safe in an expansion context.

\property_ref:nnn ⋆
\property_ref:een ⋆

```
\property_ref:nnn {⟨label⟩} {⟨property⟩} {⟨local default⟩}
```

LaTeX 2ε-interface: see \RefProperty.
Expands to the value of the ⟨*property*⟩ for the ⟨*label*⟩, if available, and to ⟨*local default*⟩ otherwise. If ⟨*property*⟩ has not been declared with \property_new:nnnn an error is issued. The command raises an internal, expandable local flag if the reference can not be resolved.

\property_item:nnn ⋆
\property_item:een ⋆

New: 2026-05-01

```
\property_item:nnn {⟨label⟩} {⟨property⟩} {⟨local default⟩}
```

Retrieves the value of the ⟨*property*⟩ for the ⟨*label*⟩, if available, and to ⟨*local default*⟩ otherwise, but the result is returned within the \unexpanded primitive (\exp_not:n), which means that the ⟨*value*⟩ does not expand further when appearing in an e-type or x-type argument expansion. This allows for example to handle values containing user commands which are not safe in an expansion context.

\property_ref_undefined_warn: \property_ref_undefined_warn:

LaTeX 2$_\varepsilon$-interface: not provided.
Triggers the standard warning

```
LaTeX Warning: There were undefined references.
```

at the end of the document if there was a recent \property_ref:nn or \property_ref:nnn which couldn't be resolved and so raised the flag. "Recent" means in the same group or in some outer group!

\property_ref_undefined_warn:n \property_ref_undefined_warn:n {⟨label⟩}
\property_ref_undefined_warn:e

LaTeX 2$_\varepsilon$-interface: not provided.
Triggers the standard warning

```
LaTeX Warning: There were undefined references.
```

at the end of the document if ⟨label⟩ is not known. At the point where it is called it also issues the warning

```
Reference '⟨label⟩' on page ⟨page⟩ undefined.
```

\property_ref_undefined_warn:nn \property_ref_undefined_warn:nn {⟨label⟩} {⟨property⟩}
\property_ref_undefined_warn:ee

LaTeX 2$_\varepsilon$-interface: see \RefUndefinedWarn.
Triggers the standard warning

```
LaTeX Warning: There were undefined references.
```

at the end of the document if the reference can not be resolved. At the point where it is called it also issues the warning

```
Reference '⟨label⟩' on page ⟨page⟩ undefined
```

if the label is unknown, or the more specific

```
Property '⟨property⟩' undefined for reference '⟨label⟩' on page ⟨page⟩
```

if the label is known but doesn't provide a value for the requested property.

\property_if_exist_p:n ⋆ \property_if_exist_p:n {⟨property⟩}
\property_if_exist_p:e ⋆ \property_if_exist:nTF {⟨property⟩} {⟨true code⟩} {⟨false code⟩}
\property_if_exist:n*TF* ⋆
\property_if_exist:e*TF* ⋆

LaTeX 2$_\varepsilon$-interface: \IfPropertyExistsTF.
Tests if the ⟨property⟩ has been declared.

\property_if_recorded_p:n ⋆ \property_if_recorded_p:n {⟨label⟩}
\property_if_recorded_p:e ⋆ \property_if_recorded:nTF {⟨label⟩} {⟨true code⟩} {⟨false code⟩}
\property_if_recorded:n*TF* ⋆
\property_if_recorded:e*TF* ⋆

LaTeX 2$_\varepsilon$-interface: \IfLabelExistsTF
Tests if the ⟨label⟩ is known. This is also true if the label has been set with the standard \label command.

\property_if_recorded_p:nn ⋆ \property_if_recorded_p:nn {⟨label⟩} {⟨property⟩}
\property_if_recorded_p:ee ⋆ \property_if_recorded:nnTF {⟨label⟩} {⟨property⟩} {⟨true code⟩} {⟨false code⟩}
\property_if_recorded:nn*TF* ⋆
\property_if_recorded:ee*TF* ⋆

LaTeX 2$_\varepsilon$-interface: \IfPropertyRecordedTF.
Tests if the label ⟨label⟩ is known and if it provides a value of the ⟨property⟩.

7.7 Auxiliary file interfaces

`\new@label@record` `\new@label@record {⟨label⟩} {⟨data⟩}`

This is a command only for use in the `.aux` file. It loads the key–value list of ⟨*data*⟩ to be available for the ⟨*label*⟩.

7.8 LaTeX 2ε interface

The LaTeXe interfaces always expand label and property arguments. This means that one must be careful when using active chars or commands in the names. UTF8-chars are protected and should be safe, similar most babel shorthands.

`\NewProperty` `\NewProperty {⟨property⟩} {⟨setpoint⟩} {⟨default⟩} {⟨code⟩}`
`\SetProperty` `\SetProperty {⟨property⟩} {⟨setpoint⟩} {⟨default⟩} {⟨code⟩}`

Sets the ⟨*property*⟩ to have the ⟨*default*⟩ specified, and at the ⟨*setpoint*⟩ (either `now` or `shipout`) to write the result of the ⟨*code*⟩ as part of a label. The ⟨*code*⟩ should be expandable. The expansion of ⟨*code*⟩ (the value of the property) is written to the `.aux` file and read back from there at the next compilation (at which point normally the standard LaTeX catcode régime with `@` a letter is active).

`\RecordProperties` `\RecordProperties {⟨label⟩} {⟨clist⟩}`

Writes the list of properties given by the ⟨*clist*⟩ to the `.aux` file with the ⟨*label*⟩ specified. Similar to the standard `\label` command the arguments are expanded. So ⟨*clist*⟩ can be a macro containing a list of properties. Also similar to the standard `\label` command, the command is surrounded by an `\@bsphack`/`\@esphack` pair to preserve spacing.

`\RefProperty ⋆` `\RefProperty [⟨local default⟩] {⟨label⟩} {⟨property⟩}`

Expands to the value of the ⟨*property*⟩ for the ⟨*label*⟩, if available, and the default value of the property or – if given – to ⟨*local default*⟩ otherwise. If {⟨*property*⟩} has not been declared an error is issued.

`\IfPropertyExistsTF` `\IfPropertyExistsTF {⟨property⟩} {⟨true code⟩} {⟨false code⟩}`
`\IfPropertyExistsT`
`\IfPropertyExistsF`

Tests if the ⟨*property*⟩ has been declared.

`\IfLabelExistsTF` `\IfLabelExistsTF {⟨label⟩} {⟨true code⟩} {⟨false code⟩}`
`\IfLabelExistsT`
`\IfLabelExistsF`

Tests if the ⟨*label*⟩ has been recorded. This is also true if a label has been set with the standard `\label` command.

`\IfPropertyRecordedTF` `\IfPropertyRecordedTF {⟨label⟩} {⟨property⟩} {⟨true code⟩} {⟨false code⟩}`
`\IfPropertyRecordedT`
`\IfPropertyRecordedF`

Tests if the label and a value of the ⟨*property*⟩ for the ⟨*label*⟩ are both known.

\RefUndefinedWarn `\RefUndefinedWarn {⟨label⟩} {⟨property⟩}`

Triggers the standard warning

`LaTeX Warning: There were undefined references.`

at the end of the document if the reference for ⟨*label*⟩ and ⟨*property*⟩ can not be resolved. At the point where it is called it also issues the warning

`Reference '⟨label⟩' on page ⟨page⟩ undefined`

if the label is unknown, or the more specific

`Property '⟨property⟩' undefined for reference '⟨label⟩' on page ⟨page⟩` if the label is known but doesn't provide a value for the requested property.

7.9 Pre-declared properties

abspage (shipout) The absolute value of the current page: starts at 1 and increases monotonically at each shipout.

page (shipout) The current page as given by `\thepage`: this may or may not be a numerical value, depending on the current style. Contrast with `\abspage`. You get this value also with the standard `\label`/`\pageref`.

pagenum (shipout) The current page as arabic number. This is suitable for integer operations and comparisons.

label (now) The content of `\@currentlabel`. This is the value that you get also with the standard `\label`/`\ref`.

title (now) The content of `\@currentlabelname`. This command is filled beside others by the nameref package and some classes (e.g. memoir).

target (now) The content of `\@currentHref`. This command is normally filled by for example hyperref and gives the name of the last destination it created.

pagetarget (shipout) The content of `\@currentHpage`. This command is filled for example by a recent version of hyperref and then gives the name of the last page destination it created.

counter (now) The content of `\@currentcounter`. This command contains after a `\refstepcounter` the name of the counter.

xpos
ypos

(shipout) This stores the x and y coordinates of a point previously stored with `\pdfsavepos`/`\savepos`. E.g. (if bidi is used it can be necessary to save the position before and after the label):

```
\tex_savepos:D
\property_record:nn{myposition}{xpos,ypos}
\tex_savepos:D
```

Part IV
Design-level tools

Chapter 8

LaTeX's socket management

8.1 Introduction

A LaTeX source file is transformed into a typeset document by executing code for each command or environment in the document source. Through various steps this code transforms the input and eventually generates typeset output appearing in a "galley" from which individual pages are cut off in an asynchronous way. This page generating process is normally not directly associated with commands in the input[38] but is triggered whenever the galley has received enough material to form another page (giving current settings).

As part of this transformation input data may get stored in some form and later reused, for example, as part of the output routine processing.

8.2 Configuration of the transformation process

There are three different major methods offered by LaTeX to configure the transformation process:

- through the template mechanism,
- through the hook mechanism, or
- through sockets and plugs.

They offer different possibilities (with different features and limitations) and are intended for specific use cases, though it is possible to combine them.

8.2.1 The template mechanism

The template mechanism is intended for more complex document-level elements (e.g., headings such as `\section` or environments like `itemize`). The template code implements the overall processing logic for such an element and offers a set of parameters to influence the final result.

The document element is then implemented by (a) selecting a suitable template (there may be more than one available for the kind of document element) and (b) by

[38] Excepts for directives such as `\newpage`.

setting its parameters to desired values. This then forms a so-called instance which is executed when the document element is found in the source.

By altering the parameter values (in a document class or in the document preamble) or, if more drastic layout changes are desired, by selecting a different template and then adjusting its parameters, a wide variety of layouts can be realized through simple configuration setups without the need to develop new code.

The target audience of this method are therefore document class developers or users who wish to alter an existing layout (implemented by a document class) in certain (minor) ways.

The template mechanism is currently documented as part of the `xtemplate` package and one more elaborate implementation can be found as part of the `latex-lab` code for lists (to be documented further).

8.2.2 The hook mechanism

Hooks are places in the kernel code (or in packages) that offer packages the possibility to inject additional code at specific points in the processing in a controlled way without the need to replace the existing code block (and thereby overwriting modifications/extensions made by other packages). The target audience is therefore mainly package developers, even though some hooks can be useful for document authors.

Obviously, what can reasonably be added into a hook depends on the individual hook (hopefully documented as part of the hook documentation), but in general the idea behind hooks is that more than one package could add code into the hook at the same time. Perhaps the most famous hook (that LaTeX had for a very long time) is `begindocument` into which many packages add code to through `\AtBeginDocument{`⟨*code*⟩`}` (which is nowadays implemented as a shorthand for `\AddToHook{begindocument}{`⟨*code*⟩`}`). To resolve possible conflicts between injections by different packages there is a rule mechanism by which code chunks in a hook can be ordered in a certain way and by which incompatible packages can be detected if a resolution is impossible.

In contrast to template code, there is no standard configuration method through parameters for hooks, i.e., the code added to a hook "is" the configuration. If it wants to provide for configuration through parameters it has to also provide its own method to set such parameters in some way. However, in that case it is likely that using a hook is not the right approach and the developer better calls a template instance instead which then offers configuration through a key/value interface.

In most cases, hooks do not take any arguments as input. Instead, the data that they can (and are allowed to) access depends on the surrounding context.

For example, the various hooks available during the page shipout process in LaTeX's output routine can (and have to) access the accumulated page material stored in a box named `\ShipoutBox`. This way, code added to, say, the `shipout/before` hook could access the page content, alter it, and then write it back into `\ShipoutBox` and any other code added to this hook could then operate on the modified content. Of course, for such a scheme to work the code prior to executing the hook would need to setup up data in appropriate places and the hook documentation would need to document what kind of storage can be accessed (and possibly altered) by the hook.

There are also hooks that take arguments (typically portions of document data) and in that case the hook code can access these arguments through `#1`, `#2`, etc.

The hook mechanism is documented in `lthooks-doc.pdf`.

8.2.3 The socket mechanism

In some cases there is code that implements a certain programming logic (for example, combining footnotes, floats, and the text for the current page to be shipped out) and if this logic should change (e.g., footnotes to be placed above bottom floats instead of below) then this whole code block needs to be replaced with different code.

In theory, this could be implemented with templates, i.e., the code simply calls some instance that implements the logic and that instance is altered by selecting a different templates and/or adjusting their parameters. However, in many cases customization through parameters is overkill in such a case (or otherwise awkward, because parameterization is better done on a higher level instead of individually for small blocks of code) and using the template mechanism just to replace one block of code with a different one results in a fairly high performance hit. It is therefore usually not a good choice.

In theory, it would also be possible to use a hook, but again that is basically a misuse of the concept, because in this use case there should never be more than one block of code inside the hook; thus, to alter the processing logic one would need to set up rules that replace code rather than (as intended) execute all code added to the hook.

For this reason LaTeX now offers a third mechanism: "sockets" into which one can place exactly one code block — a "plug".

In a nutshell: instead of having a fixed code block somewhere as part of the code, implementing a certain programming logic there is a reference to a named socket at this point. This is done by first declaring the named socket with:

`\NewSocket{`⟨*socket-name*⟩`}{`⟨*number-of-inputs*⟩`}`

This is then referenced at the point where the replaceable code block should be executed with:

`\UseSocket{`⟨*socket-name*⟩`}`

or, if the socket should take a number of inputs (additional arguments beside the name) with

`\UseSocket{`⟨*socket-name*⟩`}{`⟨arg_1⟩`}...{`⟨$arg_{\langle number\text{-}of\text{-}inputs \rangle}$⟩`}`

In addition, several code blocks (a.k.a. plugs) implementing different logic for this socket are set up, each with a declaration of the form:

`\NewSocketPlug{`⟨*socket-name*⟩`}{`⟨*socket-plug-name*⟩`}{`⟨*code*⟩`}`

Finally, one of them is assigned to the socket:

`\AssignSocketPlug{`⟨*socket-name*⟩`}{`⟨*socket-plug-name*⟩`}`

If the programming logic should change, then all that is necessary is to make a new assignment with `\AssignSocketPlug` to a different `{`⟨*socket-plug-name*⟩`}`. This assignment obeys scope so that an environment can alter a socket without the need to restore the previous setting manually.

If the socket takes inputs, then those need to be provided to `\UseSocket` and in that case they can be referenced in the ⟨*code*⟩ argument of `\NewSocketPlug` with `#1`, `#2`, etc.

In most cases a named socket is used only in a single place, but there is, of course, nothing wrong with using it in several places, as long as the code in all places is supposed to change in the same way.

Examples

We start by declaring a new socket named `foo` that expects two inputs:

```
\NewSocket{foo}{2}
```

Such a declaration has do be unique across the whole LaTeX run. Thus, if another package attempts to use the same name (regardless of the number of inputs) it will generate an error:

```
\NewSocket{foo}{2}
\NewSocket{foo}{1}
```

Both declarations would therefore produce:

```
! LaTeX socket Error: Socket 'foo' already declared!
```

You also get an error if you attempt to declare some socket plug and the socket name is not yet declared, e.g.,

```
\NewSocketPlug{baz}{undeclared}{some code}
```

generates

```
! LaTeX socket Error: Socket 'baz' undeclared!
```

Setting up plugs for the socket is done like this:

```
\NewSocketPlug{foo}{plug-A}
   {\begin{quote}\itshape foo-A: #1!#2\end{quote}}
\NewSocketPlug{foo}{plug-B}
   {\begin{quote}\sffamily foo-B: #2\textsuperscript{2}\end{quote}}
```

This will set up the plugs `plug-A` and `plug-B` for this socket.

We still have to assign one or the other to the socket, thus without doing that the line

```
\UseSocket{foo}{hello}{world}
```

produces nothing because the default plug for sockets with 2 inputs is `noop` (which grabs the additional arguments and throws them away).[39]

So let's do the assignment

```
\AssignSocketPlug{foo}{plug-A}
```

and then

```
\UseSocket{foo}{hello}{world}
```

will properly typeset

foo-A: hello!world

and after

```
\AssignSocketPlug{foo}{plug-B}
```

[39] If socket `foo` would have been a socket with one input, then the default plug would be `identity`, in which case the socket input would remain without braces and gets typeset!

and another call to

```
\UseSocket{foo}{hello}{world}
```

we get

foo-B: world[2]

If we attempt to assign a plug that was not defined, e.g.,

```
\AssignSocketPlug{foo}{plug-C}
```

then we get an error during the assignment

```
! LaTeX socket Error: Plug 'plug-C' for socket 'foo' undeclared!
```

and the previous assignment remains in place.

To see what is known about a socket and its plugs you can use `\ShowSocket` or `\LogSocket` which displays information similar to this on the terminal or in the transcript file:

```
Socket foo:
    number of inputs = 2
    available plugs = noop, plug-A, plug-B
    current plug = plug-B
    definition = \long macro:#1#2->\begin {quote}\sffamily
foo-B: #2\textsuperscript {2}\end {quote}
```

Details and semantics

In this section we collect some normative statements.

- From a functional point of view sockets are like simple TEX macros, i.e., they expect 0 to 9 mandatory arguments (the socket inputs) and get replaced by their "expansion"
- A socket is "named" and the name consists of ASCII letters `[a-z]`, `[A-Z]`, `[0-9]`, `[-/@]` only
- Socket names have to be unique, i.e., there can be only one socket named ⟨*name*⟩. This is ensured by declaring each socket with `\NewSocket`.

  However, there is no requirement that sockets and hook names have to be different. In fact, if a certain action that could otherwise be specified as hook code has to be executed always last (or first) one could ensure this by placing a socket (single action) after a hook (or vice versa) and using the same name to indicate the relationship, e.g.,

  ```
  \UseHook{foo}        % different package can add code here
  \UseSocket{foo}      % only one package can assign a plug
  ```

  This avoids the need to order the hook code to ensure that something is always last.
- Best practice naming conventions are ... *to be documented*

- A socket has documented inputs which are
  - the positional arguments (if any) with a description of what they contain when used
  - implicit data (registers and other 2e/expl3 data stores) that the socket is allowed to make use of, with a documented description of what they contain (if relevant for the task at hand—no need to describe the whole LaTeX universe)
  - information about the state of the TeX engine (again when relevant), e.g. is called in mmode or vmode or in the output routine or ...
  - ... anything missing?
- A socket has documented results/outputs which can be
  - what kind of data it should write to the current list (if that is part of its task)
  - what kind of registers and other 2e/expl3 data stores it should modify and in what way
  - what kind of state changes it should do (if any)
  - *... anything else?*
- At any time a socket has one block of code (a plug :-)) associated with it. Such code is itself named and the association is done by linking the socket name to the code name (putting a plug into the socket).
- The name of a plug consists of ASCII letters `[a-z]`, `[A-Z]`, `[0-9]`, `[-/@]` only.
- Socket plug names have to be unique within on a per socket basis, but it is perfectly allowed (and sensible in some cases) to use the same plug name with different sockets (where based on the sockets' purposes, different actions may be associated with the plug name). For example `noop` is a plug name declared for every socket, yet it action "grab the socket inputs and throw them away" obviously differs depending on how many inputs the socket has.
- When declaring a plug it is stated for which socket it is meant (i.e., its code can only be used with that socket). This means that the same plug name can be used with different sockets referring to different code in each case.
- Configuration of a socket can only be done by linking different code to it. Nevertheless the code linked to it can provide its own means of configuration (but this is outside of the spec).
- Technically execution of a socket (`\UseSocket`) involves
  - doing any house keeping (like writing debugging info, ...);
  - looking up the current code association (what plug is in the socket);
  - executing this code which will pick up the mandatory arguments (happens at this point, not before), i.e., it is like calling a csname defined with

    ```
    \def\foo#1#2...{...#1...#2...}
    ```
  - do some further house keeping (if needed).
- A socket is typically only used in one place in code, but this is not a requirement, i.e., if the same operation with the same inputs need to be carried out in several places the same named socket can be used.

8.2.4 Socket and plug names

The ⟨*socket-name*⟩ and ⟨*socket-plug-name*⟩ are always expanded using TEX's `\csname...\endcsname`, as such, (expandable) commands and Unicode characters are allowed in ⟨*hook*⟩ and ⟨*label*⟩ arguments.

Command syntax

We give both the LATEX 2ε and the L3 programming layer command names.

`\NewSocket` `\NewSocket {⟨socket-name⟩} {⟨number-of-inputs⟩}`
`\socket_new:nn` `\socket_new:nn {⟨socket-name⟩} {⟨number-of-inputs⟩}`

Declares a new socket with name ⟨*socket-name*⟩ having ⟨*number-of-inputs*⟩ inputs. There is automatically a plug `noop` declared for it, which does nothing, i.e., it gobbles the socket inputs (if any). This is made the default plug except for sockets with one input which additionally define the plug `identity` and assign that as their default.

This `identity` plug simply returns the socket input without its outer braces. The use case for this plug are situations like this:

`\UseSocket{tagsupport/footnote}{`⟨*code*⟩`}`

If tagging is not active and the socket contains the plug `identity` then this returns ⟨*code*⟩ without the outer braces and to activate tagging all that is necessary is to change the plug to say `tagpdf` so that it surrounds ⟨*code*⟩ by some tagging magic. This is the most common use case for sockets with one input, which is why they have this special default.

The socket documentation should describe its purpose, its inputs and the expected results as discussed above.

The declaration is only allowed at top-level, i.e., not inside a group.

`\NewSocketPlug` `\NewSocketPlug {⟨socket-name⟩} {⟨socket-plug-name⟩} {⟨code⟩}`
`\socket_new_plug:nnn` `\socket_new_plug:nnn {⟨socket-name⟩} {⟨socket-plug-name⟩} {⟨code⟩}`
`\socket_set_plug:nnn` `\socket_set_plug:nnn {⟨socket-name⟩} {⟨socket-plug-name⟩} {⟨code⟩}`

Declares a new plug for socket ⟨*socket-name*⟩ that runs ⟨*code*⟩ when executing. It complains if the plug was already declared previously.

The form `\socket_set_plug:nnn` changes an existing plug. As this should normally not be necessary, we currently have only an L3 layer name for the few cases it might be useful.

The declarations can be made inside a group and obey scope, i.e., they vanish if the group ends.

`\AssignSocketPlug` `\AssignSocketPlug {⟨socket-name⟩} {⟨socket-plug-name⟩}`
`\socket_assign_plug:nn` `\socket_assign_plug:nn {⟨socket-name⟩} {⟨socket-plug-name⟩}`

Assigns the plug ⟨*socket-plug-name*⟩ to the socket ⟨*socket-name*⟩. It errors if either socket or plug is not defined.

The assignment is local, i.e., it obeys scope.

\UseSocket
\socket_use:nw
\socket_use:n
\socket_use:nn
\socket_use:nnn
\socket_use:nnnn

```
\UseSocket      {⟨socket-name⟩}
\socket_use:nnn {⟨socket-name⟩} {⟨socket-arg1⟩} {⟨socket-arg2⟩}
```

Executes the socket ⟨*socket-name*⟩ by retrieving the ⟨*code*⟩ of the current plug assigned to the socket. This is the only command that would appear inside macro code in packages.

For performance reasons there is no explicit check that the socket was declared!

The different L3 programming layer commands are really doing the same thing: they grab as many arguments as defined as inputs for the socket and then pass them to the plug. The different names are only there to make the code more readable, i.e., to indicate how many arguments are grabbed in total (note that no runtime check is made to verify that this is actually true). We only provide them for sockets with up to 3 inputs (most likely those with zero or one input would have been sufficient). If you happen to have a socket with more inputs, use `\socket_use:nw`.

\socket_use_expandable:nw ⋆
\socket_use_expandable:n ⋆

```
\socket_use_expandable:n {⟨socket-name⟩}
```

Fully expandable variant of `\socket_use:n`. This can be used in macro code to retrieve code from sockets which need to appear in an expandable context.

This usually requires the plug to only contain expandable code and should therefore only be used for sockets which are clearly documented to be used in an expandable context. This command does not print any debugging info when `\DebugSocketsOn` is active and should therefore be avoided whenever possible.

For performance reasons there is no explicit check that the socket was declared!

\ShowSocket
\LogSocket
\socket_show:n
\socket_log:n

```
\ShowSocket    {⟨socket-name⟩}
\socket_show:n {⟨socket-name⟩}
```

Displays information about the socket ⟨*socket-name*⟩ and its state then stops and waits for further instructions — at the moment some what rudimentary.

`\LogSocket` and `\socket_log:n` only differ in that they don't stop.

It is sometimes necessary/helpful to know if a particular socket or plug exists (or is assigned to a certain socket) and based on that take different actions.

\IfSocketExistsTF ⋆
\socket_if_exist:nTF ⋆

```
\IfSocketExistsTF {⟨socket-name⟩} {⟨true code⟩} {⟨false code⟩}
```

If socket ⟨*socket-name*⟩ exists then execute ⟨*true code*⟩ otherwise ⟨*false code*⟩. Variants with only `T` or `F` are also available.

\IfSocketPlugExistsTF ⋆
\socket_if_plug_exist:nnTF ⋆

```
\IfSocketPlugExistsTF {⟨socket-name⟩} {⟨plug-name⟩}
      {⟨true code⟩} {⟨false code⟩}
```

If plug ⟨*plug-name*⟩ for socket ⟨*socket-name*⟩ exists then execute ⟨*true code*⟩ otherwise ⟨*false code*⟩. Variants with only `T` or `F` are also available.

\IfSocketPlugAssignedTF ⋆
\socket_if_plug_assigned:nnTF ⋆

```
\IfSocketPlugAssignedTF {⟨socket-name⟩} {⟨plug-name⟩}
      {⟨true code⟩} {⟨false code⟩}
```

If plug ⟨*plug-name*⟩ is assigned to socket ⟨*socket-name*⟩ then execute ⟨*true code*⟩ otherwise ⟨*false code*⟩. Variants with only `T` or `F` are also available.

\socket_get_plug:nN `\socket_get_plug:nN {⟨socket-name⟩} {⟨tl var⟩}`

This stores into ⟨*tl var*⟩ the name of the plug currently assigned to the socket ⟨*socket-name*⟩. If the socket is not declared an error is issued.

\DebugSocketsOn
\DebugSocketsOff
\socket_debug_on:
\socket_debug_off:

`\DebugSocketsOn ... \DebugSocketsOff`

Turns debugging of sockets on or off.

Rationale for error handling

The errors during the declarations are produced to help with typos—after all, such declarations might be part of a document preamble (not that likely, but possible). However, `\UseSocket` is not doing much checking, e.g.,

```
\UseSocket{mispelled-socket}{hello}{world}
```

will generate a rather low-level error and then typesets "helloworld" because there is no dedicated runtime check if `mispelled-socket` is a known socket.

The reason is that if the misspelling is in the code, then this is a programming error in the package and for speed reasons LaTeX does not repeately make runtime checks for coding errors unless they can or are likely to be user introduced.

Chapter 9

Templates: Prototype document functions

9.1 Introduction

There are three broad "layers" between putting down ideas into a source file and ending up with a typeset document. These layers of document writing are

1. authoring of the text with mark-up;
2. document layout design;
3. implementation (with TEX programming) of the design.

We write the text as an author, and we see the visual output of the design after the document is generated; the TEX implementation in the middle is the glue between the two.

LATEX's greatest success has been to standardize a system of mark-up that balances the trade-off between ease of reading and ease of writing to suit almost all forms of technical writing. It's other original strength was a good background in typographical design; while the standard LATEX 2_ε classes look somewhat dated now in terms of their visual design, their typography is generally sound (barring the occasional minor faults).

However, LATEX 2_ε has always lacked a standard approach to customising the visual design of a document. Changing the looks of the standard classes involved either:

- Creating a new version of the implementation code of the class and editing it.
- Loading one of the many packages to customise certain elements of the standard classes.
- Loading a completely different document class, such as KOMA-Script or memoir, that allows easy customization.

All three of these approaches have their drawbacks and learning curves.

The idea behind lttemplates is to cleanly separate the three layers introduced at the beginning of this section, so that document authors who are not programmers can easily change the design of their documents. lttemplates also makes it easier for LATEX programmers to provide their own customizations on top of a pre-existing class.

9.2 What is a document?

Besides the textual content of the words themselves, the source file of a document contains mark-up elements that add structure to the document. These elements include sectional divisions, figure/table captions, lists of various sorts, theorems/proofs, and so on. The list will be different for every document that can be written.

Each element can be represented logically without worrying about the formatting, with mark-up such as `\section`, `\caption`, `\begin{enumerate}` and so on. The output of each one of these document elements will be a typeset representation of the information marked up, and the visual arrangement and design of these elements can vary widely in producing a variety of desired outcomes.

For each type of document element, there may be design variations that contain the same sort of information but present it in slightly different ways. For example, the difference between a numbered and an unnumbered section, `\section` and `\section*`, or the difference between an itemized list or an enumerated list.

There are three distinct layers in the definition of "a document" at this level

1. semantic elements such as the ideas of sections and lists;
2. a set of design solutions for representing these elements visually;
3. specific variations for these designs that represent the elements in the document.

In the parlance of the template system, these are called types, templates, and instances, and they are discussed below in sections 9.4, 9.5, and 9.7, respectively.

9.3 Types, templates, and instances

By formally declaring documents to be composed of mark-up elements grouped into types, which are interpreted and typeset with a set of templates, each of which has one or more instances with which to compose each and every semantic unit of the text, we can cleanly separate the components of document construction.

All of the structures provided by the template system are global, and do not respect TEX grouping.

9.4 Template types

An *template type* (sometimes just "type") is an abstract idea of a document element that takes a fixed number of arguments corresponding to the information from the document author that it is representing. A sectioning type, for example, might take three inputs: "title", "short title", and "label".

Any given document class will define which types are to be used in the document, and any template of a given type can be used to generate an instance for the type. (Of course, different templates will produce different typeset representations, but the underlying content will be the same.)

\NewTemplateType `\NewTemplateType {⟨template type⟩} {⟨no. of args⟩}`

This function defines an ⟨*template type*⟩ taking ⟨*number of arguments*⟩, where the ⟨*type*⟩ is an abstraction as discussed above. For example,

```
\NewTemplateType{sectioning}{3}
```

creates a type "sectioning", where each use of that type will need three arguments.

9.5 Templates

A *template* is a generalized design solution for representing the information of a specified type. Templates that do the same thing, but in different ways, are grouped together by their type and given separate names. There are two important parts to a template:

- the parameters it takes to vary the design it is producing;
- the implementation of the design.

As a document author or designer does not care about the implementation but rather only the interface to the template, these two aspects of the template definition are split into two independent declarations, `\DeclareTemplateInterface` and `\DeclareTemplateCode`.

\DeclareTemplateInterface `\DeclareTemplateInterface {⟨type⟩} {⟨template⟩} {⟨no. of args⟩} {⟨key list⟩}`

A ⟨*template*⟩ interface is declared for a particular ⟨*type*⟩, where the ⟨*number of arguments*⟩ must agree with the type declaration. The interface itself is defined by the ⟨*key list*⟩, which is itself a key–value list taking a specialized format:

```
⟨key1⟩ : ⟨key type1⟩ ,
⟨key2⟩ : ⟨key type2⟩ ,
⟨key3⟩ : ⟨key type3⟩ = ⟨default3⟩ ,
⟨key4⟩ : ⟨key type4⟩ = ⟨default4⟩ ,
...
```

Each ⟨*key*⟩ name should consist of ASCII characters, with the exception of `,`, `=` and `:`. The recommended form for key names is to use lower case letters, with dashes to separate out different parts. Spaces are ignored in key names, so they can be included or missed out at will. Each ⟨*key*⟩ must have a ⟨*key type*⟩, which defines the type of input that the ⟨*key*⟩ requires. A full list of key types is given in Table 1. Each key may have a ⟨*default*⟩ value, which will be used in by the template if the ⟨*key*⟩ is not set explicitly. The ⟨*default*⟩ should be of the correct form to be accepted by the ⟨*key type*⟩ of the ⟨*key*⟩: this is not checked by the code. Expressions for numerical values are evaluated when the template is used, thus for example values given in terms of `em` or `ex` will be set respecting the prevailing font.

Key-type	Description of input
`boolean`	`true` or `false`
`choice{`⟨*choices*⟩`}`	A list of pre-defined ⟨*choices*⟩
`commalist`	A comma-separated list
`function{`⟨N⟩`}`	A (protected) function definition with N arguments (N from 0 to 9)
`instance{`⟨*name*⟩`}`	An instance of type ⟨*name*⟩
`integer`	An integer or integer expression
`length`	A fixed length
`muskip`	A math length with shrink and stretch components
`real`	A real (floating point) value
`skip`	A length with shrink and stretch components
`tokenlist`	A token list: any text or commands

Table 1: Key-types for defining template interfaces with `\DeclareTemplateInterface`.

`\KeyValue` `\KeyValue {`⟨*key name*⟩`}`

There are occasions where the default (or value) for one key should be taken from another. The `\KeyValue` function can be used to transfer this information without needing to know the internal implementation of the key:

```
\DeclareTemplateInterface { type } { template } { no. of args }
  {
    key-name-1 : key-type = value ,
    key-name-2 : key-type = \KeyValue { key-name-1 },
    ...
  }
```

Key-type	Description of binding
`boolean`	Boolean variable, *e.g.* `\l_tmpa_bool`
`choice`	List of choice implementations (see Section 9.6)
`commalist`	Comma list, *e.g.* `\l_tmpa_clist`
`function`	Function taking N arguments, *e.g.* `\use_i:nn`
`instance`	
`integer`	Integer variable, *e.g.* `\l_tmpa_int`
`length`	Dimension variable, *e.g.* `\l_tmpa_dim`
`muskip`	Muskip variable, *e.g.* `\l_tmpa_muskip`
`real`	Floating-point variable, *e.g.* `\l_tmpa_fp`
`skip`	Skip variable, *e.g.* `\l_tmpa_skip`
`tokenlist`	Token list variable, *e.g.* `\l_tmpa_tl`

Table 2: Bindings required for different key types when defining template implementations with `\DeclareTemplateCode`. Apart from `choice` and `function` all of these accept the keyword `global` to carry out a global assignment. Apart from `choice`, all of these accept the keyword `name` to allow passing command name without the leading backslash: this is useful for auto-generated names.

`\DeclareTemplateCode`

```
\DeclareTemplateCode
  {⟨type⟩} {⟨template⟩} {⟨no. of args⟩}
  {⟨key bindings⟩} {⟨code⟩}
```

The relationship between a templates keys and the internal implementation is created using the `\DeclareTemplateCode` function. As with `\DeclareTemplateInterface`, the ⟨*template*⟩ name is given along with the ⟨*type*⟩ and ⟨*number of arguments*⟩ required. The ⟨*key bindings*⟩ argument is a key–value list which specifies the relationship between each ⟨*key*⟩ of the template interface with an underlying ⟨*variable*⟩.

⟨*key1*⟩ = ⟨*variable1*⟩,
⟨*key2*⟩ = ⟨*variable2*⟩,
⟨*key3*⟩ = global ⟨*variable3*⟩,
⟨*key4*⟩ = global ⟨*variable4*⟩,
⟨*key5*⟩ = name { ⟨*name5*⟩ },
...

As illustrated, the keyword `global` may be included in the listing to indicate that the ⟨*variable*⟩ should be assigned globally. The keyword `name` may also be used to give the variable by (cs)name rather than as a single token. If both keywords are used, `global` should come before `name`.

A full list of variable bindings is given in Table 2.

The ⟨*code*⟩ argument of `\DeclareTemplateCode` is used as the replacement text for the template when it is used, either directly or as an instance. This may therefore accept arguments `#1`, `#2`, *etc.* as detailed by the ⟨*number of arguments*⟩ taken by the type.

\AssignTemplateKeys `\AssignTemplateKeys`

In the final argument of `\DeclareTemplateCode` the assignment of keys defined by the template may be delayed by including the command `\AssignTemplateKeys`. If this is *not* present, keys are assigned immediately before the template code. If an `\AssignTemplateKeys` command is present, assignment is delayed until this point. Note that the command must be *directly* present in the code, not placed within a nested command/macro.

`\SetKnownTemplateKeys`	`\SetKnownTemplateKeys {⟨type⟩} {⟨template⟩} {⟨keyvals⟩}`
`\SetTemplateKeys`	`\SetTemplateKeys {⟨type⟩} {⟨template⟩} {⟨keyvals⟩}`
`\SetCurrentKnownTemplateKeys`	`\SetCurrentKnownTemplateKeys {⟨keyvals⟩}`
`\UnusedTemplateKeys`	`\UnusedTemplateKeys % all ⟨keyvals⟩ unused by previous \SetKnownTemplateKeys`

In the final argument of `\DeclareTemplateCode` one can also overwrite (some of) the current template key value settings by using the command `\SetKnownTemplateKeys` or `\SetTemplateKeys`, i.e., they can overwrite the template default values and the values assigned by the instance.

The `\SetKnownTemplateKeys`, `\SetTemplateKeys` and `\SetCurrentKnownTemplateKeys` commands are only supported within the code of a template; using them elsewhere has unpredictable results. If they are used together with `\AssignTemplateKeys` then the latter command should come first in the template code.

The main use case for these commands is the situation where there is an argument (normally `#1`) to the template in which a key/value list can be specified that overwrites the normal settings. In that case one could use

`\SetKnownTemplateKeys{⟨type⟩}{⟨template⟩}{#1}`

to process this key/value list inside the template.

If `\SetKnownTemplateKeys` or `\SetCurrentKnownTemplateKeys` are executed and the ⟨*keyvals*⟩ argument contains keys not known to the ⟨*template*⟩ they are simply ignored and stored in the tokenlist `\UnusedTemplateKeys` without generating an error. This way it is possible to apply the same key/val list specified by the user on a document-level command or environment to several templates, which is useful, if the command or environment is implemented by calling several different template instances.

`\SetCurrentKnownTemplateKeys` works like `\SetKnownTemplateKeys` but obtains the name of the current ⟨*type*⟩/⟨*template*⟩ automatically based on the current context. It should be used before any other template code may be called (and thus change the value of the tracking of the current template).

As a variation of that, you can use this key/val list the first time, and for the next template instance use what remains in `\UnusedTemplateKeys` (i.e., the key/val list with only the keys that have not been processed previously). The final processing step could then be `\SetTemplateKeys`, which unconditionally attempts to set the ⟨*keyvals*⟩ received in its third argument. This command complains if any of them are unknown keys. Alternatively, you could use `\SetKnownTemplateKeys` and afterwards check whether `\UnusedTemplateKeys` is empty.[40]

For example, a list, such as `enumerate`, is made up from a `blockenv`, `block`, `list`,

[40] Using `\SetTemplateKeys` exposes the inner structure of the template keys when generating an error. This is something one may want to avoid as it can be confusing to the user, especially if several templates are involved. In that case use `\SetKnownTemplateKeys` and afterwards check whether `\UnusedTemplateKeys` is empty; if it is not empty then generate your own error message.

and a `para` template and in the single user-supplied optional argument of `enumerate` key/values for any of these templates might be specified.

In fact, in the particular example of list environments, the supplied key/value list is also saved and then applied to each `\item` which is implemented through an `item` template. This way, one can specify one-off settings for all the items of a single list (on the environment level), as well as to individual items within that list (by specifying them in the optional argument of an `\item`). With `\SetKnownTemplateKeys` and `\SetTemplateKeys` working together, it is possible to provide this flexibility and still alert the user when one of their keys is misspelled.

On the other hand you may want to allow for "misspellings" without generating an error or a warning. For example, if you define a template that accepts only a few keys, you might just want to ignore anything specified in the source when you use this template in place of a different one, without the need to alter the document source. Or you might just generate a warning message, which is easy, given that the unused key/values are available in the `\UnusedTemplateKeys` variable.

`\DeclareTemplateCopy`

`\DeclareTemplateCopy`
`{⟨type⟩} {⟨template2⟩} {⟨template1⟩}`

Copies ⟨*template1*⟩ of ⟨*type*⟩ to a new name ⟨*template2*⟩: the copy can then be edited independent of the original.

9.6 Multiple choices

The `choice` key type implements multiple choice input. At the interface level, only the list of valid choices is needed:

```
\DeclareTemplateInterface { foo } { bar } { 0 }
  { key-name : choice { A, B, C } }
```

where the choices are given as a comma-list (which must therefore be wrapped in braces). A default value can also be given:

```
\DeclareTemplateInterface { foo } { bar } { 0 }
  { key-name : choice { A, B, C } = A }
```

At the implementation level, each choice is associated with code, using a nested key–value list.

```
\DeclareTemplateCode { foo } { bar } { 0 }
  {
    key-name =
      {
        A = Code-A ,
        B = Code-B ,
        C = Code-C
      }
   }
   { ... }
```

The two choice lists should match, but in the implementation a special `unknown` choice is also available. This can be used to ignore values and implement an "else" branch:

```
\DeclareTemplateCode { foo } { bar } { 0 }
  {
    key-name =
      {
        A       = Code-A ,
        B       = Code-B ,
        C       = Code-C ,
        unknown = Else-code
      }
   }
   { ... }
```

The `unknown` entry must be the last one given, and should *not* be listed in the interface part of the template.

For keys which accept the values `true` and `false` both the boolean and choice key types can be used. As template interfaces are intended to prompt clarity at the design level, the boolean key type should be favored, with the choice type reserved for keys which take arbitrary values.

9.7 Instances

After a template is defined it still needs to be put to use. The parameters that it expects need to be defined before it can be used in a document. Every time a template has parameters given to it, an *instance* is created, and this is the code that ends up in the document to perform the typesetting of whatever pieces of information are input into it.

For example, a template might say "here is a section with or without a number that might be centered or left aligned and print its contents in a certain font of a certain size, with a bit of a gap before and after it" whereas an instance declares "this is a section with a number, which is centered and set in 12 pt italic with a 10 pt skip before and a 12 pt skip after it". Therefore, an instance is just a frozen version of a template with specific settings as chosen by the designer.

\DeclareInstance `\DeclareInstance`
`{⟨type⟩} {⟨instance⟩} {⟨template⟩} {⟨parameters⟩}`

This function uses a ⟨*template*⟩ for an ⟨*type*⟩ to create an ⟨*instance*⟩. The ⟨*instance*⟩ will be set up using the ⟨*parameters*⟩, which will set some of the ⟨*keys*⟩ in the ⟨*template*⟩.

As a practical example, consider a type for document sections (which might include chapters, parts, sections, *etc.*), which is called `sectioning`. One possible template for this type might be called `basic`, and one instance of this template would be a numbered section. The instance declaration might read:

```
\DeclareInstance { sectioning } { section-num } { basic }
  {
    numbered      = true ,
    justification = center ,
    font          =\normalsize\itshape ,
    before-skip   = 10pt ,
    after-skip    = 12pt ,
  }
```

Of course, the key names here are entirely imaginary, but illustrate the general idea of fixing some settings.

\InstanceValue ⋆ `\InstanceValue {⟨type⟩} {⟨instance⟩} {⟨key⟩}`

Expands to the current value for the ⟨*key*⟩ stored in the ⟨*instance*⟩ of ⟨*type*⟩. If the ⟨*instance*⟩ or ⟨*key*⟩ do not exist, the expansion is empty. The result is returned within the `\unexpanded` primitive (`\exp_not:n`).

\IfInstanceExistsT
\IfInstanceExistsF
\IfInstanceExistsTF
`\IfInstanceExistsTF {⟨type⟩} {⟨instance⟩} {⟨true code⟩} {⟨false code⟩}`

Tests if the named ⟨*instance*⟩ of a ⟨*type*⟩ exists, and then inserts the appropriate code into the input stream.

\DeclareInstanceCopy `\DeclareInstanceCopy`
`{⟨type⟩} {⟨instance2⟩} {⟨instance1⟩}`

Copies the ⟨*values*⟩ for ⟨*instance1*⟩ for an ⟨*type*⟩ to ⟨*instance2*⟩.

9.8 Document interface

After the instances have been chosen, document commands must be declared to use those instances in the document. `\UseInstance` calls instances directly, and this command should be used internally in document-level mark-up.

\UseInstance `\UseInstance`
`{⟨type⟩} {⟨instance⟩} ⟨arguments⟩`

Uses an ⟨*instance*⟩ of the ⟨*type*⟩, which will require ⟨*arguments*⟩ as determined by the number specified for the ⟨*type*⟩. The ⟨*instance*⟩ must have been declared before it can be used, otherwise an error is raised.

\UseTemplate `\UseTemplate {⟨type⟩} {⟨template⟩} {⟨settings⟩} ⟨arguments⟩`

Uses the ⟨*template*⟩ of the specified ⟨*type*⟩, applying the ⟨*settings*⟩ and absorbing ⟨*arguments*⟩ as detailed by the ⟨*type*⟩ declaration. This in effect is the same as creating an instance using `\DeclareInstance` and immediately using it with `\UseInstance`, but without the instance having any further existence. This command is therefore useful when a template needs to be used only once.

This function can also be used as the argument to `instance` key types:

```
\DeclareInstance { type } { template } { instance }
  {
    instance-key =
      \UseTemplate { type2 } { template2 } { <settings> }
  }
```

9.9 Changing existing definitions

Template parameters may be assigned specific defaults for instances to use if the instance declaration doesn't explicit set those parameters. In some cases, the document designer will wish to edit these defaults to allow them to "cascade" to the instances. The alternative would be to set each parameter identically for each instance declaration, a tedious and error-prone process.

\EditTemplateDefaults `\EditTemplateDefaults {⟨type⟩} {⟨template⟩} {⟨new defaults⟩}`

Edits the ⟨*defaults*⟩ for a ⟨*template*⟩ for an ⟨*type*⟩. The ⟨*new defaults*⟩, given as a key–value list, replace the existing defaults for the ⟨*template*⟩. This means that the change will apply to instances declared after the editing, but that instances which have already been created are unaffected.

\EditInstance `\EditInstance {⟨type⟩} {⟨instance⟩} {⟨new values⟩}`

Edits the ⟨*values*⟩ for an ⟨*instance*⟩ for an ⟨*type*⟩. The ⟨*new values*⟩, given as a key–value list, replace the existing values for the ⟨*instance*⟩. This function is complementary to `\EditTemplateDefaults`: `\EditInstance` changes a single instance while leaving the template untouched.

9.9.1 Expanding the values of keys

To allow the user to apply expansion of values when the key is set, key names can be followed by an expansion specifier. This is given by appending `:` and a single letter specifier to the key name. These letters are the normal argument specifiers for expl3, thus they may be one of `n` (redundant but supported), `o`, `V`, `v`, `e`, `N` (again redundant) or `c`. Expansion of a control sequence name is particularly useful when you need to refer to an internal LaTeX 2_ε or an L3 programming layer variable, e.g.,

```
key-a:c =  @itemdepth , % use \@itemdepth as the value
key-b:v =  @itemdepth   % use the current value of \@itempdepth as the value
```

9.10 Creating equivalent keys

There are times when it is useful to have different names which map to the same underlying key interfaces, for example where a more complex higher level name is implemented by several simpler underlying keys, or where a key name needs to be changed. To support this, it is possible to define equivalent keys using `\DeclareTemplateInterface`.

`\DeclareTemplateInterfaceEquivalents` `\DeclareTemplateInterfaceEquivalents {⟨type⟩} {⟨template⟩} [⟨deprecation-date⟩] {⟨key list⟩}`

Defines one or more keys to act as equivalents to the base keys of the ⟨*template*⟩. As for `\DeclareTemplateInterface`, these take a specialized format:

⟨*key1*⟩ : ⟨*key type1*⟩ = ⟨*definition1*⟩ ,
⟨*key2*⟩ : ⟨*key type2*⟩ = ⟨*definition2*⟩ ,
...

The ⟨*key type*⟩ of the key being created should be one of

- `alias`: A direct equivalence between the key being created and a single parent
- `meta`: An equivalence to one or more parent keys, where additional code may be included as well as the value passed to the key being created; the value (provided in the instance declaration or in the ad hoc setting in the document) is available as `#1`.

for example

```
\DeclareTemplateInterfaceEquivalents { type } { template }
  {
    alias-key-one : alias = base-key-one ,
    alias-key-two : meta =
      { parent-key-two = {#1}, base-key-three = \hspace{#1} }
  }
```

Equivalent key names may be reused to overwrite existing equivalent keys but cannot be the same as base keys of the template.

Optionally, the ⟨*deprecation date*⟩ may be set. Before this date, a warning will be printed concerning the keys; after this date, an error will be issued. In both cases, the keys will still be defined.

9.10.1 Modifying existing key values

Some changes to key values are relative adjustments to the previous value, for example to add 1 cm to a margin. To support this, the syntax

```
key-a += 1cm
```

can be usd, with a `+` immediately before the `=` sign. This will cause the new value to be *added* to the existing one. This naturally applies to all numeric key types, and for `tokenlist` and `commalist` means adding the new material to the end of the existing value. An error will arise if this is used for other key types, for example `function`.

9.11 Getting information about templates and instances

\ShowInstanceValues `\ShowInstanceValues {⟨type⟩} {⟨instance⟩}`

Shows the ⟨*values*⟩ for an ⟨*instance*⟩ of the given ⟨*type*⟩ at the terminal.

\ShowTemplateCode `\ShowTemplateCode {⟨type⟩} {⟨template⟩}`

Shows the ⟨*code*⟩ of a ⟨*template*⟩ for an ⟨*type*⟩ in the terminal.

\ShowTemplateDefaults `\ShowTemplateDefaults {⟨type⟩} {⟨template⟩}`

Shows the ⟨*default*⟩ values of a ⟨*template*⟩ for an ⟨*type*⟩ in the terminal.

\ShowTemplateInterface `\ShowTemplateInterface {⟨type⟩} {⟨template⟩}`

Shows the ⟨*keys*⟩ and associated ⟨*key types*⟩ of a ⟨*template*⟩ for an ⟨*type*⟩ in the terminal.

\ShowTemplateVariables `\ShowTemplateVariables {⟨type⟩} {⟨template⟩}`

Shows the ⟨*variables*⟩ and associated ⟨*keys*⟩ of a ⟨*template*⟩ for an ⟨*type*⟩ in the terminal. Note that `code` and `choice` keys do not map directly to variables but to arbitrary code. For `choice` keys, each valid choice is shown as a separate entry in the list, with the key name and choice separated by a space, for example

```
Template 'example' of type 'example' has variable mapping:
>  demo unknown  =>  \def \demo {?}
>  demo c  =>  \def \demo {c}
>  demo b  =>  \def \demo {b}
>  demo a  =>  \def \demo {a}.
```

would be shown for a choice key `demo` with valid choices `a`, `b` and `c`, plus code for an `unknown` branch.

9.12 Debugging support

\DebugTemplatesOn `\DebugTemplatesOn`
\DebugTemplatesOff

Turn on or off debugging support for use of templates. Debugging information is provided at the point of *use* of templates and instances, i.e. where normal messages should be avoided for performance reasons.

Index

The italic numbers denote the pages where the corresponding entry is described, numbers underlined point to the definition, all others indicate the places where it is used.

Symbols

/after (hook) 48, 48, 50
/before (hook) 48, 48
\⟨*addto-cmd*⟩ 21

A

A/foo (hook) 29, 29
\abspage 93
abspage 93
\ActivateGenericHook 20
\AddEverypageHook 76
\AddThispageHook 76
\AddToHook 97
\AddToHookNext 24
\AddToHookNextWithArguments 24
\AddToHookWithArguments 39
after (hook) 43
\AfterEndEnvironment 43
\Alph 74
\arabic 74
\AssignSocketPlug 102
\AssignTemplateKeys 110
\AtBeginDocument 41
\AtBeginDvi 76
\AtBeginEnvironment 43
\AtBeginShipout 76
\AtBeginShipoutAddToBox 75
\AtBeginShipoutAddToBoxForeground . . 75
\AtBeginShipoutBox 75
\AtBeginShipoutDiscard 76
\AtBeginShipoutFirst 76
\AtBeginShipoutInit 76
\AtBeginShipoutNext 76
\AtBeginShipoutOriginalShipout 75
\AtBeginShipoutUpperLeft 75
\AtBeginShipoutUpperLeftForeground . 75
\AtEndDocument 41
\AtEndDvi 72
\AtEndEnvironment 43
\AtEndOfClass 50
\AtEndOfPackage 50
\AtEndPreamble 44
\AtNextShipout 76

B

babel/⟨*language*⟩/afterextras (hook) . 38
\BeforeBeginEnvironment 43
\begin 44
begindocument (hook) 97, 97, 41, 41, 41, 44, 54
begindocument/before (hook) 44
begindocument/end (hook) 44
\bfdefault 46
bfseries (hook) 46, 46
\bfseries 46
bfseries/defaults (hook) 46, 46
bool commands:
  \l_tmpa_bool 109
\BooleanFalse 7
\BooleanTrue 7
\botmark 78
\box 63

C

\caption 55
\chapter 4
\chaptermark 84
class (hook) 38
class/ (hook) 50
class/.../after 50
class/.../before 50
class/⟨*name*⟩/after (hook) 50
class/⟨*name*⟩/before (hook) 50
class/after (hook) 50
class/after 50
class/before (hook) 50
class/before 50
\ClearHookNext 24
\ClearHookRule 28
\clearpage 51
clist commands:
  \l_tmpa_clist 109
cmd (hook) 32, 38, 38, 57, 58
cmd/⟨*name*⟩/after (hook) 43, 54, 57
cmd/⟨*name*⟩/before (hook) 43, 54, 54
cmd/foo/before (hook) 56
cmd/include/after (hook) 51
cmd/include/before (hook) 51
cmd/section/after (hook) 57, 57
cmd/section/before (hook) 57
counter 93
\csname 24

\CurrentFile 49
\CurrentFilePath 49
\CurrentFilePathUsed 49
\CurrentFileUsed 52

D

\DebugHooksOff 31
\DebugHooksOn 31
\DebugMarksOff 83
\DebugMarksOn 83
\DebugShipoutsOff 74
\DebugShipoutsOn 74
\DebugSocketsOff 104
\DebugSocketsOn 104
\DebugTemplatesOff 116
\DebugTemplatesOn 116
\Declare... 4
\DeclareDefaultHookRule 29
\DeclareDocumentCommand 4
\DeclareDocumentEnvironment 4
\DeclareExpandableDocumentCommand . . 12
\DeclareHookRule 29
\DeclareInstance 113
\DeclareInstanceCopy 113
\DeclareTemplateCode 109
\DeclareTemplateCopy 111
\DeclareTemplateInterface 108
\DeclareTemplateInterfaceEquivalents 115
dim commands:
\l_tmpa_dim 109
\DisableGenericHook 58
\DisableHook 20
\DiscardShipoutBox 73
\document 44
\documentclass 26
\dospecials 13

E

\edef 84
\EditInstance 114
\EditTemplateDefaults 114
\end 46
\endcsname 102
enddocument (hook) 40, 41, 45
\enddocument 40
enddocument/afteraux (hook) 45
enddocument/afterlastpage (hook) . . . 45
enddocument/end (hook) 45
enddocument/info (hook) 45
\endgraf 64
env (hook) 29, 38
env/⟨*env*⟩/after (hook) 42, 42, 43
env/⟨*env*⟩/before (hook) 42, 42, 43
env/⟨*env*⟩/begin (hook) 42, 43
env/⟨*env*⟩/end (hook) 42, 43
env/document/after (hook) 46
env/document/before (hook) 44
env/document/begin (hook) 44
env/document/end (hook) 46
env/quote/after (hook) 36, 36
\ERROR 66
\errorstopmode 34
\everypar 62
\EveryShipout 76
exp commands:
\exp_not:n 84
expand@font@defaults (hook) 46
\ExpandArgs 4
\ExplSyntaxOn 55

F

file (hook) 29, 38
file commands:
\l_file_search_path_seq 49
file/.../after 48
file/.../before 48
file/⟨*file name*⟩/after (hook) . . . 49, 49
file/⟨*file name*⟩/before (hook) . . 49, 49
file/⟨*filename*⟩/after (hook) 48
file/⟨*filename*⟩/before (hook) 48
file/⟨*package name*⟩.sty/after (hook) 50
file/⟨*package name*⟩.sty/before (hook) 50
file/after (hook) 48, 49, 50
file/after 48
file/array.sty/after (hook) 49
file/before (hook) 48, 49, 50
file/before 48
\FirstMark 80
\firstmark 78
foo (socket) 99, 99
\foo 7
fp commands:
\l_tmpa_fp 109

G

\glossary 79

H

\hbox 69
hook commands:
\hook_activate_generic:n 32
\hook_debug_off: 34
\hook_debug_on: 34
\hook_disable_generic:n 32
\hook_gclear_next_code:n 33
\hook_gput_code:nnn 33
\hook_gput_code_with_args:nnn . . . 33

\hook_gput_next_code:nn 33
\hook_gput_next_code_with_-
args:nn 33
\hook_gremove_code:nn 33
\hook_gset_rule:nnnn 34
\hook_if_empty:nTF 25
\hook_if_empty_p:n 34
\hook_log:n 34
\hook_new:n 32
\hook_new_pair:nn 31
\hook_new_pair_with_args:nn 32
\hook_new_pair_with_args:nnn . . . 32
\hook_new_reversed:n 31
\hook_new_reversed_with_args:nn . 32
\hook_new_with_args:nn 32
\hook_show:n 34
\hook_use:n 32
\hook_use:nnw 32
\hook_use_once:n 37
\hook_use_once:nnw 33
Hooks:
/after 48, 48, 50
/before 48, 48
A/foo 29, 29
after 43
babel/⟨language⟩/afterextras 38
begindocument 97, 97, 41, 41, 41, 44, 54
begindocument/before 44
begindocument/end 44
bfseries 46, 46
bfseries/defaults 46, 46
class 38
class/ 50
class/⟨name⟩/after 50
class/⟨name⟩/before 50
class/after 50
class/before 50
cmd 32, 38, 38, 57, 58
cmd/⟨name⟩/after 43, 54, 57
cmd/⟨name⟩/before 43, 54, 54
cmd/foo/before 56
cmd/include/after 51
cmd/include/before 51
cmd/section/after 57, 57
cmd/section/before 57
enddocument 40, 41, 45
enddocument/afteraux 45
enddocument/afterlastpage 45
enddocument/end 45
enddocument/info 45
env 29, 38
env/⟨env⟩/after 42, 42, 43
env/⟨env⟩/before 42, 42, 43
env/⟨env⟩/begin 42, 43
env/⟨env⟩/end 42, 43
env/document/after 46
env/document/before 44
env/document/begin 44
env/document/end 46
env/quote/after 36, 36
expand@font@defaults 46
file 29, 38
file/⟨file name⟩/after 49, 49
file/⟨file name⟩/before 49, 49
file/⟨filename⟩/after 48
file/⟨filename⟩/before 48
file/⟨package name⟩.sty/after . . . 50
file/⟨package name⟩.sty/before . . 50
file/after 48, 49, 50
file/array.sty/after 49
file/before 48, 49, 50
include 38, 51
include/⟨name⟩/after 51
include/⟨name⟩/before 51
include/⟨name⟩/end 51
include/⟨name⟩/excluded 51, 51
include/after 51
include/before 51
include/end 51
include/excluded 51, 51
insertmark 79, 47
mdseries 46
mdseries/defaults 46
myhook 22
normalfont 46
package 38
package/ 50
package/⟨name⟩/after 50
package/⟨name⟩/before 50
package/⟨package name⟩/after 50
package/⟨package name⟩/before . . . 50
package/after 50, 50
package/before 50, 50
para/after 63
para/before 63
para/begin 63, 65
para/end 63, 64
rmfamily 46, 46, 46
selectfont 47
sffamily 46
shipout 71, 72, 72, 73, 73
shipout/... . . . . . 70, 73
shipout/after 74, 70, 70, 70, 72, 72, 72, 74
shipout/background 75, 75, 76, 70, 71, 72, 73

shipout/before 74, 76, 76, 97, 70, 70, 70, 70, 71, 71, 71, 72, 72, 72, 73, 73, 73, 73, 74
shipout/firstpage 70, 70, 70, 71, 72, 72, 72
shipout/foreground 75, 75, 76, 70, 71, 72, 73
shipout/lastpage 45, 70, 70, 72, 72, 72, 72, 72
test 39, 39
toplevel 22, 22, 41
ttfamily 46

I

identity (plug) 99, 102, 102, 102
\IfBlankF 7
\IfBlankT 7
\IfBlankTF 7
\IfBooleanF 7
\IfBooleanT 7
\IfBooleanTF 7
\IfHookEmptyF 30
\IfHookEmptyT 30
\IfHookEmptyTF 25
\IfInstanceExistsF 113
\IfInstanceExistsT 113
\IfInstanceExistsTF 113
\IfLabelExistsF 92
\IfLabelExistsT 92
\IfLabelExistsTF 91
\IfMarksEqualF 80
\IfMarksEqualT 80
\IfMarksEqualTF 80
\IfNoValue(TF) 6
\IfNoValueF 6
\IfNoValueT 6
\IfNoValueTF 6
\ifnum 74
\IfPropertyExistsF 92
\IfPropertyExistsT 92
\IfPropertyExistsTF 91
\IfPropertyRecordedF 92
\IfPropertyRecordedT 92
\IfPropertyRecordedTF 92
\IfSocketExistsTF 103
\IfSocketPlugAssignedTF 103
\IfSocketPlugExistsTF 103
\IfValue(TF) 6
\IfValueF 6
\IfValueT 6
\IfValueTF 6
\ignorespaces 44
\immediate 45
include (hook) 38, 51
\include 43
include/.../after 51
include/.../before 51
include/.../end 51
include/⟨name⟩/after (hook) 51
include/⟨name⟩/before (hook) 51
include/⟨name⟩/end (hook) 51
include/⟨name⟩/excluded (hook) . . 51, 51
include/after (hook) 51
include/after 51
include/before (hook) 51
include/before 51
include/end (hook) 51
include/end 51
include/excluded (hook) 51, 51
\includeonly 51
\indent 65
\IndentBox 63
\index 79
\input 49
\insert 61
\InsertMark 80
insertmark (hook) 79, 47
insertmark 79
\InstanceValue 113
int commands:
\int_use:N 74
\l_tmpa_int 109
\item 60

K

\KeyValue 108

L

\label 92
label 93
\lastbox 68
\LastMark 80
\leftmark 78
\listfiles 45
\LoadPackageWithOptions 49
\LogHook 30
\LogSocket 103
\long 3

M

\marginpar 63
\mark 78
mark commands:
\mark_clear_structure:n 87
\mark_copy_structure:nn 86
\mark_debug_off: 83
\mark_debug_on: 83
\mark_get_marks_for_reinsertion:nNN 87

\mark_if_eq:nnnnnnTF 80
\mark_if_eq:nnnnTF 80
\mark_insert:nn 79
\mark_new_class:n 79
\mark_set_structure_to_err:n . . . 86
\mark_update_structure_from_-
material:nn 86
\mark_use_first:nn 80
\mark_use_last:nn 80
\mark_use_top:nn 80
mark internal commands:
__mark__update_dblcol_structures:
. 86
__mark_update_dblcol_structures:
. 87
__mark_update_singlecol_-
structures: 87
\markboth 84
\markright 84
mdseries (hook) 46
\mdseries 46
mdseries/defaults (hook) 46
\multicolumn 12
muskip commands:
\l_tmpa_muskip 109
myhook (hook) 22

N

\New... 4
\newcommand 14
\newcommand(*) 14
\NewCommandCopy 57
\NewDocumentCommand 4
\NewDocumentEnvironment 4
\newenvironment 42
\NewExpandableDocumentCommand 12
\NewHook 36
\NewHookPair 56
\NewHookWithArguments 38
\NewMarkClass 79
\newmarks 85
\NewMirroredHookPair 19
\NewMirroredHookPairWithArguments . . 19
\newpage 96
\NewProperty 92
\NewReversedHook 36
\NewReversedHookWithArguments 58
\NewSocket 102
\NewSocketPlug 98
\NewTemplateType 107
\noindent 60
noop (plug) 99, 101, 102
normalfont (hook) 46
\normalfont 46
\normalsize 23
\NoValue 2

O

\obeyedline 14
\omit 12
\OmitIndent 65
\openin 48

P

package (hook) 38
package/ (hook) 50
package/.../after 50
package/.../before 50
package/⟨*name*⟩/after (hook) 50
package/⟨*name*⟩/before (hook) 50
package/⟨*package name*⟩/after (hook) . 50
package/⟨*package name*⟩/before (hook) 50
package/after (hook) 50, 50
package/after 50
package/before (hook) 50, 50
package/before 50
page 93
pagenum 93
\pageref 93
pagetarget 93
\par 62
para commands:
\para_end: 64
\g_para_indent_box 65
\para_omit_indent: 65
\para_raw_end: 65
\para_raw_indent: 65
\para_raw_noindent: 65
para/after (hook) 63
para/after 63
para/before (hook) 63
para/before 63
para/begin (hook) 63, 65
para/begin 63
para/end (hook) 63, 64
para/end 63
\paracntvalue 66
\parbox 63
\parindent 60
\parskip 60
\pdfsavepos 94
Plugs:
identity 99, 102, 102, 102
noop 99, 101, 102
tagpdf 102
\PopDefaultHookLabel 26
pre commands:
pre_shipout_filter 73

\PreviousTotalPages 74
\ProcessedArgument 15
\ProcessList 10
\ProcessorA 9
\ProcessorB 9
property commands:
\property_gset:nnnn 90
\property_if_exist:nTF 91
\property_if_exist_p:n 91
\property_if_recorded:nnTF 91
\property_if_recorded:nTF 91
\property_if_recorded_p:n 91
\property_if_recorded_p:nn 91
\property_item:nn 90
\property_item:nnn 90
\property_new:nnnn 90
\property_record:nN 90
\property_record:nn 90
\property_ref:nn 91
\property_ref:nnn 90
\property_ref_undefined_warn: . . . 91
\property_ref_undefined_warn:n . . 91
\property_ref_undefined_warn:nn . 91
\protect 12
\Provide... 4
\ProvideDocumentCommand 4
\ProvideDocumentEnvironment 4
\ProvideExpandableDocumentCommand . . 12
\PushDefaultHookLabel 26
\put 76

R

\RawIndent 65
\RawNoindent 65
\RawParEnd 65
\RawShipout 72
\ReadonlyShipoutCounter 74
\RecordProperties 90
\ref 89
\RefProperty 90
\refstepcounter 59
\RefUndefinedWarn 93
\relax 61
\RemoveFromHook 22
\Renew... 4
\RenewDocumentCommand 3
\RenewDocumentEnvironment 4
\RenewExpandableDocumentCommand 12
\RequirePackage 26
\ReverseBoolean 10
\rightmark 84
rmfamily (hook) 46, 46, 46
\rmfamily 46
\Roman 74

S

\savepos 94
\section 8
\sectionmark 84
selectfont (hook) 47
\selectfont 46
\SetCurrentKnownTemplateKeys 110
\SetDefaultHookLabel 26
\SetKnownTemplateKeys 110
\SetProperty 92
\SetTemplateKeys 110
sffamily (hook) 46
\sffamily 46
shipout (hook) 71, 72, 72, 73, 73
\shipout 46
shipout commands:
\l_shipout_box 70
\l_shipout_box_dp_dim 70
\l_shipout_box_ht_dim 70
\l_shipout_box_ht_plus_dp_dim . . . 70
\l_shipout_box_wd_dim 70
\shipout_debug_off: 74
\shipout_debug_on: 74
\shipout_discard: 73
\g_shipout_readonly_int 74
\g_shipout_totalpage_int 74
\g_shipout_totalpages_int 74
shipout/... (hook) 70, 73
shipout/after (hook) 74, 70, 70, 70, 72, 72, 72, 74
shipout/after 71
shipout/background (hook) 75, 75, 76, 70, 71, 72, 73
shipout/background 71
shipout/before (hook) 74, 76, 76, 97, 70, 70, 70, 70, 71, 71, 71, 72, 72, 72, 73, 73, 73, 73, 74
shipout/before 71
shipout/firstpage (hook) 70, 70, 70, 71, 72, 72, 72
shipout/firstpage 71
shipout/foreground (hook) 75, 75, 76, 70, 71, 72, 73
shipout/foreground 71
shipout/lastpage (hook) 45, 70, 70, 72, 72, 72, 72, 72
shipout/lastpage 71
\ShipoutBox 76
\ShowHook 36
\ShowInstanceValues 116
\ShowSocket 103
\ShowTemplateCode 116
\ShowTemplateDefaults 116
\ShowTemplateInterface 116

\ShowTemplateVariables 116
skip commands:
\l_tmpa_skip 109
\small 23
socket commands:
\socket_assign_plug:nn 102
\socket_debug_off: 104
\socket_debug_on: 104
\socket_get_plug:nN 104
\socket_if_exist:nTF 103
\socket_if_plug_assigned:nnTF . . 103
\socket_if_plug_exist:nnTF 103
\socket_log:n 103
\socket_new:nn 102
\socket_new_plug:nnn 102
\socket_set_plug:nnn 102
\socket_show:n 103
\socket_use:n 103
\socket_use:nn 103
\socket_use:nnn 103
\socket_use:nnnn 103
\socket_use:nw 103
\socket_use_expandable:n 103
\socket_use_expandable:nw 103
Sockets:
foo 99, 99
\space 7
\special 72
\SplitArgument 9
\SplitList 10
\strut 47
\subsectionmark 84

T

tagpdf (plug) 102
target 93
test (hook) 39, 39
TEX and LATEX 2ε commands:
\@begindocumenthook 41
\@begindvi 76
\@begindvibox 71
\@bsphack 89
\@cclv 76
\@currentHpage 93
\@currentHref 93
\@currentcounter 93
\@currentlabel 93
\@currentlabelname 93
\@currenvir 66
\@esphack 92
\@firstofone 21
\@input 48
\@kernel@after@⟨hook⟩ 40
\@kernel@after@para@after 64
\@kernel@after@para@end 63
\@kernel@before@⟨hook⟩ 40
\@kernel@before@para@before 63
\@kernel@before@para@begin 64
\@opcol 87
\@outputbox 87
\@@end 45
\AddToHook 54
\AddToHookNext 54
\AddToHookNextWithArguments 54
\AddToHookWithArguments 54
\apptocmd 55
\botmark 85
\declare@file@substitution 52
\DeclareRobustCommand 56
\def 55
\disable@package@load 52
\expand@font@defaults 46
\if@firstcolumn 87
\input@path 49
\new@label@record 92
\newcommand 56
\NewDocumentCommand 56
\on@line 66
\patchcmd 56
\pretocmd 55
\protected@edef 84
\reenable@package@load 52
\section 57
\topmark 85
\topmark(s) 85
\undeclare@file@substitution . . . 52
\unexpanded 80
tex commands:
\tex_savepos:D 89
\thepage 93
title 93
tl commands:
\l_tmpa_tl 109
toplevel (hook) 22, 22, 41
\TopMark 86
\topmark 78
totalpages 74
\TrimSpaces 10
ttfamily (hook) 46
\ttfamily 46
\typeout 72
\typesetnormalchapter 4

U

\unitlength 71
\unpenalty 68
\unskip 68
\UnusedTemplateKeys 111

\unvcopy 86
use commands:
\use_i:nn 109
\UseHook 20
\UseHookWithArguments 58
\UseInstance 113
\UseOneTimeHook 20
\UseOneTimeHookWithArguments 21
\usepackage 25
\UseSocket 104
\UseTemplate 114
\usetikzlibrary 26

V

\vadjust 63
\vbox 69
\verb 13
\vskip 61

W

\write 72

X

xpos 94

Y

ypos 94